XCPC 算法模板

usedchang



2026 年 10 月 22 日

目录

1 杂项 **4**

1.1 IO 加速 4
1.2 ___int128 操作 4
1.3 对拍 4
1.4 热身赛测试语句 5
1.5 编译指令 6
1.6 pbds：平衡树 (名次树) 6
1.7 pbds：哈希表 7
1.8 pbds：堆 (可并堆、可修改) . . . 8
1.9 rope：块状链表 8
1.10 字典树 (pbds trie / map 版) . . . 9
1.11 cpp 扩栈 (win) 9

2 数据结构 **10**

2.1 ST 表 10
2.2 树状数组 10
2.3 并查集 11
2.4 线段树 (区间加 + 区间和) 13
2.5 可持久化线段树 (主席树) 14
2.6 笛卡尔树 15
2.7 字典树 16
2.8 平衡树 17
2.9 根号分治 18
2.10 无向图三元环计数 19
2.11 莫队 20

3 树 **21**

3.1 LCA 21
3.2 树的直径 22
3.3 树链剖分 23
3.4 树上启发式合并 24

4 图论 **26**

4.1 拓扑排序与 0-1 BFS 26
4.2 SPFA 26
4.3 Dijkstra 27
4.4 最小生成树 28
4.5 Kruskal 重构树 29
4.6 欧拉回路 29
4.7 二分图 30
4.7.1 二分图染色 30
4.7.2 二分图最大匹配 (匈牙利算法) 31
4.7.3 二分图最大匹配 (Dinic) . 31
4.7.4 二分图最大权匹配 (KM) . 32
4.8 最大流与最小费用最大流 33
4.9 差分约束 35
4.10 缩点 35
4.10.1 SCC(强连通分量) 35
4.10.2 DECC(边双联通分量) . . 36
4.11 two-sat 36

5 字符串 **38**

5.1 字符串哈希 38
5.2 AC 自动机 38
5.3 KMP 39
5.4 Z 函数 (扩展 KMP) 40
5.5 Manacher(马拉车) 40
5.6 回文自动机 (PAM) 41
5.7 后缀数组与 LCP 41
5.8 后缀自动机 42

6 动态规划 **44**

6.1 01 背包 44
6.2 完全背包 44
6.3 多重背包 44
6.4 树上背包 45
6.5 换根 dp 46
6.6 区间 dp 46
6.7 SOS dp 47
6.8 数位 dp 47

7 数学 **49**

7.1 筛法求因数个数 49
7.2 筛法求欧拉函数 49
7.3 筛法求莫比乌斯函数 49
7.4 矩阵快速幂 50
7.5 第二类斯特林数 50
7.6 卡特兰数 51
7.7 排列组合类-含 Lucas 51
7.8 常见计数恒等式与幂转换 51

7.9 高斯消元 53
7.10 FFT 53
7.11 NTT 54
7.12 多项式乘幂级数的分治求和 . . . 55
7.13 线性递推 (Berlekamp-Massey) . . 56
7.14 Pollard-Rho 57
7.15 异或线性基 58
7.16 数论分块 58
7.17 莫比乌斯反演 59
7.18 exgcd 60
7.19 exgcd 求逆元与线性同余 61
7.20 exBSGS 61
7.21 博弈论与 SG 打表 62

8 计算几何 63
8.1 基础定义 63
8.2 点与向量 63
8.3 直线 64
8.4 线段 65
8.5 三角形 65
8.6 圆 66
8.7 多边形 67
8.8 球 67
8.9 极角排序 68
8.10 凸包与旋转卡壳 69

9 Other Trick 71
9.1 多路归并 (含二分解) 71
9.2 使用栈做递归防溢出 73
9.3 Log Trick 73
9.4 异或前缀模 4 小规律 75
9.5 高阶差分下等差求通项 75
9.6 曼哈顿距离与切比雪夫距离的相互转化 75
9.7 判定可重集合是否相等 76

1 杂项

1.1 IO 加速

Listing 1: 快速读入模板

```cpp
#include <bits/stdc++.h>
using namespace std;

// ======================
通用快读（支持任意整数类型）======================
template<typename T>
inline T read() {
    T x = 0; int f = 1; char ch = getchar();
    while (ch < '0' || ch > '9') { if (ch == '-')
f = -1; ch = getchar(); }
    while (ch >= '0' && ch <= '9') { x = x * 10 +
(ch - '0'); ch = getchar(); }
    return x * f;
}

// ======================
通用快写（支持任意整数类型）======================
template<typename T>
inline void write(T x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) write(x / 10);
    putchar(x % 10 + '0');
}

// ====================== 使用示例 ================
======
int main() {
    int a = read<int>();
    long long b = read<long long>();
    short c = read<short>();
    __int128 d = read<__int128>();   // 支持
__int128!

    write(a); putchar('\n');
    write(b); putchar(' ');
    write(c); putchar('\n');
    write(d); putchar('\n');         // __int128
直接输出完美
    return 0;
}
```

1.2 ___int128 操作

```cpp
#include <iostream>
#include <cctype>
using namespace std;

// 重载 >> 运算符
istream& operator>>(istream &is, __int128 &n) {
    n = 0;
    int f = 1;
    char ch = is.get();
    while (!isdigit(ch)) {
        if (ch == '-') f = -1;
        ch = is.get();
    }
    while (isdigit(ch)) {
        n = n * 10 + (ch - '0');
        ch = is.get();
    }
    n *= f;
    return is;
}

// 重载 << 运算符
ostream& operator<<(ostream &os, __int128 n) {
    if (n < 0) {
        os << '-';
        n = -n;
    }
    if (n > 9) os << n / 10;
    os << (char)(n % 10 + '0');
    return os;
}

// 绝对值函数
__int128 abs128(__int128 x) {
    return x < 0 ? -x : x;
}

// 整数平方根（向下取整）
__int128 sqrt128(__int128 x) {
    if (x < 0) return -1;
    __int128 l = 0, r = 1;
    while (r <= x / r) r *= 2; // 避免溢出
    while (l < r) {
        __int128 mid = (l + r + 1) / 2;
        if (mid <= x / mid) l = mid;
        else r = mid - 1;
    }
    return l;
}

int main() {
    __int128 a, b;
    cin >> a >> b;
    cout << "a = " << a << ", b = " << b << endl;
    cout << "abs(a) = " << abs128(a) << endl;
    cout << "sqrt(b) = " << sqrt128(b) << endl;
    return 0;
}
```

1.3 对拍

Listing 2: 对拍 windows

```cpp
#include<bits/stdc++.h>
using namespace std;
int main(){
    //一直循环，直到找到不一样的数据
```

```
    while (1) {
        system("data.exe > data.txt");
        system("bf.exe < data.txt>bf.txt");
        system("test.exe < data.txt > test.txt");
        if (system("fc test.txt bf.txt")) break;
        //不一样就跳出循环
    }//当 fc 返回 1 时，说明这时数据不一样
    return 0;
}
```

Listing 3: 对拍 linux

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    //一直循环，直到找到不一样的数据
    while (1) {
        system("./data > data.txt");
        system("./bf < data.txt > bf.txt");
        system("./test < data.txt > test.txt");
        if (system("diff test.txt bf.txt")) break;
        //不一样就跳出循环
    }//当 diff 返回非0 时，说明这时数据不一样
    return 0;
}
```

1.4 热身赛测试语句

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
void test_speed() {
    long long cnt = 0;
    auto start =
chrono::high_resolution_clock::now();
    for(int i = 0; i < 1000000000; ++i) cnt += i;
// 10^9 次加法
    auto end =
chrono::high_resolution_clock::now();
    chrono::duration<double> diff = end - start;
    cout << "10^9 additions: " << diff.count() <<
"s" << endl;
}
volatile ll sink = 0;// 防止编译器优化：将结果写入
volatile 变量
void test_time() {
    const int batch = 100000; // 每批执行 10
万次运算，减少时间检查开销
    ll add_cnt = 0, mul_cnt = 0, mod_cnt = 0;

    // ========== 1. 测试加法 ==========
    {
        ll v = 0;
        auto start =
chrono::high_resolution_clock::now();
        while (true) {
            // 执行一批运算
            for (int i = 0; i < batch; ++i) v += 1;
            add_cnt += batch;

            // 检查时间
            auto now =
chrono::high_resolution_clock::now();
            if
(chrono::duration_cast<chrono::seconds>(now -
start).count() >= 1)
                break;
        }
        sink = v; // 确保 v 被使用，防止优化
    }

    // ========== 2. 测试乘法 ==========
    {
        ll v = 1;
        auto start =
chrono::high_resolution_clock::now();
        while (true) {
            for (int i = 0; i < batch; ++i) {
                v *= 2;
                if (v > 1e18) v = 1; // 防止溢出
            }
            mul_cnt += batch;

            auto now =
chrono::high_resolution_clock::now();
            if
(chrono::duration_cast<chrono::seconds>(now -
start).count() >= 1)
                break;
        }
        sink = v;
    }

    // ========== 3. 测试取模 ==========
    {
        const ll MOD = 1e9+7;
        ll v = 0;
        auto start =
chrono::high_resolution_clock::now();
        while (true) {
            for (int i = 0; i < batch; ++i) v = (v
+ 1) % MOD;
            mod_cnt += batch;

            auto now =
chrono::high_resolution_clock::now();
            if
(chrono::duration_cast<chrono::seconds>(now -
start).count() >= 1)
                break;
        }
        sink = v;
    }
    cout << "1s add nums " << add_cnt << endl;
    cout << "1s mul nums " << mul_cnt << endl;
    cout << "1s mod&add nums " << mod_cnt << endl;
}
```

```cpp
int main() {
    // 1. 测试编译器版本
    #ifdef __GNUC__
        cout << "GCC Version: " << __GNUC__ << "."
<< __GNUC_MINOR__ << endl;
    #endif
    // 2. 测试 __int128
    #ifdef __SIZEOF_INT128__
        cout << "__int128 is supported" << endl;
    #endif
    test_speed();// 3. 测试速度
    test_time(); // 4. 测试各运算1秒内执行次数
    return 0;
}
```

Listing 4: 随机数

```cpp
#include<bits/stdc++.h>
using namespace std;
mt19937 rng;                    // 32 位梅森旋转生成器
mt19937_64 rng64;               // 64 位梅森旋转生成器
struct Random {
    // 构造函数：用随机设备 + 时间戳初始化种子
    Random() {
        auto seed = random_device{}();
        auto time_seed = chrono::steady_clock::now
().time_since_epoch().count();
        rng.seed(seed ^ time_seed);
        rng64.seed(seed ^ time_seed);
    }
    template<typename T>
    T randInt(T l, T r) {
    //static_assert(is_integral<T>::value, "T must
be an integral type");
        uniform_int_distribution<T> dist(l, r);
        return dist(rng);
    }// 生成 [l, r] 范围内的随机整数（支持 int/long
long/short 等）
    template<typename T>
    T randDouble(T l, T r) {
    //static_assert(is_floating_point<T>::value, "T
 must be a floating point type");
        uniform_real_distribution<T> dist(l, r);
        return dist(rng);
    }// 生成 [l, r) 范围内的随机浮点数（支持 float/
double/long double）
};
// ---------------- 示例用法 ----------------
int main() {
    Random rnd;  // 全局或局部声明均可
    int a = rnd.randInt(1, 100);          // [1,
100]
    long long b = rnd.randInt(0LL, 114514LL); //
[0, 114514]
    double c = rnd.randDouble(0.0, 1e18);  // [0.0,
 1e18)
    std::vector<int> vec = {1, 2, 3, 4, 5};
    shuffle(vec.begin(), vec.end(),rng);
    cout<<"a="<<a<<endl;
    cout<<"b="<<b<<endl;
    cout<<fixed<<setprecision(10)<<c<<endl;
    cout<<"vec:=";
    for(int x:vec) cout<<x<<' ';
    return 0;
}
```

1.5 编译指令

常用选项：-Wall / -Wextra 打开警告，-O2 二级优化（竞赛最常用），-o a.exe 指定输出文件名（Linux 下写 -o a）；末尾的 && a.exe 表示编译成功就立刻运行。

Listing 5: 编译指令

```cpp
// 编译指令：各选项含义见正文。
g++ -std=c++20 -Wall -Wextra -O2 a.cpp -o a.exe &&
a.exe//编译程序并运行
```

Listing 6: .bat/.cmd (win) 批处理文件编译

```cpp
@echo off
g++ -std=c++17 -O2 -Wall -o sol.exe sol.cpp &&
sol.exe
pause
```

Listing 7: .sh (linux) 文件编译

```cpp
#!/bin/bash
g++ -std=c++17 -O2 -Wall -o sol sol.cpp && ./sol
```

1.6 pbds：平衡树 (名次树)

pbds 的 tree：order_of_key 查排名、find_by_order 查第 k 小，还有 lower_bound / upper_bound，比手写平衡树省事。可重集合（multiset）把键换成 pair< 值，唯一编号 > 保证键唯一，此时”值 $\le x$ 的个数”用 order_of_key({x, 无穷大})，”值 $\ge x$ 的个数”用 size() 减去 order_of_key({x, 0})。头文件 <ext/pb_ds/assoc_container.hpp> 与 <ext/pb_ds/tree_policy.hpp>。

Listing 8: pbds 平衡树 (名次树)

```cpp
// pbds 名次树（tree +
tree_order_statistics_node_update）：用法与可重集合
技巧见正文。
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

// tree<键, 映射, 比较器, 容器标签, 更新策略>
//   null_type            —— 当 set 用（不用映射）
```

```cpp
//   rb_tree_tag            —— 红黑树（另有 splay_tree_tag / ov_tree_tag）
//   tree_order_statistics_node_update —— 提供 order_of_key / find_by_order
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update> ordered_set;

int main() {
    ordered_set t;
    t.insert(3); t.insert(1); t.insert(4); t.insert(1);   // 等价于 set，重复插入无效
    // order_of_key(x)：严格小于 x 的元素个数（0-indexed 排名）
    cout << t.order_of_key(3) << '\n';
       // 1
    // find_by_order(k)：第 k 小（k 从 0 计数），越界返回 end()
    cout << *t.find_by_order(0) << '\n';
       // 1
    cout << *t.find_by_order(2) << '\n';
       // 4
    // 前驱后继：lower_bound(x) 是 >= x，upper_bound(x) 是 > x
    cout << *t.lower_bound(3) << ' ' << *t.upper_bound(3) << '\n';   // 3 4
    auto it = t.find_by_order(t.order_of_key(4));
       // 4 的位置
    cout << *prev(it) << '\n';
       // 3（4 的前驱）
    t.erase(4);
    cout << (t.find(4) == t.end()) << '\n';
       // 1
    cout << t.size() << '\n';
       // 2

    // 可重集合（multiset）：键改成 pair<值，唯一编号> 保证键唯一；
    // 查询"值 <= x 的个数"用 order_of_key({x, 无穷大})，"值 >= x 的个数"用 size - order_of_key({x, 0})
    tree<pair<int, int>, null_type, less<pair<int, int>>, rb_tree_tag,
         tree_order_statistics_node_update> ms;
    int id = 0;
    ms.insert({5, id++});
    ms.insert({5, id++});
    ms.insert({7, id++});
    cout << ms.order_of_key({5, INT_MAX}) << '\n';
       // 2（值 <= 5 的个数）
    cout << ms.size() - ms.order_of_key({5, 0}) << '\n';  // 3（值 >= 5 的个数）
    return 0;
}
```

1.7 pbds：哈希表

gp_hash_table（探测法）/ cc_hash_table（拉链法），比 unordered_map 快，还能塞自定义哈希防被卡。头文件 <ext/pb_ds/assoc_container.hpp> 与 <ext/pb_ds/hash_policy.hpp>。

Listing 9: pbds 哈希表

```cpp
// pbds 哈希表 gp_hash_table / cc_hash_table：用法见正文。
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/hash_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

// 防卡的自定义哈希：把 key 混入随机数再走 splitmix64
struct chash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

int main() {
    // gp_hash_table<键，值，哈希>；第三个参数不写就是 default_hash<键>
    gp_hash_table<long long, int, chash> mp;
    mp[123] = 1;
    mp[456] = 2;
    cout << mp[123] + mp[456] << '\n';
    // 3
    cout << (mp.find(999) == mp.end()) << '\n';
    // 1
    mp.erase(123);
    cout << mp.size() << '\n';
    // 1
    // 当作集合用：映射写成 null_type
    gp_hash_table<int, null_type, chash> st;
    st.insert(7);
    cout << (st.find(7) != st.end()) << '\n';
    // 1
    // 多组数据复用：clear() 比重新构造快（内部容量会保留）
    mp.clear();
```

```
    cout << mp.size() << '\n';
    // 0
    // 说明：cc_hash_table<键，值，哈希> 是拉链法版本，用法完全一样；
    //       键是 string 这类非整数时，自定义哈希要按实际键类型写
    return 0;
}
```

1.8 pbds：堆 (可并堆、可修改)

priority_queue 的配对堆版本：支持 join 合并两个堆、按句柄 erase / modify。头文件 <ext/pb_ds/priority_queue.hpp>；名字与 std::priority_queue 撞车，要写全命名空间。

Listing 10: pbds 堆 (可并堆/可修改)

```
// pbds 堆（可并堆、可按句柄改/删）：pairing_heap_tag；用法见正文。
#include <bits/stdc++.h>
#include <ext/pb_ds/priority_queue.hpp>
using namespace std;
using namespace __gnu_pbds;

// 容器标签：pairing_heap_tag（配对堆，最常用，支持 modify/erase）
//           binary_heap_tag、binomial_heap_tag、rc_binomial_heap_tag、thin_heap_tag
int main() {
    // 注意：std 和 __gnu_pbds 里都有 priority_queue，同时 using 两个命名空间时
    //       必须写全 __gnu_pbds::priority_queue，否则报 ambiguous
    __gnu_pbds::priority_queue<int, less<int>, pairing_heap_tag> pq, pq2;   // less = 大根堆
    pq.push(3);
    pq.push(1);
    pq.push(5);
    auto it = pq.push(4);               // push 返回迭代器（句柄），后面用它删除/修改
    cout << pq.top() << '\n';           // 5
    pq.modify(it, 9);                   // 把 4 改成 9（O(1) 摊还）
    cout << pq.top() << ' ' << pq.size() << '\n';  // 9 4
    pq.erase(it);                       // 按句柄删除
    cout << pq.size() << '\n';          // 3

    pq2.push(7);
    pq2.push(2);
    pq.join(pq2);                       // 合并两个堆（pq2 会被清空），O(1) 摊还
    cout << pq.size() << ' ' << pq2.size() << '\n';  // 5 0
    while (!pq.empty()) {               // 非清空式遍历也可以：for (auto x : pq)
        cout << pq.top() << ' ';
        pq.pop();
    }
    cout << '\n';                       // 7 5 3 2 1
    return 0;
}
```

1.9 rope：块状链表

GNU 扩展 rope：$O(\log n)$ 插入、删除、区间提取与拼接，赋值是 $O(1)$ 的写时复制，所以也能当可持久化序列用。头文件 <ext/rope>，命名空间是 __gnu_cxx（不是 __gnu_pbds）。

Listing 11: rope 块状链表

```
// GNU 扩展 rope（块状链表 / 可持久化序列）：用法见正文。
#include <bits/stdc++.h>
#include <ext/rope>
using namespace std;
using namespace __gnu_cxx;

int main() {
    rope<int> a, b;
    for (int i = 1; i <= 5; ++i) a.push_back(i);
    // 1 2 3 4 5
    a.insert(1, 99);
    // 在下标 1 前插入 -> 1 99 2 3 4 5
    a.erase(1, 1);
    // 从下标 1 起删 1 个 -> 1 2 3 4 5
    rope<int> mid = a.substr(1, 3);
    // 取出 [1, 1+3) -> 2 3 4
    b = a;
    // O(1) 复制，底层共享
    b.append(mid);
    // 1 2 3 4 5 2 3 4
    cout << a.size() << ' ' << b.size() << '\n';
    // 5 8
    for (int i = 0; i < (int)b.size(); ++i) cout << b[i] << " \n"[i + 1 == (int)b.size()];
    a += b;
    // 拼接，a 变成 13 个元素
    cout << a.size() << '\n';
    // 13
    cout << a[0] << ' ' << a[12] << '\n';
    // 1 4（随机下标访问是 O(log n)）

    // 字符序列同理：rope<char> s; 或直接写 crope
    crope s;
    s.push_back('a');
    s += "bc";
    s.insert(1, "XY");
    // a X Y b c
    cout << s.size() << ' ' << s[1] << s[2] << '\n';  // 5 XY
    return 0;
}
```

1.10 字典树 (pbds trie / map 版)

pbds 的 PATRICIA trie（pat_trie_tag 配 trie_prefix_search_node_update）提供 prefix_range 做前缀搜索；字符集大、节点稀疏时可以用 std::map 动态开点写法省内存。头文件 <ext/pb_ds/assoc_container.hpp>、<ext/pb_ds/trie_policy.hpp> 与 <map>。

Listing 12: 字典树 (pbds trie 与 map 版)

```cpp
// 字典树：pbds PATRICIA trie（前缀搜索）与 std::map 动态开点两种写法；说明见正文。
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/trie_policy.hpp>
#include <map>
using namespace std;
using namespace __gnu_pbds;

// <键，映射，键访问特性，容器标签，更新策略>
typedef trie<string, null_type, trie_string_access_traits<>, pat_trie_tag,
             trie_prefix_search_node_update> Ptrie;

int main() {
    Ptrie t;
    t.insert("apple");
    t.insert("apply");
    t.insert("app");
    t.insert("banana");

    // 前缀搜索：遍历所有以 "app" 开头的键（按字典序）
    auto range = t.prefix_range("app");
    for (auto it = range.first; it != range.second; ++it) cout << *it << '\n';
    // app / apple / apply
    cout << t.size() << '\n';
    // 4
    cout << (t.find("banana") != t.end()) << '\n';
    // 1
    t.erase("app");
    cout << t.size() << '\n';
    // 3

    // —— 字符集大 / 节点稀疏时：std::map 动态开点 trie（每个节点只存实际用到的边）
    //    比 26/62 叉定长数组省内存，代价是一次转移 O(log 字符集)
    vector<map<int, int>> nxt(1);
    // nxt[p][c] = 子节点编号
    vector<int> cnt(1, 0);
    // 经过该节点的串数
    auto insert = [&](const string& s) {
        int p = 0;
        for (char c : s) {
            if (!nxt[p].count(c)) {
                nxt.emplace_back();
                cnt.push_back(0);
                nxt[p][c] = (int)nxt.size() - 1;
            }
            p = nxt[p][c];
            ++cnt[p];
        }
    };
    insert("abc");
    insert("abd");
    insert("abc");
    // 查询：以 "ab" 为前缀的串有多少个
    int p = 0;
    for (char c : string("ab")) p = nxt[p].count(c) ? nxt[p][c] : -1;
    if (p < 0) p = 0;
    cout << cnt[p] << '\n';
    // 3
    return 0;
}
```

1.11 cpp 扩栈 (win)

Listing 13: 扩栈指令

```cpp
int size(256<<20);
__asm__ ( "movq %0, %%rsp\n"::"r"((char*)malloc(size)+size));
/*
主程序
*/
exit(0);
```

2 数据结构

2.1 ST 表

Listing 14: ST 表

```cpp
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const ll INF=4e18;
struct ST{
    ST(){};
    vector<vector<ll>>mx,mn;
    ST(const vector<ll>&a){
        int n=a.size()-1;
        int LOG=n>0?__lg(n)+1:1;   // 列数按 n 的实际对数取，避免写越界
        mx=vector<vector<ll>>(n+1,vector<ll>(LOG,-INF));
        mn=vector<vector<ll>>(n+1,vector<ll>(LOG,INF));
        for(int i=1;i<=n;i++) mx[i][0]=mn[i][0]=a[i];
        for(int i=1;i<LOG;i++){
            for(int j=1;j+(1<<i)-1<=n;j++){
                mx[j][i]=(mx[j][i-1]>mx[j+(1<<i-1)][i-1])?mx[j][i-1]:mx[j+(1<<i-1)][i-1];
                mn[j][i]=(mn[j][i-1]<mn[j+(1<<i-1)][i-1])?mn[j][i-1]:mn[j+(1<<i-1)][i-1];
            }
        }
    };
    ll qry_mx(int l,int r){
        int len=__lg(r-l+1);//31-__builtin_clz(r-l+1)
        return (mx[l][len]>mx[r-(1<<len)+1][len])?mx[l][len]:mx[r-(1<<len)+1][len];
    }
    ll qry_mn(int l,int r){
        int len=__lg(r-l+1);//31-__builtin_clz(r-l+1)
        return (mn[l][len]<mn[r-(1<<len)+1][len])?mn[l][len]:mn[r-(1<<len)+1][len];
    }
};
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n,q,l,r;cin>>n>>q;
    vector<ll>a(n+1);
    for(int i=1;i<=n;i++) cin>>a[i];
    ST st(a);
    while(q--){
        cin>>l>>r;
        cout<<st.qry_mx(l,r)<<endl;
    }
    return 0;
}
```

2.2 树状数组

所有下标均从 0 开始，区间按左闭右开 $[l,r)$ 表示。BIT<T> 支持单点加与区间和，BITR<T> 支持区间加与区间和，BIT2D<T> 支持二维单点加与矩形和；一维操作为 $O(\log n)$，二维操作为 $O(\log n \log m)$。

Listing 15: 一维、区间更新与二维树状数组

```cpp
#include <bits/stdc++.h>
using namespace std;

// 0-indexed 单点加、半开区间 [l,r) 求和
template <class T>
struct BIT {
    int n;
    vector<T> tr;

    explicit BIT(int n = 0) : n(n), tr(n + 1, T{}) {}

    void add(int i, T v) {
        for (++i; i <= n; i += i & -i) {
            tr[i] += v;
        }
    }

    T sum(int r) const {
        T res{};
        for (; r > 0; r -= r & -r) {
            res += tr[r];
        }
        return res;
    }

    T query(int l, int r) const {
        return sum(r) - sum(l);
    }
};

// 0-indexed 区间加、区间和
template <class T>
struct BITR {
    int n;
    BIT<T> d, c;

    explicit BITR(int n = 0) : n(n), d(n + 1), c(n + 1) {}

    void add(int l, int r, T v) {
        d.add(l, v);
        d.add(r, -v);
        c.add(l, v * l);
        c.add(r, -v * r);
    }

```

```
    T sum(int r) const {
        return d.sum(r) * r - c.sum(r);
    }

    T query(int l, int r) const {
        return sum(r) - sum(l);
    }
};

// 0-indexed 二维单点加、半开矩形 [x1,x2) x [y1,y2) 求和
template <class T>
struct BIT2D {
    int n, m;
    vector<vector<T>> tr;

    BIT2D(int n, int m) : n(n), m(m), tr(n + 1, vector<T>(m + 1, T{})) {}

    void add(int x, int y, T v) {
        for (int i = x + 1; i <= n; i += i & -i) {
            for (int j = y + 1; j <= m; j += j & -j) {
                tr[i][j] += v;
            }
        }
    }

    T sum(int x, int y) const {
        T res{};
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                res += tr[i][j];
            }
        }
        return res;
    }

    T query(int x1, int y1, int x2, int y2) const {
        return sum(x2, y2) - sum(x1, y2)
             - sum(x2, y1) + sum(x1, y1);
    }
};
```

2.3 并查集

Listing 16: 普通并查集

```
struct DSU{
    vector<int>f;
    vector<int>siz;
    DSU(){};
    DSU(int n){
        f.resize(n+2);          // n+1 作为 nxt 的哨兵
        iota(f.begin(),f.end(),0);
        siz.assign(n+2,1);
    }
    int find(int x){
        return x==f[x]?x:f[x]=find(f[x]);
    }
    bool merge(int x,int y){
        int fx=find(x),fy=find(y);
        if(fx==fy) return false;
        f[fx]=fy;
        siz[fy]+=siz[fx];
        return true;
    }
    int nxt(int l,int r){
        int res=0;
        l=find(l);
        while(l<=r){
            ++res;
            l=f[l]=find(l+1);
        }
        return res;

}//l需要多少步跳到r? 模拟链表的模式删除l-r所有中转点
    bool same(int x,int y){
        return find(x)==find(y);
    }
};
```

Listing 17: weighted-dsu

```cpp
//P1196银河英雄传说
struct DSU{
    DSU(){}
    vector<int>f,siz;
    vector<ll>dis;
    DSU(int n){
        dis.resize(n+1);
        f.resize(n+1);
        iota(f.begin(),f.end(),0);
        siz.assign(n+1,1);
    }
    int find(int x){
        int fx=f[x];
        if(fx==x) return x;
        int rt=find(f[x]);
        dis[x]+=dis[f[x]];
        return f[x]=rt;
    }
    bool merge(int x,int y){
        int fx=find(x),fy=find(y);
        if(fx==fy) return false;
        f[fx]=fy;
        dis[fx]+=siz[fy];
        siz[fy]+=siz[fx];
        return true;
    }
    bool same(int x,int y){
        return find(x)==find(y);
    }
    ll Dis(int x,int y){
        return dis[x]-dis[y];
    }
};
void solve(){
    int T;cin>>T;
    DSU dsu(30000);   // P1196 的飞船编号固定为 1..30000，T 只是操作数
    for(int i=1;i<=T;i++){
        char c;int x,y;cin>>c>>x>>y;
        if(c=='M') dsu.merge(x,y);
        else{
            if(dsu.same(x,y))
cout<<abs(dsu.Dis(x,y))-1<<endl;
            else cout<<-1<<endl;
        }
    }
}
```

Listing 18: 可撤销并查集

```cpp
struct DSU {
    vector<int> f, siz;
    struct Record {
        int smallRoot;
        int bigRoot;
        int oldSizeBig;
    };
    vector<Record> st;
    DSU() {}
    DSU(int n) {
        f.resize(n);
        siz.assign(n, 1);
        iota(f.begin(), f.end(), 0);
    }
    int find(int x) const {
        while (f[x] != x) x = f[x];
        return x;
    }
    bool merge(int x, int y) {
        int fx = find(x), fy = find(y);
        if (fx == fy) return false;
        if (siz[fx] > siz[fy]) swap(fx, fy);
        st.push_back({fx, fy, siz[fy]});
        f[fx] = fy;
        siz[fy] += siz[fx];
        return true;
    }
    void undo() {
        auto [small, big, oldSz] = st.back();
        st.pop_back();
        f[small] = small;
        siz[big] = oldSz;
    }
    size_t save() {
        return st.size();
    }
    void rollback(size_t checkpoint) {
        while (st.size() > checkpoint) undo();
    }
    int size(int x) {
        return siz[find(x)];
    }
};
```

可持久化并查集：用主席树维护 `fa` 与秩，按秩合并（**不能路径压缩**，压缩会改写历史版本），支持合并、回到历史版本、询问连通性。输入格式（P3402）：`1 a b` 合并并产生新版本；`2 k` 回到第 k 次操作之后的状态；`3 a b` 询问是否同集合（输出 1/0）。单次操作 $O(\log^2 n)$——沿 `fa` 走 $O(\log n)$ 层，每层查询 $O(\log n)$。

Listing 19: 可持久化并查集

```cpp
#include <bits/stdc++.h>
using namespace std;

// 可持久化并查集：主席树上维护 fa 与秩，按秩合并、不路径压缩；
// 输入格式与用法见正文。
const int N = 2e5 + 5;
int n, m, tot;
int ls[N * 25], rs[N * 25], fa[N * 25], dep[N * 25];
```

```
int root[N];                    // root[i] = 版本 i 的根（版本 0 是初始状态）

void build(int &p, int l, int r) {
    p = ++tot;
    if (l == r) { fa[p] = l; dep[p] = 1; return; }
    int mid = (l + r) >> 1;
    build(ls[p], l, mid);
    build(rs[p], mid + 1, r);
}
/* 把位置 pos 的 (fa, dep) 改成 (nf, nd)，返回新根 */
void mod(int &p, int pre, int l, int r, int pos, int nf, int nd) {
    p = ++tot;
    ls[p] = ls[pre];
    rs[p] = rs[pre];
    if (l == r) { fa[p] = nf; dep[p] = nd; return; }
    int mid = (l + r) >> 1;
    if (pos <= mid) mod(ls[p], ls[pre], l, mid, pos, nf, nd);
    else            mod(rs[p], rs[pre], mid + 1, r, pos, nf, nd);
}
int qfa(int p, int l, int r, int pos) {
    while (l < r) {
        int mid = (l + r) >> 1;
        if (pos <= mid) { p = ls[p]; r = mid; }
        else { p = rs[p]; l = mid + 1; }
    }
    return fa[p];
}
int qdep(int p, int l, int r, int pos) {
    while (l < r) {
        int mid = (l + r) >> 1;
        if (pos <= mid) { p = ls[p]; r = mid; }
        else { p = rs[p]; l = mid + 1; }
    }
    return dep[p];
}
int find(int rt, int x) {                    // 不压缩路径，按秩合并保证 O(log n) 层
    int f = qfa(rt, 1, n, x);
    while (f != x) { x = f; f = qfa(rt, 1, n, x); }
    return x;
}

int main() {
    scanf("%d %d", &n, &m);
    build(root[0], 1, n);
    for (int i = 1; i <= m; ++i) {
        int op;
        scanf("%d", &op);
        if (op == 1) {
            int a, b;
            scanf("%d %d", &a, &b);
            int ra = find(root[i - 1], a), rb = find(root[i - 1], b);
            if (ra == rb) { root[i] = root[i - 1]; continue; }
            int da = qdep(root[i - 1], 1, n, ra), db = qdep(root[i - 1], 1, n, rb);
            if (da < db) { swap(ra, rb); swap(da, db); }   // 让 ra 的秩不小于 rb
            int t;
            mod(t, root[i - 1], 1, n, rb, ra, db);     // rb 的父亲改成 ra
            if (da == db) {
                // 秩相同：ra 的秩 +1
                int t2;
                mod(t2, t, 1, n, ra, ra, da + 1);
                root[i] = t2;
            } else {
                root[i] = t;
            }
        } else if (op == 2) {
            int k;
            scanf("%d", &k);
            root[i] = root[k];
        // 回到版本 k
        } else {
            int a, b;
            scanf("%d %d", &a, &b);
            root[i] = root[i - 1];
        // 询问不产生新版本
            printf("%d\n", find(root[i], a) == find(root[i], b) ? 1 : 0);
        }
    }
    return 0;
}
```

2.4 线段树 (区间加 + 区间和)

处理下标区间 $[0, n]$：先准备好 `ll` 的 typedef 和初始数组 `a[]`（`a[l]` 是叶子 l 的初值），再 `Seg seg(n);` 建树，`seg.add(l, r, k, 1);` 区间加、`seg.qry(l, r, 1);` 区间求和。

Listing 20: common-Seg

```
// 线段树：区间加 + 区间和（依赖外部的 ll 与初始数组 a[]）；用法见正文。
struct Seg{
    #define lc (p<<1)
    #define rc (p<<1|1)
    struct S{
        int l,r;
        ll val,laz;
    };
    vector<S> tre;                  // 节点池：必须在这里声明，下面的成员函数才能用
    Seg(){};
    Seg(int n){
```

```
        tre.resize((n+2)<<2);   // 处理下标 [0,n] 共 n+1 个点，节点要开 4*(n+1) 个
        build(0,n,1);
    };
    void pushup(int p){
        tre[p].val=tre[lc].val+tre[rc].val;
    }
    void pushdown(int p){
        if(tre[p].laz==0) return;
        tre[lc].laz+=tre[p].laz;
        tre[rc].laz+=tre[p].laz;

tre[lc].val+=tre[p].laz*(tre[lc].r-tre[lc].l+1);
// 孩子每个元素都加 laz，要乘区间长度

tre[rc].val+=tre[p].laz*(tre[rc].r-tre[rc].l+1);
        tre[p].laz=0;
    }
    void build(int l,int r,int p){
        tre[p]={l,r,0,0};
        if(l==r){
            tre[p].val=a[l];    // a[]：调用方提供的初始数组（ll）
            return;
        }
        int mid=l+r>>1;
        build(l,mid,lc);
        build(mid+1,r,rc);
        pushup(p);
    }
    void add(int l,int r,ll k,int p){
        if(l<=tre[p].l&&r>=tre[p].r){
            tre[p].val+=k*(tre[p].r-tre[p].l+1);
            tre[p].laz+=k;
            return;
        }
        int mid=tre[p].l+tre[p].r>>1;
        pushdown(p);
        if(l<=mid) add(l,r,k,lc);
        if(r>mid) add(l,r,k,rc);
        pushup(p);
    }
    ll qry(int l,int r,int p){
        if(l<=tre[p].l&&r>=tre[p].r){
            return tre[p].val;
        }
        int mid=tre[p].l+tre[p].r>>1;
        pushdown(p);
        ll ans=0;
        if(l<=mid) ans+=qry(l,r,lc);
        if(r>mid) ans+=qry(l,r,rc);
        return ans;
    }
};
```

2.5 可持久化线段树 (主席树)

写时复制 + 标记永久化（懒标记不 push-down），所以历史版本能安全共享同一批节点；支持区间加、区间和、历史版本查询与回退。下面的代码即 HDU 4348 *To the moon* 的做法（命令 `C/Q/H/B`）：建树 $O(n\log n)$，每次修改与查询 $O(\log n)$。命令含义：`C l r d` 区间 $[l,r]$ 加 d 并把当前时间 $+1$；`Q l r` 询问当前时间的区间和；`H l r t` 询问时间 t 的区间和；`B t` 回到时间 t（之后的版本作废）。

用法：先声明 `PSeg<Info> seg;`，节点池按需增长，不需要预估空间。模板参数 `Info` 需要满足：默认构造为空元、支持 `operator+`、`Info(int)` 与 `Info × int`（按区间长度缩放）；取 `Info = ll` 就是”区间加 + 区间和”，换成结构体可做区间最值、区间 gcd 等。

- `int root = seg.build(1, n, A);`——A 是 1-indexed 数组，返回版本 0 的根；
- `root = seg.upd(root, 1, n, l, r, d);`——区间 $[l,r]$ 每个数加 d，返回新版本的根，旧版本不受影响；
- `seg.sum(root, 1, n, l, r);`——当前版本的区间和；
- `seg.sum(his[t], 1, n, l, r);`——历史版本 t 的区间和，只要把每个版本的根记到 `his[t]` 里即可。

空间上建树占 $4n$ 个节点，每次修改只复制访问路径上的 $O(\log n)$ 个节点。

Listing 21: 可持久化线段树 (主席树)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// 可持久化线段树（主席树）：写时复制 + 标记永久化；接口用法与例题说明见正文。
template <typename Info>
struct PSeg {
    struct Node { int l = 0, r = 0; Info val = Info(), tag = Info(); };
    vector<Node> tr;
    PSeg() { tr.push_back(Node()); }          // 0 号哨兵：孩子为 0，val/tag 为空元
```

```
    int newnode(Node o) { tr.push_back(o); return (int)tr.size() - 1; }   // 按值传入，扩容也不会失效
    /* 区间加：返回新版本的根，不动 pre 那一版 */
    int upd(int pre, int l, int r, int ql, int qr, Info d) {
        int u = newnode(tr[pre]);              // 写时复制
        if (ql <= l && r <= qr) {
            tr[u].val = tr[u].val + d * Info(r - l + 1);    // 整段覆盖：只改 val 与 tag
            tr[u].tag = tr[u].tag + d;
            return u;
        }
        int mid = (l + r) >> 1;
        if (ql <= mid) tr[u].l = upd(tr[pre].l, l, mid, ql, qr, d);
        if (qr > mid)  tr[u].r = upd(tr[pre].r, mid + 1, r, ql, qr, d);
        // 标记永久化：本节点区间和 = 左右孩子和 + 本节点 tag × 区间长度
        tr[u].val = tr[tr[u].l].val + tr[tr[u].r].val + tr[u].tag * Info(r - l + 1);
        return u;
    }
    /* 由初始数组建版本 0 */
    int build(int l, int r, const vector<Info>& a) {
        int u = newnode(Node());
        if (l == r) { tr[u].val = a[l]; return u; }
        int mid = (l + r) >> 1;
        tr[u].l = build(l, mid, a);
        tr[u].r = build(mid + 1, r, a);
        tr[u].val = tr[tr[u].l].val + tr[tr[u].r].val;
        return u;
    }
    /* 区间和：add 为祖先链上累计的 tag */
    Info sum(int u, int l, int r, int ql, int qr, Info add = Info()) {
        if (ql <= l && r <= qr) return tr[u].val + add * Info(r - l + 1);
        int mid = (l + r) >> 1;
        Info res = Info();
        add = add + tr[u].tag;
        if (ql <= mid) res = res + sum(tr[u].l, l, mid, ql, qr, add);
        if (qr > mid)  res = res + sum(tr[u].r, mid + 1, r, ql, qr, add);
        return res;
    }
};

// 例题：HDU 4348 To the moon
//   C l r d：区间 [l,r] 每个数加 d，当前时间 +1
//   Q l r  ：询问当前时间的区间和
//   H l r t：询问时间 t 的区间和
//   B t    ：当前时间回到 t（之后再 C 会覆盖原来的时间 t+1）
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m) {
        vector<ll> A(n + 1);
        for (int i = 1; i <= n; ++i) cin >> A[i];
        PSeg<ll> seg;
        int root = seg.build(1, n, A);
        vector<int> his;                  // his[t] = 时间 t 的根
        his.push_back(root);
        while (m--) {
            char c;
            cin >> c;
            int l, r, t;
            ll d;
            if (c == 'C') {
                cin >> l >> r >> d;
                root = seg.upd(root, 1, n, l, r, d);
                his.push_back(root);
            } else if (c == 'Q') {
                cin >> l >> r;
                cout << seg.sum(root, 1, n, l, r) << '\n';
            } else if (c == 'H') {
                cin >> l >> r >> t;
                cout << seg.sum(his[t], 1, n, l, r) << '\n';
            } else {                      // B t：回到时间 t，之后的版本作废
                cin >> t;
                root = his[t];
                his.resize(t + 1);
            }
        }
    }
    return 0;
}
```

2.6 笛卡尔树

Listing 22: cartesian-tree

```
//P7972 Self Permutation
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const ll mod=1e9+7;
void solve(){
    int n;cin>>n;
    vector<int>L(n+1),R(n+1),st,a(n+1);
    vector<ll>dp(n+1);
    for(int i=1;i<=n;i++){
        cin>>a[i];
        int lc=0;
```

```
        while(!st.empty()&&a[st.back()]>=a[i]){
            lc=st.back();
            st.pop_back();
        }
        L[i]=lc;
        if(!st.empty()) R[st.back()]=i;
        st.emplace_back(i);
    }
    auto dfs=[&](auto &&dfs,int u,int l,int r) ->
void {
        if(L[u]) dfs(dfs,L[u],l,u-1);
        if(R[u]) dfs(dfs,R[u],u+1,r);

(dp[u]+=(dp[L[u]]+1)*(dp[R[u]]+1)%mod)%=mod;
        if(l>1) (dp[u]+=dp[R[u]])%=mod;
        if(r<n) (dp[u]+=dp[L[u]])%=mod;
    };
    dfs(dfs,st.front(),1,n);
    cout<<dp[st.front()]<<endl;
}
```

2.7 字典树

Trie trie(n); 里的 n 是**所有串的总长度上限**（也就是节点数上界），要开够；字符映射为 A-Z $\to 0..25$、a-z $\to 26..51$、0-9 $\to 52..61$。

Listing 23: Trie

```
// 字典树：统计每个前缀被多少个串经过；用法见正文。
struct Trie{
    vector<int>cnt;
    vector< array<int,65> >son;
    int idx=0;
    Trie(int n){
        cnt.resize(n+1);
        son.resize(n+1);
    }
    void insert(string &s){
        int num;
        int p=0;
        for(char &c:s){
            if(isdigit(c)) num=c-'0'+52;
            else if(islower(c)) num=c-'a'+26;
            else if(isupper(c)) num=c-'A';
            if(!son[p][num]) son[p][num]=(++idx);
            p=son[p][num];
            ++cnt[p];
        }
    }
    int query(string &s){
        int num;
        int p=0;
        for(char &c:s){
            if(isdigit(c)) num=c-'0'+52;
            else if(islower(c)) num=c-'a'+26;
            else if(isupper(c)) num=c-'A';
            if(!son[p][num]) return 0;
            p=son[p][num];
        }
        return cnt[p];
    }
};
```

Listing 24: 01-Trie

```
//P4551 最长异或路径
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
struct ZTrie{
    vector<array<int,2>>son;
    ZTrie(int n){
        son.reserve(n+1);
        son.push_back({0,0});   //
动态开点，节点数不再受固定上界限制
    }
    void insert(int x){
        int p=0;
        for(int i=31;i>=0;i--) {
            int bit=(x>>i&1);
            if(!son[p][bit]) {
                son[p][bit]=(int)son.size();
                son.push_back({0,0});
            }
            p=son[p][bit];
        }
    }
    int query(int x){
        int ans=0,p=0;
        for(int i=31;i>=0;i--){
            int bit=(x>>i&1);
            if(son[p][bit^1])
p=son[p][bit^1],ans|=(1<<i);
            else if(son[p][bit]) p=son[p][bit];
            else return ans;
        }
        return ans;
    }
}trie(2e6);
const int N=2e5+5;
struct node{
    int u,w;
};
vector<node>G[N];
int ans=0;
void dfs(int u,int fa,int sum){
    trie.insert(sum);
    ans=max(ans,trie.query(sum));
    for(auto &[v,w]:G[u]){
        if(v==fa) continue;
        dfs(v,u,sum^w);
    }
}
void solve(){
    int n;cin>>n;
```

```
    for(int i=1;i<n;i++){
        int x,y,w;cin>>x>>y>>w;
        G[x].push_back({y,w});
        G[y].push_back({x,w});
    }
    dfs(1,0,0);
    cout<<ans<<endl;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    solve();
    return 0;
}
```

2.8 平衡树

Listing 25: FHQ-Treap

```
template <typename T>
struct FHQ_Treap {
    struct Node {
        T data;
        int l = 0, r = 0, sz = 1;
        unsigned rnd;
        bool rev = false;   // 翻转懒标记
    };
    std::vector<Node> t;   // 下标从1开始，0为空节点
    int root = 0;
    FHQ_Treap() {
        t.push_back(Node{});   // 占位0号
        t[0].sz = 0;
    }
    // 完全清空
    void clear() {
        t.clear();
        t.push_back(Node{});
        t[0].sz = 0;
        root = 0;
    }
    bool empty() const { return t[root].sz == 0; }
    int size() const { return t[root].sz; }
    void push(int u) {
        if (!u || !t[u].rev) return;
        swap(t[u].l, t[u].r);
        if (t[u].l) t[t[u].l].rev ^= 1;
        if (t[u].r) t[t[u].r].rev ^= 1;
        t[u].rev = false;
    }
    // 新建节点（随机优先级）
    int new_node(const T& val) {
        static std::mt19937 rng(std::chrono::
steady_clock::now().time_since_epoch().count());
        t.push_back({val, 0, 0, 1, static_cast<
unsigned>(rng())});
        return static_cast<int>(t.size()) - 1;
    }
    void pull(int u) {
        if (!u) return;
        t[u].sz = 1 + t[t[u].l].sz + t[t[u].r].sz;
        // 若T支持区间信息合并，可在此调用 t[u].data.
pull(t[t[u].l].data, t[t[u].r].data);
    }
    // 按大小分裂：左树得到前k个元素，右树为剩余部分
    void split(int u, int k, int& L, int& R) {
        if (!u) { L = R = 0; return; }
        push(u);   // 关键！
        int left_sz = t[t[u].l].sz;
        if (k <= left_sz) {
            R = u;
            split(t[u].l, k, L, t[u].l);
        } else {
            L = u;
            split(t[u].r, k - left_sz - 1, t[u].r,
R);
        }
        pull(u);
    }
    // 合并两棵树（L所有元素在R之前）
    int merge(int L, int R) {
        if (!L || !R) return L ? L : R;
        push(L);
        push(R);   // 关键！
        if (t[L].rnd < t[R].rnd) {
            t[L].r = merge(t[L].r, R);
            pull(L);
            return L;
        } else {
            t[R].l = merge(L, t[R].l);
            pull(R);
            return R;
        }
    }
    // ---------- 位置操作（1-indexed） ----------
    // 在第pos个元素之后插入（pos = 0 表示头部，pos =
 size() 表示尾部）
    void insert(int pos, const T& val) {
        int L, R;
        split(root, pos, L, R);
        int u = new_node(val);
        root = merge(merge(L, u), R);
    }
    void reverse(int l, int r) {  // 翻转 [l, r]
(1-indexed)
        //assert(1 <= l && l <= r && r <= size());
        int L, M, R;
        split(root, l - 1, L, M);
        split(M, r - l + 1, M, R);
        t[M].rev ^= 1;              // 打上翻转标记
        root = merge(merge(L, M), R);
    }
    // 删除第k个元素（1-indexed），返回被删除的值
    T erase(int k) {
        int L, M, R;
        split(root, k - 1, L, M);
        split(M, 1, M, R);
        T removed = t[M].data;
```

```
        root = merge(L, R);
        // 可选：回收节点 M （本例中忽略，vector不回收）
        return removed;
    }
    // 删除区间 [l, r]（1-indexed 闭区间）
    void erase_range(int l, int r) {
        int L, M, R;
        split(root, l - 1, L, M);
        split(M, r - l + 1, M, R);
        root = merge(L, R);
        // M 整棵子树被丢弃
    }
    // 查询第k个元素（1-indexed）
    T get_kth(int k)  {
        int u = root;
        while (u) {
            push(u);   // 关键！
            int left_sz = t[t[u].l].sz;
            if (k <= left_sz) {
                u = t[u].l;
            } else if (k == left_sz + 1) {
                return t[u].data;
            } else {
                k -= left_sz + 1;
                u = t[u].r;
            }
        }
        return T{}; // 不会到达
    }
    // 便捷访问（同 get_kth）
    T operator[](int k) { return get_kth(k); }
    // ---------- 构建操作 ----------
    // 线性时间构建（笛卡尔树）—— 输入序列的顺序即为最终中序顺序
    void build(const std::vector<T>& arr) {
        clear();                        // 清空原有树
        if (arr.empty()) return;
        std::vector<int> nodes;         // 存储每个节点在 t 中的下标
        nodes.reserve(arr.size());
        for (const auto& val : arr) {
            nodes.push_back(new_node(val));
        }
        // 单调栈维护右链，确保按随机优先级满足堆性质
        std::vector<int> St;
        for (int cur : nodes) {
            int last = 0;
            while (!St.empty() && t[St.back()].rnd > t[cur].rnd) {
                last = St.back();
                St.pop_back();
            }
            t[cur].l = last;                // 当前节点的左子 = 最后一个被弹出的节点
            if (!St.empty()) {
                t[St.back()].r = cur;    // 栈顶的右子指向当前节点
            }
            St.push_back(cur);
        }
        // 栈底即为根
        root = St.empty() ? 0 : St[0];
        // 后序遍历更新所有节点的 sz（因为只设置了左右关系，sz 仍为1）
        update_subtree_size(root);
    }
private:
    // 后序遍历更新子树大小
    void update_subtree_size(int u) {
        if (!u) return;
        update_subtree_size(t[u].l);
        update_subtree_size(t[u].r);
        pull(u);
    }
};
```

2.9 根号分治

- 第一行：两个整数 n, q（$1 \le n, q \le 10^5$），分别表示序列长度和询问次数。
- 第二行：n 个整数 $A_1, A_2, \ldots, A_n$（$1 \le A_i \le 10^9$），表示给定的整数序列。
- 接下来 q 行：每行两个整数 x, y（$1 \le x, y \le 10^9$），表示一次询问。

输出格式

输出 q 行，每行一个整数，表示对应询问的答案。

简要分析

给定序列 $A[1..n]$ 和询问 (x, y)，需统计满足 $i < j$ 且 $A[i] = x, A[j] = y$ 的数对数量。记 $pos[v]$ 为值 v 在序列中出现的位置集合（升序排列），则答案可表示为：

$$\sum_{p \in pos[x]} (|pos[y]| - \text{upper_bound}(pos[y], p))$$

或等价表示为：

$$\sum_{p \in pos[y]} \text{lower_bound}(pos[x], p)$$

核心划分

设阈值 $B = \sqrt{n}$（取整），将所有值分为两类：

1. **高频值（Heavy）**：满足 $|pos[v]| > B$ 的值，记高频值集合为 H；

2. **低频值（Light）**：满足 $|pos[v]| \le B$ 的值。

预处理阶段（离线预处理高频值对）

对于任意两个高频值 $u, v \in H$，预计算数对 (u, v) 的答案并存储在哈希表 mp 中：

$$mp[(u,v)] = \sum_{p \in pos[u]} \Big(|pos[v]| - \text{upper_bound}(pos[v], p) \Big)$$

预处理时间复杂度：$O(|H|^2 \cdot B)$，由于 $|H| \le \frac{n}{B}$，故总复杂度为 $O(nB)$。

查询阶段（分情况处理）

对每个询问 (x, y)，分三类处理（结合记忆化避免重复计算）：

1. 若 (x, y) 已计算过（记忆化哈希表 memo 存在），直接返回结果；

2. 若 x, y 均为高频值，直接调用预处理结果 $mp[(x, y)]$；

3. 若至少一个为低频值，选择更小的集合遍历计算：

   - 若 $|pos[x]| \le |pos[y]|$：遍历 $pos[x]$，对每个位置 p 统计 $pos[y]$ 中大于 p 的元素数量；
   - 否则：遍历 $pos[y]$，对每个位置 p 统计 $pos[x]$ 中小于 p 的元素数量。

   单次查询时间复杂度：$O(B)$。

复杂度分析

预处理复杂度：$O(n\sqrt{n})$；单次查询复杂度：$O(\sqrt{n})$，总查询复杂度：$O(q\sqrt{n})$；整体时间复杂度：$O((n+q)\sqrt{n})$，可通过 10^5 规模的数据。

Listing 26: 根号分治

```
void solve(){
    int n,q;cin>>n>>q;
    vector<int>a(n+1);
    unordered_map< int,vector<int> >pos;//存储重数
    for(int i=1;i<=n;i++){
        cin>>a[i];
        pos[a[i]].push_back(i);
    }
    int siz=sqrt(n+0.5);
    vector<int>H;
    for(auto const&[x,y]:pos){
        if(y.size()>siz) H.emplace_back(x);
    }
    map<pair<int,int>,ll>mp;
    for(int i=0;i<H.size();i++){
        for(int j=0;j<H.size();j++){
            for(int &p:pos[H[i]]){
                mp[{H[i],H[j]}]+=pos[H[j]].end()-
upper_bound(pos[H[j]].begin(),pos[H[j]].end(),p);
            }
        }
    }
    map<pair<int,int>,ll>memo;
    for(int i=1;i<=q;i++){
        int x,y;cin>>x>>y;
        if(memo.count({x,y})){
            cout<<memo[{x,y}]<<endl;
            continue;
        }
        if(pos[x].size()>siz&&pos[y].size()>siz) {
            cout<<(memo[{x,y}]=mp[{x,y}])<<endl;
        }
        else{
            ll ans=0;
            if(pos[x].size()<=pos[y].size()){
                for(int &p:pos[x]){
                    ans+=pos[y].end()-upper_bound(
pos[y].begin(),pos[y].end(),p);
                }
            }
            else{
                for(int &p:pos[y]){
                    ans+=lower_bound(pos[x].begin()
,pos[x].end(),p)-pos[x].begin();
                }
            }
            cout<<(memo[{x,y}]=ans)<<endl;
        }
    }
}
```

2.10 无向图三元环计数

题面简述

给定一个 n 个点 m 条边的简单无向图（无重边、无自环，不保证连通），求图中三元环的个数。

三元环指恰好包含三个点 u, v, w 及三条边 $\langle u, v\rangle, \langle v, w\rangle, \langle w, u\rangle$ 的子图。两个三元环不同当且仅当存在一个点属于其中一个而不属于另一

个。

数据范围：$1 \le n \le 10^5$，$1 \le m \le 2 \times 10^5$。

做法

采用经典的**无向边定向 + 枚举**算法。

1. **定向**：对每条无向边 (u, v)，令其从度数小的点指向度数大的点；若度数相同，则从编号小的点指向编号大的点。这样将无向图转化为一个 DAG，记有向边集为 E'。

2. **枚举**：对每个点 u，先用标记数组标记 u 的所有出邻居。然后对每条出边 $u \to v$，枚举 v 的所有出邻居 w，若 w 被 u 标记过（即 $u \to w \in E'$），则 (u, v, w) 构成一个三元环，答案加一。

由于定向后每个三元环的三条边恰好形成一条有向路径 $u \to v \to w$（其中 $u \to w$ 也存在），因此每个三元环恰好被统计一次，无需去重。

复杂度分析

引理：定向后每个点的出度 $d^+(u) \le \sqrt{2m}$。

证明. 对任意点 u，其每个出邻居 v 都满足 $\deg(v) \ge \deg(u) \ge d^+(u)$。因此 u 的 $d^+(u)$ 个出邻居的度数之和满足：

$$\sum_{v \in N^+(u)} \deg(v) \ge (d^+(u))^2$$

又因为所有边的度数贡献总和不超过 $2m$，故 $(d^+(u))^2 \le 2m$，即 $d^+(u) \le \sqrt{2m}$。 □

算法的核心操作量为：

$$\begin{aligned} T &= \sum_{(u \to v) \in E'} d^+(v) \le \sum_{(u \to v) \in E'} \sqrt{2m} \\ &= |E'| \cdot \sqrt{2m} = m\sqrt{2m} \end{aligned}$$

因此总时间复杂度为 $O(m\sqrt{m})$。

对于本题 $m = 2 \times 10^5$，操作量约为 $2 \times 10^5 \times 632 \approx 1.26 \times 10^8$，可以通过。空间复杂度为 $O(n + m)$。

Listing 27: 根号分治求三元环计数

```cpp
#include <bits/stdc++.h>
using namespace std;
int main(){
    cin.tie(nullptr)->ios::sync_with_stdio(false);
    int n, m;
    cin >> n >> m;
    vector<pair<int,int>> edges(m);
    vector<int>deg(n + 1, 0);
    for(int i = 0; i < m; i++){
        cin >> edges[i].first >> edges[i].second;
        deg[edges[i].first]++;
        deg[edges[i].second]++;
    }
    // 定向：度数小 -> 度数大，度数相同则编号小 -> 编号大
    vector<vector<int>> adj(n + 1);
    for(int i = 0; i < m; i++){
        int u = edges[i].first, v = edges[i].second;
        if(deg[u] < deg[v] || (deg[u] == deg[v] && u < v)){
            adj[u].push_back(v);
        } else {
            adj[v].push_back(u);
        }
    }
    // 枚举三元环
    vector<int> mark(n + 1, 0); // mark[w] = u 表示 u->w 这条边存在
    long long ans = 0;
    for(int u = 1; u <= n; u++){
        // 标记 u 的所有出边邻居
        for(int v : adj[u]){
            mark[v] = u;
        }
        // 枚举 u -> v -> w，检查 u -> w
        for(int v : adj[u]){
            for(int w : adj[v]){
                if(mark[w] == u){
                    ans++;
                }
            }
        }
    }
    cout << ans << "\n";
    return 0;
}
```

2.11 莫队

Listing 28: 普通莫队算法

```cpp
//P2709 小B的询问
const int N=5e4+10;
int a[N],pos[N],cnt[N];
ll res,ans[N];
struct Q{
```

```
    int l,r,k;
}q[N];
void Add(int n){
    cnt[a[n]]++;
    res+=cnt[a[n]]*cnt[a[n]]-(cnt[a[n]]-1)*(cnt[a[n]]-1);
}
void Sub(int n){
    cnt[a[n]]--;
    res-=(cnt[a[n]]+1)*(cnt[a[n]]+1)-(cnt[a[n]])*(cnt[a[n]]);
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0),cout.tie(0);
    int n,m,k;cin>>n>>m>>k;
    int siz=sqrt(n+0.5);
    for(int i=1;i<=n;i++){
        cin>>a[i];
        pos[i]=i/siz;
    }
    for(int i=0;i<m;i++){
        cin>>q[i].l>>q[i].r;
        q[i].k=i;
    }
    sort(q,q+m,[](Q x,Q y){
        return pos[x.l]==pos[y.l]?x.r<y.r:pos[x.l]<pos[y.l];
    });
    int l=1,r=0;
    for(int i=0;i<m;i++){
        while(q[i].l<l) Add(--l);
        while(q[i].r>r) Add(++r);
        while(q[i].l>l) Sub(l++);
        while(q[i].r<r) Sub(r--);
        ans[q[i].k]=res;
    }
    for(int i=0;i<m;i++){
        cout<<ans[i]<<endl;
    }
    return 0;
}
```

3 树

3.1 LCA

Listing 29: dfn 大跳 LCA

```
//P3379 最近公共祖先
const int N=5e5+5;
vector<int>G[N];
int f[N],d[N],dfn[N],top[N],cnt,siz[N],son[N];
void dfs1(int u,int fa){
    d[u]=d[fa]+1;
    f[u]=fa;
    siz[u]=1;
    for(int v:G[u]){
        if(fa==v) continue;
        dfs1(v,u);
        siz[u]+=siz[v];
        if(siz[v]>siz[son[u]]) son[u]=v;
    }
}
void dfs2(int u,int node){
    dfn[u]=(++cnt);//维护dfn序
    top[u]=node;
    if(son[u]) dfs2(son[u],node);
    for(int v:G[u]){
        if(v==f[u]||v==son[u]) continue;
        dfs2(v,v);
    }
}
int LCA(int l,int r){
    while(top[l]!=top[r]){
        if(d[top[l]]<d[top[r]]) swap(l,r);
        l=f[top[l]];
    }
    if(d[l]<d[r]) return l;
    return r;
}//树上LCA跳
void solve(){
    int n,m,s;
    cin>>n>>m>>s;
    for(int i=1;i<n;i++){
        int x,y;cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    dfs1(s,0);
    dfs2(s,s);
    for(int i=1;i<=m;i++){
        int x,y;cin>>x>>y;
        cout<<LCA(x,y)<<endl;
    }
}
```

Listing 30: dfn 实现 O1 LCA

```
//P3379 最近公共祖先
void solve(){
    int n,q,s;
    cin>>n>>q>>s;
    vector<vector<int>>G(n+1);
    for(int i=1;i<n;i++){
        int x,y;
        cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    vector<int>d(n+1);
    vector<int>f(n+1);
    vector<int>dfn(n+1);
    int cnt=0;
    auto dfs=[&](auto &&dfs,int u,int fa) ->void {
        f[u]=fa;
        dfn[u]=(++cnt);
```

```
        d[u]=d[fa]+1;
        for(int v:G[u]){
            if(v==fa) continue;
            dfs(dfs,v,u);
        }
    };
    dfs(dfs,s,0);
    vector<vector<int>>st(n+1,vector<int>(21));
    for(int i=1;i<=n;i++) st[dfn[i]][0]=f[i];
    for(int j=1;j<=20;j++){
        for(int i=1;i+(1<<j)-1<=n;i++){
            int l=st[i][j-1];
            int r=st[i+(1<<j-1)][j-1];
            if(d[l]<d[r]) st[i][j]=l;
            else st[i][j]=r;
        }
    }
    auto lca=[&](int l,int r) ->int {
        if(l==r) return l;
        l=dfn[l],r=dfn[r];
        if(l>r) swap(l,r);
        l++;
        int len=__lg(r-l+1);
        int L=st[l][len];
        int R=st[r-(1<<len)+1][len];
        return (d[L]<d[R])?L:R;
    };
    while(q--){
        int x,y;
        cin>>x>>y;
        cout<<lca(x,y)<<endl;
    }
}
```

3.2 树的直径

Listing 31: 两遍 dfs 查直径

```
const int N=2e5+10;
vector<int>G[N];
bool vis[N];
ll d[N],s=1;
void dfs(int u,int f){
    for(auto &v:G[u]){
        if(v==f) continue;
        d[v]=d[u]+1;
        dfs(v,u);
    }
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n;cin>>n;
    for(int i=1,x,y;i<n;i++){
        cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    d[s]=0;dfs(s,0);
    for(int i=1;i<=n;i++) if(d[i]>d[s]) s=i;
    d[s]=0;dfs(s,0);
    ll ans=0;
    for(int i=1;i<=n;i++) ans=max(ans,d[i]);
    cout<<ans<<endl;
    return 0;
}
```

Listing 32: 树形 dp 求直径

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n;cin>>n;
    vector<vector<int>>G(n+1);
    vector<int>down(n+1),up(n+1);
    down[0] = -1;   // 空儿子深度为 -1
    for(int i=1;i<n;i++){
        int x,y;cin>>x>>y;
        G[x].emplace_back(y);
        G[y].emplace_back(x);
    }
    auto dfs1=[&](auto &&dfs1,int u,int fa) ->void
{
        for(int v:G[u]){
            if(v==fa) continue;
            dfs1(dfs1,v,u);
            down[u]=max(down[u],down[v]+1);
        }
    };
    dfs1(dfs1,1,0);
    auto dfs2=[&](auto &&dfs2,int u,int fa) ->void
{
        int m1=0,m2=0;//最长路径，次长路径
        for(int v:G[u]){
            if(v==fa) continue;
            if(down[m1]<down[v]) m2=m1,m1=v;
            else if(down[m2]<down[v]) m2=v;
        }
        for(int v:G[u]){
            if(v==fa) continue;
            up[v]=up[u]+1;
            if(m1!=0&&m1!=v)
up[v]=max(up[v],down[m1]+2);
            else if(m2!=0)
up[v]=max(up[v],down[m2]+2);
            dfs2(dfs2,v,u);
        }
    };
    dfs2(dfs2,1,0);
    int ans=0;
    for(int i=1;i<=n;i++)
ans=max(ans,down[i]+up[i]);
    cout<<ans<<endl;
    return 0;
}
```

3.3 树链剖分

Listing 33: 树链剖分

```
//P3384 [模板]重链剖分 / 树链剖分
const int N=1e5+5;
ll mod;
int n,m,r;
vector<int>G[N];
int d[N],f[N],siz[N],dfn[N],rev[N],cnt,top[N],son[N];
ll val[N],new_val[N];
struct Seg{
    struct S{
        int l,r;
        ll sum,laz;
    };
    int n;
    vector<S>tre;
    Seg(){};
    Seg(int n):n(n){
        tre.resize(n<<2);
        build(0,n,1);
    };
    #define lc (p<<1)
    #define rc (p<<1|1)
    void pushup(int p){
        tre[p].sum=(tre[lc].sum+tre[rc].sum)%mod;
    }
    void pushdown(int p){
        if(tre[p].laz==0) return;
        tre[lc].laz=(tre[lc].laz+tre[p].laz)%mod;
        tre[rc].laz=(tre[rc].laz+tre[p].laz)%mod;
        tre[lc].sum=(tre[lc].sum+tre[p].laz*(tre[lc].r-tre[lc].l+1)%mod)%mod;
        tre[rc].sum=(tre[rc].sum+tre[p].laz*(tre[rc].r-tre[rc].l+1)%mod)%mod;
        tre[p].laz=0;
    }
    void build(int l,int r,int p){
        tre[p]={l,r,0LL,0LL};
        if(l==r) {
            tre[p].sum=new_val[l]%mod;
            return;
        }
        int mid=l+r>>1;
        build(l,mid,lc);
        build(mid+1,r,rc);
        pushup(p);
    }
    void Add(int l,int r,ll k,int p){
        if(l<=tre[p].l&&r>=tre[p].r){
            tre[p].sum=(tre[p].sum+k*(tre[p].r-tre[p].l+1)%mod)%mod;
            tre[p].laz=(tre[p].laz+k)%mod;
            return;
        }
        int mid=tre[p].l+tre[p].r>>1;
        pushdown(p);
        if(l<=mid) Add(l,r,k,lc);
        if(r>mid) Add(l,r,k,rc);
        pushup(p);
    }
    ll qry(int l,int r,int p){
        if(l<=tre[p].l&&r>=tre[p].r) return tre[p].sum;
        ll ans=0;
        int mid=tre[p].l+tre[p].r>>1;
        pushdown(p);
        if(l<=mid) ans=(ans+qry(l,r,lc))%mod;
        if(r>mid) ans=(ans+qry(l,r,rc))%mod;
        return ans;
    }
}S;
void dfs1(int u,int fa){
    f[u]=fa;
    d[u]=d[fa]+1;
    siz[u]=1;
    for(int v:G[u]){
        if(v==fa) continue;
        dfs1(v,u);
        siz[u]+=siz[v];
        if(siz[v]>siz[son[u]]) son[u]=v;
    }
}//第一遍初始化节点信息
void dfs2(int u,int node){
    top[u]=node;
    dfn[u]=(++cnt);
    rev[cnt]=u;
    new_val[cnt]=val[u];
    if(son[u]) dfs2(son[u],node);
    for(int v:G[u]){
        if(v!=son[u]&&v!=f[u]) dfs2(v,v);
    }
}
void Upd_path(int l,int r,ll k){
    k%=mod;
    while(top[l]!=top[r]){
        if(d[top[l]]<d[top[r]]) swap(l,r);
        S.Add(dfn[top[l]],dfn[l],k,1);
        l=f[top[l]];
    }
    if(d[l]>d[r]) swap(l,r);
    S.Add(dfn[l],dfn[r],k,1);
}
ll qry_path(int l,int r){
    int res=0;
    while(top[l]!=top[r]){
        if(d[top[l]]<d[top[r]]) swap(l,r);
        res=(res+S.qry(dfn[top[l]],dfn[l],1))%mod;
        l=f[top[l]];
    }//不在同一个重链中LCA跳
    if(d[l]>d[r]) swap(l,r);
    res=(res+S.qry(dfn[l],dfn[r],1))%mod;
    return res;
}
```

```cpp
void Upd(int u,ll k){
    k%=mod;
    S.Add(dfn[u],dfn[u]+siz[u]-1,k,1);
}
ll qry(int u){
    return S.qry(dfn[u],dfn[u]+siz[u]-1,1);
}
void solve(){
    cin>>n>>m>>r>>mod;
    for(int i=1;i<=n;i++) cin>>val[i];
    for(int i=1;i<n;i++){
        int u,v;cin>>u>>v;
        G[u].push_back(v);
        G[v].push_back(u);
    }
    dfs1(r,0);
    dfs2(r,r);
    S=Seg(n+1);
    for(int i=1;i<=m;i++){
        int op,l,r;ll k;cin>>op;
        if(op==1){
            cin>>l>>r>>k;
            Upd_path(l,r,k);
        }
        else if(op==2){
            cin>>l>>r;
            cout<<qry_path(l,r)<<endl;
        }
        else if(op==3){
            cin>>l>>k;
            Upd(l,k);
        }
        else if(op==4){
            cin>>l;
            cout<<qry(l)<<endl;
        }
    }
}
```

3.4 树上启发式合并

题面简述

给一棵以 1 为根的树，共 n 个结点，每个结点有一个颜色 $c[i]$。每次询问给出一个结点 u，求以 u 为根的子树中有多少种不同的颜色。

第一步：求 DFS 序与子树区间

对树进行一次 DFS，记录每个结点的进入时间 $\text{in}[u]$ 和离开时间 $\text{out}[u]$。则以 u 为根的子树恰好对应 DFS 序中的连续区间 $[\text{in}[u],\ \text{out}[u]]$。

同时在 DFS 过程中记录每个结点的**重儿子**（子树大小最大的儿子），记为 $\text{heavy}[u]$。

第二步：启发式合并（DSU on Tree）

对每个结点 u，按如下顺序处理：

1. 递归处理所有**轻儿子**，处理完后**清除**其对全局计数数组的贡献。
2. 递归处理**重儿子**，处理完后**保留**其对全局计数数组的贡献。
3. 暴力遍历所有轻儿子的子树中的结点，将它们的颜色加入全局计数数组 $\text{cnt}[\cdot]$：

$$\text{cnt}[c[v]] \leftarrow \text{cnt}[c[v]] + 1$$

若某颜色 $c[v]$ 的计数从 0 变为 1，则当前不同颜色种类数加一：

$$\text{tot} \leftarrow \text{tot} + [\text{cnt}[c[v]] = 1]$$

4. 将结点 u 自身的颜色也加入计数数组，同理更新 tot。
5. 此时 tot 即为以 u 为根的子树的颜色种类数，记录答案：

$$\text{ans}[u] = \text{tot}$$

6. 若 u 本身是其父结点的轻儿子，则暴力清除整棵子树的贡献（将 $\text{cnt}[\cdot]$ 和 tot 恢复）。

第三步：回答询问

所有询问离线处理。对于每个询问的结点 u，直接输出 $\text{ans}[u]$ 即可。

正确性说明

启发式合并的核心思想在于：每次只保留重儿子的信息，轻儿子的信息暴力重算。由于每个结点到根的路径上至多有 $O(\log n)$ 条轻边（每经过一条轻边，子树大小至少翻倍），所以每个结点被暴力遍历的总次数为 $O(\log n)$。

复杂度分析

- **时间复杂度**：每个结点被暴力加入/清除计数数组的次数为其到根路径上的轻边数，至多 $O(\log n)$ 次。因此总时间复杂度为：

$$O(n \log n + m)$$

其中 m 为询问数。

Listing 34: 树上启发式合并

```cpp
//U41492 树上数颜色
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int N=1e5+5;
vector<int>G[N];
int cnt[N],siz[N],son[N],f[N],res,a[N],ans[N];
void dfs1(int u,int fa){
    f[u]=fa;
    siz[u]=1;
    for(int v:G[u]){
        if(v==fa) continue;
        dfs1(v,u);
        siz[u]+=siz[v];
        if(siz[v]>siz[son[u]]) son[u]=v;
    }
}
void Upd(int u,bool F,int skiped){
    if(F){
        cnt[a[u]]++;
        if(cnt[a[u]]==1) res++;
    }
    else{
        cnt[a[u]]--;
        if(cnt[a[u]]==0) res--;
    }
    for(int v:G[u]){
        if(v==f[u]||v==skiped) continue;
        Upd(v,F,0);
    }
}//更新轻儿子贡献
void dfs2(int u,bool F){
    for(int v:G[u]){
        if(v==son[u]||v==f[u]) continue;
        dfs2(v,0);
    }
    if(son[u]) {
        dfs2(son[u],1);
    }
    Upd(u,1,son[u]);
    ans[u]=res;
    if(!F) Upd(u,0,0);
}
void solve(){
    int n;cin>>n;
    for(int i=1;i<n;i++){
        int x,y;cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    for(int i=1;i<=n;i++) cin>>a[i];
    dfs1(1,0);
    dfs2(1,1);
    int q;cin>>q;
    while(q--){
        int x;cin>>x;
        cout<<ans[x]<<endl;
    }
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    solve();
    return 0;
}
```

4 图论

4.1 拓扑排序与 0-1 BFS

拓扑排序用 Kahn 算法，返回是否为 DAG 及一个拓扑序；0-1 BFS 要求边权只有 0 和 1，可在线性复杂度 $O(n+m)$ 内求单源最短路。

Listing 35: 拓扑排序与 0-1 BFS

```cpp
#include <bits/stdc++.h>
using namespace std;

// Kahn 拓扑排序：返回 {是否 DAG, 一个拓扑序}
pair<bool, vector<int>> topo(const
vector<vector<int>> &g) {
    int n = g.size();
    vector<int> deg(n);
    for (int u = 0; u < n; ++u) {
        for (int v : g[u]) {
            ++deg[v];
        }
    }

    queue<int> q;
    for (int u = 0; u < n; ++u) {
        if (deg[u] == 0) {
            q.push(u);
        }
    }

    vector<int> ord;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ord.push_back(u);
        for (int v : g[u]) {
            if (--deg[v] == 0) {
                q.push(v);
            }
        }
    }
    return {(int)ord.size() == n, move(ord)};
}

// 边权只有 0/1 的单源最短路
vector<int> bfs01(const vector<vector<pair<int,
int>>> &g, int s) {
    constexpr int INF = numeric_limits<int>::max()
/ 2;
    int n = g.size();
    vector<int> dis(n, INF);
    deque<int> q;
    dis[s] = 0;
    q.push_front(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop_front();
        for (auto [v, w] : g[u]) {
            assert(w == 0 || w == 1);
            if (dis[v] <= dis[u] + w) {
                continue;
            }
            dis[v] = dis[u] + w;
            if (w == 0) {
                q.push_front(v);
            } else {
                q.push_back(v);
            }
        }
    }
    return dis;
}
```

4.2 SPFA

Listing 36: SPFA

```cpp
struct node{
    int u;ll w;
};
vector<node>G[N];
ll dis[N];
bool inq[N];
int n,m,s;
void spfa(int s){
    fill(dis+1,dis+1+n,(ll)4e18);
    fill(inq+1,inq+1+n,false);
    queue<int>Q;
    Q.push(s);
    dis[s]=0;
    inq[s]=1;
    while(!Q.empty()){
        int u=Q.front();
        Q.pop();
        inq[u]=0;
        for(auto &[v,w]:G[u]){
            if(dis[v]>dis[u]+w){
                dis[v]=dis[u]+w;
                if(!inq[v]){
                    Q.push(v);
                    inq[v]=1;
                }
            }
        }
    }
}
void solve(){
    cin>>n>>m>>s;
    for(int i=1;i<=m;i++){
        int x,y;ll w;
        cin>>x>>y>>w;
        G[x].push_back({y,w});
    }
    spfa(s);
```

```
    for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
    cout<<endl;
}
```

4.3 Dijkstra

Listing 37: Dijkstra(稀疏图堆优化版本)

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int N=2e5+5;
struct node{
    int u;ll w;
    friend bool operator<(node a,node b){
        return a.w>b.w;
    }
};
vector<node>G[N];
bool vis[N];
ll dis[N];
int n,m,s;
void Dijkstra(int s){
    fill(dis+1,dis+1+n,(ll)4e18);
    fill(vis+1,vis+1+n,false);
    priority_queue<node>Q;
    dis[s]=0;
    Q.push({s,0});
    while(!Q.empty()){
        int u=Q.top().u;
        Q.pop();
        if(vis[u]) continue;
        vis[u]=1;
        for(auto &[v,w]:G[u]){
            if(dis[v]>dis[u]+w){
                dis[v]=dis[u]+w;
                Q.push({v,dis[v]});
            }
        }
    }
}
void solve(){
    cin>>n>>m>>s;
    for(int i=1;i<=m;i++){
        int x,y;ll w;
        cin>>x>>y>>w;
        G[x].push_back({y,w});
    }
    Dijkstra(s);
    for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
    cout<<endl;
}
int main(){
    solve();
    return 0;
}
```

Listing 38: Dijkstra(稠密图邻接矩阵版)

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int N=1010;      // 注意:n<=1000
推荐使用稠密版，否则改用稀疏图堆优化!
const ll INF = 1e18;

ll g[N][N];
ll dis[N];
bool vis[N];
int n,m,s;

void Dijkstra(int s){
    for(int i=1;i<=n;i++){
        dis[i]=INF;
        vis[i]=false;
    }
    dis[s]=0;
    for(int i=1;i<=n;i++){
        int u=-1; ll minv=INF;
        for(int j=1;j<=n;j++){
            if(!vis[j] && dis[j]<minv){
                minv=dis[j];
                u=j;
            }
        }
        if(u==-1) break;
        vis[u]=true;
        for(int j=1;j<=n;j++){
            if(g[u][j]<INF &&
dis[j]>dis[u]+g[u][j]){
                dis[j]=dis[u]+g[u][j];
            }
        }
    }
}

void solve(){
    cin>>n>>m>>s;
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            g[i][j]=(i==j?0:INF);
    for(int i=1;i<=m;i++){
        int x,y;ll w;
        cin>>x>>y>>w;
        g[x][y]=min(g[x][y],w);   // 支持重边取最优
    }
    Dijkstra(s);
    for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
    cout<<endl;
}
int main(){
    solve();
    return 0;
}
```

4.4 最小生成树

Listing 39: Kruskal 求最小生成树

```cpp
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
ll ans;int n,m,cntn;
const int N=2e5+10;
int f[N];
int find(int x){
    if(x==f[x]) return x;
    return f[x]=find(f[x]);
}
struct node{
    int x,y,w;
    friend bool operator <(node a,node b){
        return a.w<b.w;
    }
}a[N];
void kruskal(){
    cntn=n;
    for(int i=1;i<=m;i++){
        int fx=find(a[i].x);
        int fy=find(a[i].y);
        if(fx!=fy){
            f[fx]=fy;
            cntn--;
            ans+=a[i].w;
            if(cntn==1) break;
        }
    }
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    cin>>n>>m;
    for(int i=1;i<=n;i++) f[i]=i;
    for(int i=1;i<=m;i++){
        int x,y,w;
        cin>>a[i].x>>a[i].y>>a[i].w;
    }
    sort(a+1,a+1+m);
    kruskal();
    if(cntn==1) cout<<ans<<endl;
    else cout<<"none"<<endl;
    return 0;
}
```

Listing 40: Prim 求最小生成树 (小根堆优化)

```cpp
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int inf=0x3f3f3f3f;
const int N=5e5+10;
int dis[N];
int n,m,cntn;
bool vis[N];
ll ans;
struct edge{
    int u,w;
};
vector<edge>G[N];
void add(int x,int y,int w){
    G[x].push_back({y,w});
    G[y].push_back({x,w});
}
struct node{
    int u,w;
    friend bool operator <(node a,node b){
        return a.w>b.w;
    }
}a[N];
priority_queue<node>Q;
void Prim(){
    cntn=n;
    dis[1]=0;
    Q.push({1,0});
    while(!Q.empty()){
        auto [u,w]=Q.top();
        Q.pop();
        if(vis[u]) continue;
        cntn--;
        ans+=w;
        vis[u]=1;
        for(int i=0;i<G[u].size();i++){
            auto [v,w]=G[u][i];
            if(vis[v]) continue;
            if(w<dis[v]){
                dis[v]=w;
                Q.push({v,dis[v]});
            }
        }
    }
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(0),cout.tie(0);
    cin>>n>>m;
    memset(dis,0x3f,sizeof(dis));
    for(int i=1;i<=m;i++){
        int x,y,w;cin>>x>>y>>w;
        add(x,y,w);
        add(y,x,w);
    }
    Prim();
    if(cntn==0) cout<<ans<<endl;
    else cout<<"orz"<<endl;
    return 0;
}
```

4.5 Kruskal 重构树

Kruskal 重构树上任意两点 lca 权值代表两点间最大边权最小值

Listing 41: Kruskal 重构树

```
struct DSU{
    vector<ll>f;
    DSU(){}
    DSU(int n){
        f.resize(n+1);
        iota(f.begin(),f.end(),0);
    };
    int find(int x){
        return x==f[x]?x:f[x]=find(f[x]);
    }
    bool merge(int x,int y){
        int fx=find(x),fy=find(y);
        if(fx==fy) return false;
        f[fx]=fy;
        return true;
    }
};
struct Edge{
    int x,y;ll w;
    Edge(){};
    Edge(int x,int y,ll w):x(x),y(y),w(w){};
    friend bool operator<(Edge A,Edge B){
        return A.w<B.w;
    }
};
inline void solve(){
    int n,m;
    cin>>n>>m;
    DSU dsu(n<<1);
    vector<vector<int>>G(n<<1|1);
    vector<Edge>a;
    vector<ll>val(n<<1|1);//存新节点(边)权值
    for(int i=1;i<=m;i++){
        int x,y;ll w;
        cin>>x>>y>>w;
        a.emplace_back(x,y,w);
    }
    sort(a.begin(),a.end());
    int tot=n;
    for(auto &[x,y,w]:a){
        int fx=dsu.find(x),fy=dsu.find(y);
        if(fx!=fy){
            ++tot;
            dsu.merge(fx,tot);
            dsu.merge(fy,tot);
            G[tot].push_back(fx);
            G[tot].push_back(fy);
            val[tot]=w;
        }
    }
    vector<int>dfn(n<<1|1);
    vector<ll>d(n<<1|1);
    vector<vector<int>>st(n<<1|1,vector<int>(20));
    int top=0;
    auto dfs=[&](auto &&dfs,int u,int fa) ->void {
        d[u]=d[fa]+1;
        dfn[u]=++top;
        st[dfn[u]][0]=fa;
        for(int v:G[u]) dfs(dfs,v,u);
    };
    for(int i=1;i<=(n<<1);i++){
        if(dsu.find(i)==i){
            dfs(dfs,i,0);
        }
    }
    for(int i=1;i<20;i++){
        for(int j=1;j+(1<<i)-1<=(n<<1);j++){
            int L=st[j][i-1];
            int R=st[j+(1<<i-1)][i-1];
            st[j][i]=(d[L]<d[R])?L:R;
        }
    }
    auto lca=[&](int l,int r) ->int {
        if(l==r) return l;
        l=dfn[l],r=dfn[r];
        if(l>r) swap(l,r);
        ++l;
        int len=__lg(r-l+1);
        int L=st[l][len];
        int R=st[r-(1<<len)+1][len];
        return (d[L]<d[R])?L:R;
    };
    int q;cin>>q;
    for(int i=1;i<=q;i++){
        int x,y;cin>>x>>y;
        if(dsu.find(x)!=dsu.find(y))
cout<<"impossible"<<endl;
        else cout<<val[lca(x,y)]<<endl;
    }
}
```

4.6 欧拉回路

Listing 42: 有向图欧拉回路

```
void solve(){
    int n,m;
    cin>>n>>m;
    vector<vector<int>>G(n+1);
    vector<int>nxt(n+1);
    vector<int>in(n+1),out(n+1);
    for(int i=1;i<=m;i++){
        int x,y;
        cin>>x>>y;
        G[x].emplace_back(y);
        ++in[y],++out[x];
    }
    auto find=[&]() ->int {
        int cnt1=0,cnt2=0;
        for(int i=1;i<=n;i++){
```

```
            if(in[i]-out[i]<-1||in[i]-out[i]>1)
return -1;
            cnt1+=(in[i]-out[i]==-1);
            cnt2+=(in[i]-out[i]==1);
        }
        if(cnt1+cnt2==0){
            for(int i=1;i<=n;i++){
                if(out[i]>0) return i;
            }
        }//存在欧拉回路,需要保证图联通
        else if(cnt1==1&&cnt2==1){
            for(int i=1;i<=n;i++){
                if(out[i]-in[i]==1) return i;
            }
        }//存在欧拉路径
        return -1;
    };//找到起点
    int s=find();
    if(s==-1){
        cout<<"No"<<endl;
        return;
    }
    vector<int>res;
    for(int i=1;i<=n;i++)
sort(G[i].begin(),G[i].end());
    auto dfs=[&](auto &&dfs,int u) ->void {
        while(nxt[u]<G[u].size()){
            int v=G[u][nxt[u]];
            nxt[u]++;//当前弧优化
            dfs(dfs,v);
        }
        res.push_back(u);
    };
    dfs(dfs,s);
    if(res.size()!=m+1){
        cout<<"No"<<endl;
        return;
    }
    reverse(res.begin(),res.end());
    for(int x:res) cout<<x<<' ';
    cout<<endl;
}
```

Listing 43: 无向图欧拉回路

```
void solve(){
    int m,n=500;cin>>m;
    vector<vector<pair<int,int>>>G(n+1);
    vector<int>in(n+1);
    for(int i=1;i<=m;i++){
        int x,y;cin>>x>>y;
        G[x].emplace_back(y,i);
        G[y].emplace_back(x,i);
        ++in[x],++in[y];
    }
    auto find=[&]() ->int {
        int cnt=0;
        for(int i=1;i<=n;i++){
            if(in[i]&1) ++cnt;
        }
        if(cnt!=2&&cnt!=0) return -1;
        else if(cnt==0) {
            for(int i=1;i<=n;i++){
                if(in[i]) return i;
            }
        }
        else if(cnt==2){
            for(int i=1;i<=n;i++){
                if(in[i]&1) return i;
            }
        }
        return -1;
    };
    int s=find();
    if(s==-1){
        cout<<"No"<<endl;
        return;
    }
    vector<int>nxt(n+1),vis(m+1),res;
    for(int i=1;i<=n;i++)
sort(G[i].begin(),G[i].end());
    auto dfs=[&](auto &&dfs,int u) ->void {
        while(nxt[u]<G[u].size()){
            int v=G[u][nxt[u]].first;
            int id=G[u][nxt[u]].second;
            ++nxt[u];
            if(vis[id]) continue;
            vis[id]=1;
            dfs(dfs,v);
        }
        res.push_back(u);
    };
    dfs(dfs,s);
    reverse(res.begin(),res.end());
    for(int x:res) cout<<x<<endl;
}
```

4.7 二分图

4.7.1 二分图染色

Listing 44: 二分图染色

```
void solve(){
    int n,m;
    cin>>n>>m;
    vector<vector<int>>G(n+1);
    for(int i=1;i<=m;i++){
        int x,y;cin>>x>>y;
        G[x].emplace_back(y);
        G[y].emplace_back(x);
    }
    int A=0,B=0;
    vector<int>col(n+1);
    auto dfs=[&](auto &dfs,int u) ->bool {
        bool f=true;
        for(int v:G[u]){
            if(!col[v]) {
```

```
                col[v]=3-col[u];
                if(col[v]==1) ++A;
                else ++B;
                f&=dfs(dfs,v);
            }
            else if(col[v]!=3-col[u])
f=false;//当前联通分量无法染色
        }
        return f;
    };
    ll ans=0;
    for(int i=1;i<=n;i++){
        if(!col[i]){
            A=B=0;
            col[i]=1;
            ++A;
            if(dfs(dfs,i)) ans+=max(A,B);
        }
    }
    cout<<ans<<endl;
}
```

4.7.2 二分图最大匹配 (匈牙利算法)

Listing 45: 二分图最大匹配 (匈牙利)

```
const int N=505;
vector<int>G[N];
int match[N];
bool vis[N];
bool dfs(int u){
    for(auto &v:G[u]){
        if(vis[v]) continue;
        vis[v]=1;
        if(!match[v]||dfs(match[v])){
            match[v]=u;
            return 1;
        }
    }
    return 0;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n,m,e;cin>>n>>m>>e;
    for(int i=1;i<=e;i++){
        int u,v;
        cin>>u>>v;
        G[u].push_back(v);
    }
    ll ans=0;
    for(int i=1;i<=n;i++) {
        // 每轮增广前清空右部标记
        fill(vis+1,vis+1+m,false);
        ans+=dfs(i);
    }
    cout<<ans<<endl;
    return 0;
}
```

4.7.3 二分图最大匹配 (Dinic)

Listing 46: Dinic 求二分图最大匹配

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const ll INF=4e18;
struct Dinic{
    struct E{
        int to;ll cap;int rev;
        //to:目标点,cap:容量,rev:反向边索引
    };//边集
    vector<vector<E>>g;
    vector<int>lv,it;//bfs层次, 当前弧优化迭代器
    int n;
    Dinic(int n):g(n),lv(n),it(n),n(n){}
    int add(int u,int v,ll c){
        g[u].push_back({v,c,(int)g[v].size()});
        g[v].push_back({u,0,(int)g[u].size()-1});
        return (int)g[u].size()-1;
    }//添加边,返回正向边索引
    void bfs(int s){
        fill(lv.begin(),lv.end(),-1);
        queue<int>q;
        lv[s]=0;
        q.push(s);
        while(!q.empty()){
            int v=q.front();
            q.pop();
            for(auto &e:g[v]){
                //只走残余容量为正的边,且未访问
                if(e.cap>0&&lv[e.to]<0){
                    lv[e.to]=lv[v]+1;
                    q.push(e.to);
                }
            }
        }
    }//使用队列计算层次图
    ll dfs(int v,int t,ll f){
        if(v==t) return f;
        for(int &i=it[v];i<(int)g[v].size();i++){
            E &e=g[v][i];
            if(e.cap>0&&lv[v]<lv[e.to]){
                ll d=dfs(e.to,t,min(f,e.cap));
                if(d>0){
                    e.cap-=d;
                    g[e.to][e.rev].cap+=d;
                    return d;
                }
            }
        }
        return 0;
    }//递归寻找增广路
    ll maxflow(int s,int t){
        ll f=0;
        while(true){
```

```
            bfs(s);//构建层次图
            if(lv[t]<0){
                return f;
            }//汇点不可达,返回

fill(it.begin(),it.end(),0);//重置当前弧
            ll d;
            while((d=dfs(s,t,INF))>0) f+=d;
        }
    }
    vector<char>cut(int s){
        vector<char>vis(n);
        queue<int>Q;
        Q.emplace(s);
        vis[s]=1;
        while(!Q.empty()){
            int u=Q.front();
            Q.pop();
            for(auto &e:g[u]){
                if(e.cap>0&&!vis[e.to]){
                    vis[e.to]=1;
                    Q.emplace(e.to);
                }
            }
        }
        return vis;
    }
};
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int m,n;
    cin>>m>>n;
    int S=m+n,T=S+1;
    Dinic din(T+1);
    for(int i=0;i<m;i++)
din.add(S,i,1);//S->m个左部节点
    for(int i=0;i<n-m;i++)
din.add(i+m,T,1);//n-m个右部节点->T
    int a,b;
    while(cin>>a>>b&&!(a==-1&&b==-1)){
        --a,--b;
        din.add(a,b,1);
    }
    ll ans=din.maxflow(S,T);//求解最大流
    cout<<ans<<endl;
    for(int i=m;i<n;i++){
        for(auto &e:din.g[i]){
            if(e.to<m&&e.cap>0){
                cout<<e.to+1<<' '<<i+1<<endl;
            }
        }
    }
    return 0;
}
```

4.7.4 二分图最大权匹配 (KM)

KM 算法（匈牙利算法的带权版）：左部 n 个点、右部 m 个点（要求 $n \le m$），求权和最大的匹配。不存在的边把边权填成足够小的负数（代码里是 `NEG`），并保证存在完美匹配即可，复杂度 $O(n^2 m)$。若题目允许" 不匹配"，把每条边权改成 $\max(0, w)$ 再跑完美匹配即可。输入：第一行 $n\ m\ e$，随后 e 行 $u\ v\ x$（左部 u 到右部 v、边权 x）；输出：第一行最大权完美匹配的权值和，第二行每个左部点匹配到的右部点（为 0 说明该点没匹配上，即数据不存在完美匹配）。

Listing 47: 二分图最大权匹配 (KM)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 505;
const ll INF = (ll)4e18;
// 不存在的边用 NEG
表示：足够小（不会被选中），又不会让 lx+ly-w 溢出 ll
const ll NEG = -(ll)4e15;

// 二分图最大权完美匹配（KM 算法，O(n^2 m)）：左部 n
点、右部 m 点（要求 n <= m），
// 不存在的边填
NEG。输入输出格式、变体改法与用法见正文。
int n, m, e;
ll w[MAXN][MAXN];
ll lx[MAXN], ly[MAXN], slack[MAXN];  //
左右部顶点标号、slack
int match[MAXN], matchL[MAXN];       //
match[j]=右部 j 匹配的左部点
int pre[MAXN];
bool visy[MAXN];

void km() {
    fill(lx + 1, lx + n + 1, 0);
    fill(ly + 1, ly + m + 1, 0);
    fill(match, match + m + 1, 0);
    // 初始标号：左部取关联边的最大权，右部取 0
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) lx[i] =
max(lx[i], w[i][j]);
    for (int u = 1; u <= n; ++u) {
        fill(slack, slack + m + 1, INF);
        fill(visy, visy + m + 1, false);
        match[0] = u;                        // 0
号虚点：本轮要增广的左部点
        int y = 0;
        do {
            visy[y] = true;                  // y
进入交错树
            int x = match[y], ny = 0;
```

```
            ll d = INF;
            for (int j = 1; j <= m; ++j) {
                if (visy[j]) continue;
                ll t = lx[x] + ly[j] - w[x][j];
// 松驰量
                if (t < slack[j]) { slack[j] = t;
pre[j] = y; }
                if (slack[j] < d) { d = slack[j];
ny = j; }
            }
            // 更新标号：交错树上的点 lx 减 d、ly
加 d，其余 slack 减 d
            for (int j = 0; j <= m; ++j) {
                if (visy[j]) { lx[match[j]] -= d;
ly[j] += d; }
                else slack[j] -= d;
            }
            y = ny;
        } while (match[y]);                 //
直到找到一个未匹配的右部点
        while (y) { int py = pre[y]; match[y] =
match[py]; y = py; }  // 沿 pre 回溯增广
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m >> e;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) w[i][j] = NEG;
    for (int k = 0; k < e; ++k) {
        int u, v; ll x;
        cin >> u >> v >> x;
        w[u][v] = x;
    }
    km();
    // 没有完美匹配时，KM 会用 NEG
占位；这些左部点应当输出 0（未匹配）
    for (int j = 1; j <= m; ++j)
        if (match[j] && w[match[j]][j] == NEG)
match[j] = 0;
    ll ans = 0;
    for (int i = 1; i <= n; ++i) matchL[i] = 0;
    for (int j = 1; j <= m; ++j)
        if (match[j]) { ans += w[match[j]][j];
matchL[match[j]] = j; }
    cout << ans << '\n';
    for (int i = 1; i <= n; ++i) cout << matchL[i]
<< " \n"[i == n];
    return 0;
}
```

4.8 最大流与最小费用最大流

Dinic 支持有向边非负容量，`cut(s)` 返回残量网络中从源点可达的一侧。最小费用最大流用 SPFA 找增广路，支持负费用边；要求残量网络中不存在从源点可达的负费用环，最坏复杂度较高，单位容量场景尤其要留意增广次数。

Listing 48: Dinic 通用最大流

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct Dinic {
    struct E {
        int to, rev;
        i64 cap;
    };

    int n;
    vector<vector<E>> g;
    vector<int> lv, it;

    explicit Dinic(int n) : n(n), g(n), lv(n),
it(n) {}

    void add(int u, int v, i64 c) {
        int fu = g[u].size();
        int fv = g[v].size() + (u == v);
        g[u].push_back({v, fv, c});
        g[v].push_back({u, fu, 0});
    }

    bool bfs(int s, int t) {
        fill(lv.begin(), lv.end(), -1);
        queue<int> q;
        lv[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const auto &e : g[u]) {
                if (e.cap > 0 && lv[e.to] == -1) {
                    lv[e.to] = lv[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return lv[t] != -1;
    }

    i64 dfs(int u, int t, i64 f) {
        if (u == t || f == 0) {
            return f;
        }
        for (int &i = it[u]; i < (int)g[u].size();
++i) {
            E &e = g[u][i];
            if (e.cap == 0 || lv[e.to] != lv[u] +
1) {
                continue;
```

```
            }
            i64 d = dfs(e.to, t, min(f, e.cap));
            if (d == 0) {
                continue;
            }
            e.cap -= d;
            g[e.to][e.rev].cap += d;
            return d;
        }
        return 0;
    }

    i64 flow(int s, int t, i64 lim = numeric_limits<i64>::max()) {
        if (s == t || lim <= 0) {
            return 0;
        }
        i64 f = 0;
        while (f < lim && bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (f < lim) {
                i64 d = dfs(s, t, lim - f);
                if (d == 0) {
                    break;
                }
                f += d;
            }
        }
        return f;
    }

    // 残量网络中从 s 可达的一侧
    vector<bool> cut(int s) const {
        vector<bool> vis(n);
        queue<int> q;
        vis[s] = true;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const auto &e : g[u]) {
                if (e.cap > 0 && !vis[e.to]) {
                    vis[e.to] = true;
                    q.push(e.to);
                }
            }
        }
        return vis;
    }
};
```

Listing 49: SPFA 最小费用最大流

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct MCMF {
    struct E {
        int to, rev;
        i64 cap, cost;
    };

    int n;
    vector<vector<E>> g;

    explicit MCMF(int n) : n(n), g(n) {}

    void add(int u, int v, i64 cap, i64 cost) {
        int fu = g[u].size();
        int fv = g[v].size() + (u == v);
        g[u].push_back({v, fv, cap, cost});
        g[v].push_back({u, fu, 0, -cost});
    }

    // 最大流中的最小费用；要求残量网络无源点可达负环
    pair<i64, i64> flow(int s, int t, i64 lim = numeric_limits<i64>::max()) {
        if (s == t || lim <= 0) {
            return {0, 0};
        }
        constexpr i64 INF = numeric_limits<i64>::max() / 4;
        i64 f = 0, c = 0;
        vector<i64> dis(n);
        vector<int> pv(n), pe(n);
        vector<bool> inq(n);

        while (f < lim) {
            fill(dis.begin(), dis.end(), INF);
            fill(inq.begin(), inq.end(), false);
            queue<int> q;
            dis[s] = 0;
            q.push(s);
            inq[s] = true;

            while (!q.empty()) {
                int u = q.front();
                q.pop();
                inq[u] = false;
                for (int i = 0; i < (int)g[u].size(); ++i) {
                    const E &e = g[u][i];
                    if (e.cap == 0 || dis[e.to] <= dis[u] + e.cost) {
                        continue;
                    }
                    dis[e.to] = dis[u] + e.cost;
                    pv[e.to] = u;
                    pe[e.to] = i;
                    if (!inq[e.to]) {
                        inq[e.to] = true;
                        q.push(e.to);
                    }
                }
            }

```

```
            if (dis[t] == INF) {
                break;
            }
            i64 d = lim - f;
            for (int v = t; v != s; v = pv[v]) {
                d = min(d, g[pv[v]][pe[v]].cap);
            }
            for (int v = t; v != s; v = pv[v]) {
                E &e = g[pv[v]][pe[v]];
                e.cap -= d;
                g[v][e.rev].cap += d;
            }
            f += d;
            c += d * dis[t];
        }
        return {f, c};
    }
};
```

4.9 差分约束

给定 n 个变量 $x_1, x_2, \ldots, x_n$ 与 m 个差分约束 $x_c - x_{c'} \leq y$。

求任意一组满足所有约束的可行解。无解输出 NO。

Listing 50: spfa 差分约束

```
const int N=2e5+5;
struct node{
    int u;ll w;
};
vector<node>G[N];
int cnt[N];
bool inq[N];
ll dis[N];
int n,m;
bool spfa(){
    fill(dis+1,dis+1+n,(ll)4e18);
    dis[0]=0;
    fill(inq,inq+n+1,false);
    fill(cnt,cnt+n+1,0);
    queue<int>Q;
    Q.push(0);
    inq[0]=1;
    cnt[0]=1;
    while(!Q.empty()){
        int u=Q.front();
        Q.pop();
        inq[u]=false;
        for(auto &[v,w]:G[u]){
            if(dis[v]>dis[u]+w){
                dis[v]=dis[u]+w;
                if(!inq[v]){
                    cnt[v]++;
                    if(cnt[v]>n) return false;
                    Q.push(v);
                    inq[v]=1;
                }
            }
        }
    }
    return true;
}
void solve(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        int x,y;ll w;cin>>x>>y>>w;
        G[y].push_back({x,w});
    }
    for(int i=1;i<=n;i++){
        G[0].push_back({i,0});
    }
    if(spfa()){
         for(int i=1;i<=n;i++) cout<<dis[i]<<' ';
        cout<<endl;
    }
    else cout<<"NO"<<endl;
}
```

4.10 缩点

4.10.1 SCC(强连通分量)

Listing 51: SCC

```
struct SCC{
    int n;
    vector<vector<int>>g;
    vector<vector<int>>ng;
    vector<int>in;
    vector<int>dfn,low,stk,scc;
    vector<int>sz;
    vector<bool>instk;
    int idx,cnt,top;
    SCC(int n_=0){
        n=n_;
        g.assign(n+1,{});
        dfn.assign(n+1,0);
        low.assign(n+1,0);
        instk.assign(n+1,false);
        scc.assign(n+1,-1);
        stk.assign(n+1,0);
        idx=cnt=top=0;
    }
    void add(int u,int v){
        g[u].emplace_back(v);
    }
    void dfs(int u){
        dfn[u]=low[u]=(++idx);
        stk[++top]=u;
        instk[u]=true;
        for(int v:g[u]){
            if(!dfn[v]){
                dfs(v);
                low[u]=min(low[u],low[v]);
            }
```

```
            else if(instk[v]){
                low[u]=min(low[u],dfn[v]);
            }
        }
        if(low[u]==dfn[u]){
            int v;
            do{
                v=stk[top--];
                instk[v]=false;
                scc[v]=cnt;//编号从0开始
            }while(v!=u);
            cnt++;
        }
    }
    void work(){
        for(int i=1;i<=n;i++){
            if(!dfn[i]) dfs(i);
        }
        ng.assign(cnt,{});
        sz.assign(cnt,0);
        for(int u=1;u<=n;u++){
            sz[scc[u]]++;
            for(int v:g[u]){
                if(scc[u]!=scc[v]){

ng[scc[u]].emplace_back(scc[v]);
                }
            }
        }
        for(auto &e:ng){
            sort(e.begin(),e.end());

e.erase(unique(e.begin(),e.end()),e.end());
        }
        // 去重后重新统计入度
        in.assign(cnt,0);
        for(int u=0;u<cnt;u++){
            for(int v:ng[u]) ++in[v];
        }
    }
};
```

4.10.2 DECC(边双联通分量)

Listing 52: DECC

```
const int N=2e5+5;
vector<pair<int,int>>G[N];
struct DECC{
    vector< vector<pair<int,int> >>G;
    vector< vector<int> >B;// 存储所有边双连通分量
    vector<int>dfn,low;//时间戳
    stack<int>st;
    vector<char>vis;
    vector<int>mp;// 每个点属于哪个分量
    int cnt,n,id;
    DECC(int n):n(n),cnt(0),id(0){
        G.resize(n+1);
        low.resize(n+1);
        dfn.resize(n+1);
        mp.resize(n+1);
    }
    void add(int x,int y){
        ++id;
        G[x].push_back({y,id});
        G[y].push_back({x,id});
    }
    void tarjan(int u,int fa){
        dfn[u]=low[u]=(++cnt);
        st.push(u);
        for(auto &[v,id]:G[u]){
            if(!dfn[v]){
                tarjan(v,id);
                low[u]=min(low[u],low[v]);
            }
            else if(id!=fa&&dfn[v]<dfn[u]){
                low[u]=min(low[u],dfn[v]);
            }
        }
        if(dfn[u]==low[u]){
            vector<int>tmp;
            int y;
            do{
                y=st.top();
                mp[y]=B.size();
                st.pop();
                tmp.push_back(y);
            }while(y!=u);
            B.push_back(tmp);
        }
    }
};
void solve(){
    int n,m;
    cin>>n>>m;
    DECC bcc(n+1);
    for(int i=1;i<=m;i++){
        int x,y;cin>>x>>y;
        bcc.add(x,y);
    }
    for(int i=1;i<=n;i++){
        if(!bcc.dfn[i]) bcc.tarjan(i,0);
    }
    cout<<bcc.B.size()<<endl;
    for(auto &p:bcc.B){
        cout<<p.size()<<' ';
        for(int &x:p) cout<<x<<' ';
        cout<<endl;
    }
}
```

4.11 two-sat

Listing 53: two-sat

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
struct TwoSAT {
    int n;
    vector<vector<int>> G;
    vector<int> dfn, low, scc_id;
    vector<bool> in_stk;
    stack<int> st;
    int tim, scc_cnt;
    TwoSAT(int n) : n(n), G(2 * n), dfn(2 * n), low(2 * n), scc_id(2 * n), in_stk(2 * n) {
        tim = scc_cnt = 0;
    }
    void add_edge(int u, int v) {
        G[u].push_back(v);
    }
    // i, j 从 1 开始, a, b ∈ {0, 1}
    void add_clause(int i, bool a, int j, bool b) {
        --i; --j;
        // 2*i: false , 2*i+1: true
        add_edge((i << 1) | (!a), (j << 1) | b); // !a -> b
        add_edge((j << 1) | (!b), (i << 1) | a); // !b -> a
    }
    void tarjan(int u) {
        dfn[u] = low[u] = ++tim;
        st.push(u);
        in_stk[u] = true;
        for (int v : G[u]) {
            if (!dfn[v]) {
                tarjan(v);
                low[u] = min(low[u], low[v]);
            } else if (in_stk[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u]) {
            ++scc_cnt;
            int v;
            do {
                v = st.top(); st.pop();
                in_stk[v] = false;
                scc_id[v] = scc_cnt;
            } while (v != u);
        }
    }

    bool solve(vector<int> &ans) {
        ans.assign(n + 1, 0);
        for (int i = 0; i < 2 * n; ++i)
            if (!dfn[i]) tarjan(i);

        for (int i = 0; i < n; ++i) {
            if (scc_id[2 * i] == scc_id[2 * i + 1])
                return false;   // 无解
            ans[i + 1] = (scc_id[2 * i] < scc_id[2 * i + 1]) ? 0 : 1;
        }
        return true;
    }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TwoSAT solver(n);
    for (int k = 0; k < m; ++k) {
        int i, a, j, b;
        cin >> i >> a >> j >> b;
        solver.add_clause(i, a, j, b);
    }
    vector<int> ans;
    if (solver.solve(ans)) {
        cout << "POSSIBLE\n";
        for (int i = 1; i <= n; ++i)
            cout << ans[i] << " \n"[i == n];
    } else {
        cout << "IMPOSSIBLE\n";
    }
    return 0;
}
```

5 字符串

5.1 字符串哈希

双哈希前缀，区间统一使用左闭右开 $[l, r)$；哈希相等是高概率判等，不是严格证明。

Listing 54: 双模数区间字符串哈希

```cpp
#include <bits/stdc++.h>
using namespace std;

struct HV {
    uint32_t x, y;

    bool operator==(const HV &o) const {
        return x == o.x && y == o.y;
    }
};

struct Hash {
    static constexpr uint32_t mod1 = 1'000'000'007;
    static constexpr uint32_t mod2 = 1'000'000'009;
    static constexpr uint32_t base1 = 911'382'323;
    static constexpr uint32_t base2 = 972'663'749;

    vector<uint32_t> p1, p2, h1, h2;

    explicit Hash(string_view s)
        : p1(s.size() + 1, 1), p2(s.size() + 1, 1),
          h1(s.size() + 1), h2(s.size() + 1) {
        for (int i = 0; i < (int)s.size(); ++i) {
            uint32_t c = (unsigned char)s[i] + 1;
            p1[i + 1] = (uint64_t)p1[i] * base1 %
mod1;
            p2[i + 1] = (uint64_t)p2[i] * base2 %
mod2;
            h1[i + 1] = ((uint64_t)h1[i] * base1 +
c) % mod1;
            h2[i + 1] = ((uint64_t)h2[i] * base2 +
c) % mod2;
        }
    }

    // 子串 s[l..r) 的哈希
    HV get(int l, int r) const {
        uint32_t x = (h1[r] + mod1 -
(uint64_t)h1[l] * p1[r - l] % mod1) % mod1;
        uint32_t y = (h2[r] + mod2 -
(uint64_t)h2[l] * p2[r - l] % mod2) % mod2;
        return {x, y};
    }
};
```

5.2 AC 自动机

先对所有非空小写模式串调用 `add`，再调用 `build`；`query(text)` 按加入顺序返回每个模式串在文本中的出现次数，重复模式串会分别返回。构建复杂度 $O(26V)$，单次统计为 $O(|text| + V + P)$，其中 V 是自动机节点数、P 是模式串数。

Listing 55: Aho-Corasick 多模式匹配

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

// 多模式串匹配：统计每个模式串在文本中的出现次数。
struct AC {
    static constexpr int S = 26;

    struct Node {
        array<int, S> ch;
        int fail = 0;

        Node() {
            ch.fill(-1);
        }
    };

    vector<Node> tr;
    vector<int> ed, ord;
    bool ok = false;

    AC() : tr(1) {}

    int add(string_view s) {
        if (ok) {
            throw logic_error("add all patterns
before build()");
        }
        if (s.empty()) {
            throw invalid_argument("patterns must
be non-empty");
        }
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            assert(0 <= x && x < S);
            int v = tr[u].ch[x];
            if (v == -1) {
                v = tr.size();
                tr[u].ch[x] = v;
                tr.emplace_back();
            }
            u = v;
        }
        ed.push_back(u);
        return (int)ed.size() - 1;
    }

    void build() {
        if (ok) {
```

```
            return;
        }
        queue<int> q;
        ord.clear();
        ord.push_back(0);
        for (int c = 0; c < S; ++c) {
            int u = tr[0].ch[c];
            if (u == -1) {
                tr[0].ch[c] = 0;
            } else {
                tr[u].fail = 0;
                q.push(u);
                ord.push_back(u);
            }
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int c = 0; c < S; ++c) {
                int v = tr[u].ch[c];
                if (v == -1) {
                    tr[u].ch[c] =
tr[tr[u].fail].ch[c];
                } else {
                    tr[v].fail =
tr[tr[u].fail].ch[c];
                    q.push(v);
                    ord.push_back(v);
                }
            }
        }
        ok = true;
    }

    // 按 add 的顺序返回每个模式串的出现次数
    vector<i64> query(string_view s) const {
        if (!ok) {
            throw logic_error("call build() before
query()");
        }
        vector<i64> cnt(tr.size());
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            assert(0 <= x && x < S);
            u = tr[u].ch[x];
            ++cnt[u];
        }
        for (int i = (int)ord.size() - 1; i > 0;
--i) {
            int u = ord[i];
            cnt[tr[u].fail] += cnt[u];
        }
        vector<i64> ans(ed.size());
        for (int i = 0; i < (int)ed.size(); ++i) {
            ans[i] = cnt[ed[i]];
        }
        return ans;
    }
};
```

5.3 KMP

Kmp 返回前缀函数；kmp 返回模式串在文本中的所有 0 起始位置。

Listing 56: KMP1

```
#include <bits/stdc++.h>
using namespace std;

// pi[i] 是最长真前缀且同时是 s[0..i] 后缀的长度。
vector<int> Kmp(string_view s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) {
            j = pi[j - 1];
        }
        if (s[i] == s[j]) {
            ++j;
        }
        pi[i] = j;
    }
    return pi;
}
```

Listing 57: KMP2

```
#include <bits/stdc++.h>
using namespace std;

// 返回 t 在 s 中所有 0 起始位置。
vector<int> kmp(string_view s, string_view t) {
    int n = s.size(), m = t.size();
    if (m == 0) {
        vector<int> ans(n + 1);
        iota(ans.begin(), ans.end(), 0);
        return ans;
    }

    vector<int> pi(m);
    for (int i = 1; i < m; ++i) {
        int j = pi[i - 1];
        while (j > 0 && t[i] != t[j]) {
            j = pi[j - 1];
        }
        if (t[i] == t[j]) {
            ++j;
        }
        pi[i] = j;
    }

    vector<int> ans;
    for (int i = 0, j = 0; i < n; ++i) {
        while (j > 0 && s[i] != t[j]) {
            j = pi[j - 1];
```

```
        }
        if (s[i] == t[j]) {
            ++j;
        }
        if (j == m) {
            ans.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return ans;
}
```

5.4 Z 函数 (扩展 KMP)

z[i] 表示从位置 i 开始的后缀与整个串的最长公共前缀长度。

Listing 58: Z 函数

```
#include <bits/stdc++.h>
using namespace std;

// z[i] 是 s 与 s.substr(i) 的 LCP 长度; z[0]=|s|。
vector<int> zf(string_view s) {
    int n = s.size();
    vector<int> z(n);
    if (n == 0) {
        return z;
    }
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            ++z[i];
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

5.5 Manacher(马拉车)

奇偶半径分别存于 od 和 ev；check(l,r) 判断左闭右开区间。

Listing 59: 最长回文子串与区间回文查询

```
#include <bits/stdc++.h>
using namespace std;

struct Manacher {
    string s;
    vector<int> od, ev;

    explicit Manacher(string s) : s(move(s)), od(this->s.size()), ev(this->s.size()) {
        int n = this->s.size();
        for (int i = 0, l = 0, r = -1; i < n; ++i) {
            int k = (i > r ? 1 : min(od[l + r - i], r - i + 1));
            while (i - k >= 0 && i + k < n && this->s[i - k] == this->s[i + k]) {
                ++k;
            }
            od[i] = k;
            if (i + k - 1 > r) {
                l = i - k + 1;
                r = i + k - 1;
            }
        }
        for (int i = 0, l = 0, r = -1; i < n; ++i) {
            int k = (i > r ? 0 : min(ev[l + r - i + 1], r - i + 1));
            while (i - k - 1 >= 0 && i + k < n && this->s[i - k - 1] == this->s[i + k]) {
                ++k;
            }
            ev[i] = k;
            if (i + k - 1 > r) {
                l = i - k;
                r = i + k - 1;
            }
        }
    }

    int maxlen() const {
        int ans = 0;
        for (int x : od) {
            ans = max(ans, 2 * x - 1);
        }
        for (int x : ev) {
            ans = max(ans, 2 * x);
        }
        return ans;
    }

    // s[l..r) 是否回文
    bool check(int l, int r) const {
        int len = r - l;
        if (len <= 1) {
            return true;
        }
        if (len & 1) {
            return od[(l + r - 1) / 2] >= len / 2 + 1;
        }
        return ev[(l + r) / 2] >= len / 2;
    }
};
```

5.6 回文自动机 (PAM)

在线加入小写字符，`add(c)` 返回当前前缀的回文后缀个数；节点数减去两个根的数量，即本质不同回文子串数。`counts()` 沿失配边汇总每个节点代表的回文串出现次数。

Listing 60: PAM(回文自动机)

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct PAM {
    struct Node {
        int len = 0;
        int fail = 0;
        int num = 0;        // 以该节点结尾的回文后缀个数
        i64 cnt = 0;        // 出现次数
        array<int, 26> ch;

        Node() {
            ch.fill(-1);
        }
    };

    vector<Node> tr;
    string s;
    int last;

    PAM() {
        tr.resize(2);
        tr[0].len = -1;     // 奇根
        tr[0].fail = 0;
        tr[1].len = 0;      // 偶根
        tr[1].fail = 0;
        last = 1;
    }

    int getfail(int u, int pos, char c) const {
        while (pos - 1 - tr[u].len < 0 || s[pos - 1 - tr[u].len] != c) {
            u = tr[u].fail;
        }
        return u;
    }

    // 加入一个小写字符，返回当前前缀的回文后缀个数
    int add(char c) {
        int x = c - 'a';
        assert(0 <= x && x < 26);
        s.push_back(c);
        int pos = (int)s.size() - 1;
        int p = getfail(last, pos, c);

        if (tr[p].ch[x] == -1) {
            int cur = tr.size();
            tr.emplace_back();
            tr[cur].len = tr[p].len + 2;
            if (tr[cur].len == 1) {
                tr[cur].fail = 1;
            } else {
                int q = getfail(tr[p].fail, pos, c);
                tr[cur].fail = tr[q].ch[x];
            }
            tr[cur].num = 1 + tr[tr[cur].fail].num;
            tr[p].ch[x] = cur;
        }

        last = tr[p].ch[x];
        ++tr[last].cnt;
        return tr[last].num;
    }

    int size() const {
        return (int)tr.size() - 2;
    }

    // 沿 fail 边汇总每个节点的出现次数；两个根的下标为 0/1
    vector<i64> counts() const {
        vector<int> ord(tr.size());
        iota(ord.begin(), ord.end(), 0);
        sort(ord.begin(), ord.end(), [&](int a, int b) {
            return tr[a].len > tr[b].len;
        });
        vector<i64> cnt(tr.size());
        for (int i = 0; i < (int)tr.size(); ++i) {
            cnt[i] = tr[i].cnt;
        }
        for (int u : ord) {
            if (u >= 2) {
                cnt[tr[u].fail] += cnt[u];
            }
        }
        return cnt;
    }
};
```

5.7 后缀数组与 LCP

`SA` 返回按字典序排列的后缀起点（0 起始）；`LCP[i]` 是排序后第 $i-1$ 与第 i 个后缀的 LCP，`LCP[0]=0`。构造后缀数组 $O(n \log n)$，LCP 为 $O(n)$。

Listing 61: 倍增后缀数组与 Kasai LCP

```cpp
#include <bits/stdc++.h>
using namespace std;

// 返回后缀起点按字典序排列的数组，O(n log n)
vector<int> SA(string_view s) {
```

```
    int n = s.size();
    int m = n + 1;                  // 末尾补一个小于所有字符的哨兵
    vector<int> v(m), sa(m), c(m);
    for (int i = 0; i < n; ++i) {
        v[i] = (unsigned char)s[i] + 1;
    }

    vector<int> cnt(max(m, 257));
    for (int x : v) {
        ++cnt[x];
    }
    for (int i = 1; i < (int)cnt.size(); ++i) {
        cnt[i] += cnt[i - 1];
    }
    for (int i = m - 1; i >= 0; --i) {
        sa[--cnt[v[i]]] = i;
    }

    int cc = 1;
    for (int i = 1; i < m; ++i) {
        if (v[sa[i]] != v[sa[i - 1]]) {
            ++cc;
        }
        c[sa[i]] = cc - 1;
    }

    vector<int> sh(m), nc(m);
    for (int len = 1; len < m; len <<= 1) {
        for (int i = 0; i < m; ++i) {
            sh[i] = sa[i] - len;
            if (sh[i] < 0) {
                sh[i] += m;
            }
        }
        cnt.assign(cc, 0);
        for (int x : sh) {
            ++cnt[c[x]];
        }
        for (int i = 1; i < cc; ++i) {
            cnt[i] += cnt[i - 1];
        }
        for (int i = m - 1; i >= 0; --i) {
            sa[--cnt[c[sh[i]]]] = sh[i];
        }

        nc[sa[0]] = 0;
        int ncc = 1;
        for (int i = 1; i < m; ++i) {
            pair<int, int> a = {c[sa[i]], c[(sa[i] + len) % m]};
            pair<int, int> b = {c[sa[i - 1]], c[(sa[i - 1] + len) % m]};
            if (a != b) {
                ++ncc;
            }
            nc[sa[i]] = ncc - 1;
        }
        c.swap(nc);
        cc = ncc;
    }

    sa.erase(sa.begin());           // 去掉哨兵后缀
    return sa;
}

// lcp[i] = LCP(sa[i-1], sa[i]), lcp[0] = 0
vector<int> LCP(string_view s, const vector<int> & sa) {
    int n = s.size();
    vector<int> rk(n), lcp(n);
    for (int i = 0; i < n; ++i) {
        rk[sa[i]] = i;
    }
    for (int i = 0, k = 0; i < n; ++i) {
        int r = rk[i];
        if (r == 0) {
            k = 0;
            continue;
        }
        int j = sa[r - 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
            ++k;
        }
        lcp[r] = k;
        if (k > 0) {
            --k;
        }
    }
    return lcp;
}
```

5.8 后缀自动机

当前实现面向小写字母；`build(s)` 建立自动机，支持子串判定、本质不同子串数和按状态统计出现次数，构造与空间复杂度均为 $O(|s|)$。

Listing 62: Suffix Automaton

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct SAM {
    struct Node {
        array<int, 26> ch;
        int fail = -1;
        int len = 0;
        i64 cnt = 0;

        Node() {
            ch.fill(-1);
        }
    };
```

```

    vector<Node> tr;
    int last;

    SAM() {
        tr.emplace_back();
        last = 0;
    }

    void add(char c) {
        int x = c - 'a';
        assert(0 <= x && x < 26);
        int cur = tr.size();
        tr.emplace_back();
        tr[cur].len = tr[last].len + 1;
        tr[cur].cnt = 1;

        int p = last;
        while (p != -1 && tr[p].ch[x] == -1) {
            tr[p].ch[x] = cur;
            p = tr[p].fail;
        }
        if (p == -1) {
            tr[cur].fail = 0;
        } else {
            int q = tr[p].ch[x];
            if (tr[p].len + 1 == tr[q].len) {
                tr[cur].fail = q;
            } else {
                int clone = tr.size();
                tr.push_back(tr[q]);
                tr[clone].len = tr[p].len + 1;
                tr[clone].cnt = 0;
                while (p != -1 && tr[p].ch[x] ==
q) {
                    tr[p].ch[x] = clone;
                    p = tr[p].fail;
                }
                tr[q].fail = tr[cur].fail = clone;
            }
        }
        last = cur;
    }

    void build(string_view s) {
        tr.clear();
        tr.emplace_back();
        last = 0;
        for (char c : s) {
            add(c);
        }
    }

    bool find(string_view s) const {
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            if (x < 0 || x >= 26 || tr[u].ch[x] ==
-1) {
                return false;
            }
            u = tr[u].ch[x];
        }
        return true;
    }

    i64 distinct() const {
        i64 ans = 0;
        for (int i = 1; i < (int)tr.size(); ++i) {
            ans += tr[i].len - tr[tr[i].fail].len;
        }
        return ans;
    }

    // 沿 fail 边汇总每个状态的出现次数
    vector<i64> counts() const {
        vector<int> ord(tr.size());
        iota(ord.begin(), ord.end(), 0);
        sort(ord.begin(), ord.end(), [&](int a,
int b) {
            return tr[a].len > tr[b].len;
        });
        vector<i64> cnt(tr.size());
        for (int i = 0; i < (int)tr.size(); ++i) {
            cnt[i] = tr[i].cnt;
        }
        for (int u : ord) {
            if (u != 0) {
                cnt[tr[u].fail] += cnt[u];
            }
        }
        return cnt;
    }
};
```

6 动态规划

6.1 01 背包

Listing 63: 01 背包

```
void solve(){
    int w,n;cin>>w>>n;
    vector<int>a(n+1),b(n+1);
    vector<int>f(w+1);
    for(int i=1;i<=n;i++) cin>>a[i]>>b[i];
    for(int i=1;i<=n;i++) {
        for(int j=w;j>=a[i];j--){
            f[j]=max(f[j-a[i]]+b[i],f[j]);
        }
    }
    cout<<f[w]<<endl;
}
```

Listing 64: bitset 优化可行 01 背包

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
void solve(){
    int w, n; cin >> w >> n;
    bitset<10001> dp;
    dp[0] = 1;
    for(int i = 1; i <= n; i++) {
        int a; cin >> a;
        dp |= dp << a;
    }
    for(int j = w; j >= 0; j--) {
        if(dp[j]) {
            cout << j << endl;
            return;
        }
    }
}
```

bitset 版把容量上界固定为 10000，w 更大时需要改模板里的 bitset 大小。

Listing 65: 双向搜索优化纯搜索

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
vector<pair<ll, int>> gen(vector<ll> a) {
    vector<pair<ll, int>> res;
    int k = a.size();
    for (int mask=0; mask<(1<<k); mask++) {
        ll s=0;
        for (int i=0; i<k; i++) if (mask&(1<<i)) s
+= a[i];
        res.emplace_back(s, mask);
    }
    sort(res.begin(), res.end());
    return res;
}

int main() {
    int n; ll c;
    scanf("%d%lld", &n, &c);
    vector<ll> w(n);
    for (int i=0; i<n; i++) scanf("%lld", &w[i]);
    int mid = n/2;
    vector<ll> L(w.begin(), w.begin()+mid),
R(w.begin()+mid, w.end());
    auto ls = gen(L), rs = gen(R);
    ll maxw=0; int lm=0, rm=0;
    for (auto [sb, m]: rs) {
        if (sb > c) continue;
        ll rem = c - sb;
        int pos = upper_bound(ls.begin(),
ls.end(), make_pair(rem, (1<<mid)-1)) - ls.begin()
- 1;
        if (pos >=0 && ls[pos].first + sb > maxw) {
            maxw = ls[pos].first + sb;
            lm = ls[pos].second;
            rm = m;
        }
    }
    vector<int> ans(n);
    for (int i=0; i<mid; i++) ans[i] = (lm >> i) &
1;
    for (int i=0; i<n-mid; i++) ans[mid+i] = (rm >>
 i) & 1;
    printf("%lld\n", maxw);//最优代价
    for (int i=0; i<n; i++) printf("%d ",
ans[i]);//最优方案输出
    return 0;
}
```

6.2 完全背包

Listing 66: 朴素完全背包

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n,w;cin>>w>>n;
    vector<ll>a(n+1),b(n+1),f(w+1);
    for(int i=1;i<=n;i++) cin>>a[i]>>b[i];
    for(int i=1;i<=n;i++){
        for(int j=a[i];j<=w;j++){
            f[j]=max(f[j-a[i]]+b[i],f[j]);
        }
    }
    cout<<f[w]<<endl;
    return 0;
}
```

6.3 多重背包

Listing 67: 二进制优化多重背包

```
const ll INF=1e15;
void solve(){
    int n,w;
    cin>>n>>w;
    vector<ll>dp(w+1,INF);
    dp[0]=0;
    for(int i=1;i<=n;i++){
        int l; ll c;cin>>l>>c;
        for(ll k=1;c>0;k*=2){
            ll cur=min<ll>(c,k);
            for(int j=w;j>=1LL*cur*l;j--){

dp[j]=min(dp[j],dp[j-1LL*cur*l]+cur);
            }
            c-=cur;
        }
    }
    if(dp[w]>=INF) cout<<-1<<endl;
    else cout<<dp[w]<<endl;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    solve();
    return 0;
}
```

6.4 树上背包

25 上海市赛 I

题目大意

给定一棵以 1 为根的有根树，每个节点上的人要么是“诚实者”（恒说真话），要么是“说谎者”（恒说假话）。每个节点 i 的人声称：“以我为根的子树中，恰好有 a_i 个诚实者。”求所有满足以下逻辑自洽条件的身份分配方案数：

- 若节点 i 是诚实者，则其子树中诚实者的实际数量必须等于 a_i；
- 若节点 i 是说谎者，则其子树中诚实者的实际数量必须不等于 a_i。
- 第一行：一个整数 T（$1 \le T \le 5000$），表示数据组数。
- 每组数据第一行：一个正整数 n（$1 \le n \le 5000$，$\sum n \le 5000$），表示节点数量。
- 每组数据第二行：n 个非负整数 $a_1, a_2, \ldots, a_n$（$0 \le a_i \le n$），表示每个节点的陈述。
- 接下来 $n-1$ 行：每行两个正整数 u_i, v_i（$1 \le u_i, v_i \le n$），表示树的一条边，保证输入构成一棵以 1 为根的有根树。

输出格式

对每组数据输出一行一个整数，表示满足条件的真假分配情况数量，答案对 998244353 取模。

简要分析

采用树形背包 DP 求解，通过后序遍历合并子树状态，根据节点身份（诚实/说谎）约束转移，统计合法方案数。

状态定义

- $f[u][j]$：仅用节点 u 的子树，产生 j 个诚实者的合法方案数（不包含 u 自身）；
- $dp[u][j]$：用节点 u 的子树 + u 自身，产生 j 个诚实者的合法方案数；
- $siz[u]$：以 u 为根的子树大小。

状态转移

1. 初始化：$f[u][0] = 1$，$siz[u] = 1$；2. 子树合并：遍历子节点 v，用 ** 树形背包 ** 倒序枚举合并状态：

$$nxt[j+k] \mathrel{+}= f[u][j] \times dp[v][k] \pmod{mod}$$

用 nxt 更新 $f[u]$，并更新子树大小；3. 身份转移：- 说谎者：$dp[u][j] = f[u][j]$ $(j \neq a[u])$；- 诚实者：$dp[u][a[u]] = f[u][a[u]-1]$ $(a[u] \ge 1)$。

复杂度分析

代码虽为三重循环，本质是树形背包均摊复杂度：任意两个节点仅在其最近公共祖先处被枚举一次，总操作数等于子树大小乘积和，** 严格为 $O(n^2)$**；结合题目约束 $\sum n \le 5000$，总计算量不超过 2.5×10^7，可轻松通过。

答案统计

最终答案为根节点 1 的所有合法状态之和：$\sum_{i=0}^{siz[1]} dp[1][i] \pmod{998244353}$。

Listing 68: 树上背包

```
const ll mod=998244353;
void solve(){
    int n;cin>>n;
    vector<int>a(n+1);
    vector< vector<int> >G(n+1);
    for(int i=1;i<=n;i++) cin>>a[i];
    for(int i=1;i<n;i++){
        int x,y;cin>>x>>y;
        G[x].emplace_back(y);
        G[y].emplace_back(x);
    }
    vector<vector<ll>>dp(n+1,vector<ll>(n+1));//使用子树和当前点产生j个诚实者的合法情况
    vector<vector<ll>>f(n+1,vector<ll>(n+1));//仅使用子树产生j个诚实者的合法情况
    vector<int>siz(n+1);
    auto dfs=[&](auto &&dfs,int u,int fa) ->void {
        f[u][0]=siz[u]=1;
        for(int v:G[u]){
            if(v==fa) continue;
            dfs(dfs,v,u);
            vector<ll>nxt(siz[u]+siz[v]+1);
            for(int j=siz[u];j>=0;j--){
                for(int k=siz[v];k>=0;k--){
                    (nxt[j+k]+=(f[u][j]*dp[v][k]%mod))%=mod;
                }
            }
            f[u]=move(nxt);
            siz[u]+=siz[v];
        }
        for(int j=siz[u];j>=0;j--) dp[u][j]=f[u][j];
        dp[u][a[u]]=0;
        if(a[u]>=1&&a[u]<=siz[u]) {
            dp[u][a[u]]=f[u][a[u]-1];
        }
    };
    dfs(dfs,1,0);
    ll ans=0;
    for(int i=0;i<=siz[1];i++) (ans+=dp[1][i])%=mod;
    cout<<ans<<endl;
}
```

6.5 换根 dp

Listing 69: 换根 dp

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int N=1e6+5;
vector<ll>G[N];
vector<ll>ans,siz;
void add(int x,int y){
    G[x].emplace_back(y);
    G[y].emplace_back(x);
}
void dfs(int u,int f){
    siz[u]=1;
    for(auto &v:G[u]){
        if(v==f) continue;
        dfs(v,u);
        siz[u]+=siz[v];
    }
}
void __dfs(int u,int f){
    for(auto &v:G[u]){
        if(v==f) continue;
        ans[v]=(ans[u]+siz[1]-siz[v]*2);
        __dfs(v,u);
    }
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n;cin>>n;
    ans.resize(n+1);
    siz.resize(n+1);
    for(int i=1;i<n;i++){
        int x,y;cin>>x>>y;
        add(x,y);
    }
    dfs(1,0);
    ans[1]=siz[1];
    __dfs(1,0);
    cout<<max_element(ans.begin()+1,ans.begin()+1+n)-ans.begin()<<endl;
    return 0;
}
```

6.6 区间 dp

Listing 70: 区间 dp 典

```
//P1995 石子合并
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int N=105;
const ll INF=1e12;
ll a[N<<1],pre[N<<1];
ll Mx[N<<1][N<<1],Mn[N<<1][N<<1];
ll mx(int l,int r){
    if(Mx[l][r]!=-1) return Mx[l][r];
    if(l==r) return a[l];
    ll ans=0;
    for(int k=l;k+1<=r;k++){
        ans=max(ans,mx(l,k)+mx(k+1,r));
    }
    return Mx[l][r]=ans+(pre[r]-pre[l-1]);
}
ll mn(int l,int r){
```

```
    if(Mn[l][r]!=-1) return Mn[l][r];
    if(l==r) return a[l];
    ll ans=INF;
    for(int k=l;k+1<=r;k++){
        ans=min(ans,mn(l,k)+mn(k+1,r));
    }
    return Mn[l][r]=ans+(pre[r]-pre[l-1]);
}
void solve(){
    int n;cin>>n;
    for(int i=1;i<=n;i++) cin>>a[i],a[i+n]=a[i];
    for(int i=1;i<=n*2;i++) pre[i]=pre[i-1]+a[i];
    memset(Mx,-1,sizeof Mx);
    memset(Mn,-1,sizeof Mn);
    ll ans1=0,ans2=INF;
    for(int i=1;i<=n;i++){
        int l=i,r=n+i-1;
        ans1=max(ans1,mx(l,r)-(pre[r]-pre[l-1]));
        ans2=min(ans2,mn(l,r)-(pre[r]-pre[l-1]));
    }
    cout<<ans2<<endl<<ans1<<endl;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    solve();
    return 0;
}
```

6.7 SOS dp

例

给定长度为 N 的整数序列 $A = (A_1, A_2, \ldots, A_N)$，求满足 $1 \le i < j \le N$ 且列加法计算 $A_i + A_j$ 时不发生进位的整数对 (i, j) 的个数。

限制条件

- $1 \le N \le 2 \times 10^5$
- $1 \le A_i < 10^6$
- 所有输入均为整数

Listing 71: SOSdp 解决数位不发生进位问题

```
//ARC136D
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const int N=1e6+1;
int Flip(int x){
    int p=1,res=0;
    for(int i=0;i<6;i++){
        res+=(9-x%10)*p;
        x/=10;
        p*=10;
    }
    return res;
}//这个数的反集,如123->876
bool judge(int x){
    while(x){
        if(x%10>=5) return false;
        x/=10;
    }
    return true;
}//判定这个数与自身相加是否发生进位情况
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n;cin>>n;
    vector<ll>dp(N);
    vector<int>a(n+1);
    for(int i=1,x;i<=n;i++){
        cin>>a[i];
        dp[a[i]]++;
    }
    int p=1;
    for(int i=1;i<=6;i++){//本质是对每个十进制数位进行一次前缀和,又称高维前缀和(SOS dp)
        for(int j=0;j<N;j++){
            if((j/p)%10) dp[j]+=dp[j-p];
        }
        p*=10;
    }
    ll ans=0;
    for(int i=1;i<=n;i++){
        int x=Flip(a[i]);
        ans+=dp[x];
        if(judge(a[i])) ans--;
    }
    cout<<ans/2<<endl;
    return 0;
}
```

6.8 数位 dp

Listing 72: 统计 a-b 中各个数位出现个数

```
ll dp[20][20][2][2];
int digit[20],target;
ll dfs(int pos,int cnt,bool limit,bool lead){
    if(pos==-1) return cnt;
    if(dp[pos][cnt][limit][lead]!=-1) return dp[pos][cnt][limit][lead];
    int up=(limit)?digit[pos]:9;
    ll res=0;
    for(int i=0;i<=up;i++){
        int ncnt=cnt;
        if(i==target){
            if(!(lead&&i==0)){
                ncnt++;
            }
        }
```

```
        bool nlead=lead&&(i==0);
        bool nlimit=limit&&(i==up);
        res+=dfs(pos-1,ncnt,nlimit,nlead);
    }
    return dp[pos][cnt][limit][lead]=res;
}
ll solve(ll x){
    if(x<0) return 0;
    int len=0;
    while(x>0){
        digit[len++]=x%10;
        x/=10;
    }
    memset(dp,-1,sizeof(dp));
    return dfs(len-1,0,true,true);
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    ll a,b;cin>>a>>b;
    for(int i=0;i<10;i++){
        target=i;
        cout<<solve(b)-solve(a-1)<<' ';
    }
    cout<<endl;
    return 0;
}
```

7 数学

7.1 筛法求因数个数

Listing 73: 因数筛

```
const int N = 1000010;
int p[N];     // p[i]: i 的最小质因子（spf）
int pr[N];    // pr[]: 质数表
int cnt;
int d[N];     // d[i]: i 的正因数个数
int a[N];     // a[i]: i 的最小质因子的指数
void init(int n) {
    d[1] = 1; a[1] = 0; p[1] = 1;
    for(int i = 2; i <= n; i++) {
        if(!p[i]) {                    // i 是质数
            p[i] = pr[++cnt] = i;
            d[i] = 2;
            a[i] = 1;
        }
        for(int j = 1; j <= cnt && 1LL * i * pr[j] <= n; j++) {
            int m = i * pr[j];
            p[m] = pr[j];              // 记录 m 的最小质因子
            if(p[i] == pr[j]) {        // 还是同一个质因子
                d[m] = d[i] / (a[i] + 1) * (a[i] + 2);
                a[m] = a[i] + 1;
                break;
            }
            else {                     // 新增一个不同质因子
                d[m] = d[i] * 2;       // 等价于 d[i] * d[pr[j]]
                a[m] = 1;
            }
        }
    }
}
```

7.2 筛法求欧拉函数

`init(n)` 预处理 $1..n$（要求 $n \le N-1$，N 是文件里给出的数组上界），之后直接用 `phi[i]`；素数表在 `p[0..cnt-1]`。

Listing 74: 欧拉筛

```
// 线性筛：一次求出素数表与欧拉函数；用法见正文。
const int N=1e6+5;
int p[N],vis[N],cnt;
int phi[N];
void init(int n){
    phi[1]=1;
    for(int i=2;i<=n;i++){
        if(!vis[i]){
            p[cnt++]=i;
            phi[i]=i-1;
        }
        for(int j=0;i*p[j]<=n;j++){
            int m=i*p[j];
            vis[m]=1;
            if(i%p[j]==0){
                phi[m]=p[j]*phi[i];
                break;
            }
            else phi[m]=(p[j]-1)*phi[i];
        }
    }
}
```

7.3 筛法求莫比乌斯函数

同样 `init(n)` 之后直接用 `mu[i]`；与欧拉筛共用同一套线性筛骨架。

Listing 75: 莫比乌斯函筛

```
// 线性筛：一次求出素数表与莫比乌斯函数；用法见正文。
const int N=1e6+5;
int p[N],vis[N],cnt;
int mu[N];
void init(int n){
    mu[1]=1;
    for(int i=2;i<=n;i++){
        if(!vis[i]){
            p[cnt++]=i;
            mu[i]=-1;
        }
        for(int j=0;i*p[j]<=n;j++){
            int m=i*p[j];
            vis[m]=1;
            if(i%p[j]==0){
                mu[m]=0;
                break;
            }
            else{
                mu[m]=-mu[i];
            }
        }
    }
}
```

7.4 矩阵快速幂

Listing 76: 矩阵快速幂 basic

```
struct Mat{
    vector<vector<ll>>a;
    int n;
    Mat(int n):n(n){
        a=vector<vector<ll>>(n+1,vector<ll>(n+1));
    };
    Mat(){};
    friend Mat operator+(Mat &A,Mat &B){
        Mat C(A.n);
        for(int i=0;i<=A.n;i++){
            for(int j=0;j<=A.n;j++){

C.a[i][j]=(A.a[i][j]+B.a[i][j])%mod;
            }
        }
        return C;
    }
    friend Mat operator+=(Mat &A,Mat &B){
        A=A+B;
        return A;
    }
    friend Mat operator*(Mat &A,Mat &B){
        Mat C(A.n);
        for(int k=0;k<=A.n;k++){
            for(int i=0;i<=A.n;i++){
                if(A.a[i][k]==0) continue;
                for(int j=0;j<=A.n;j++){

C.a[i][j]=(C.a[i][j]+A.a[i][k]*B.a[k][j]%mod)%mod;
                }
            }
        }
        return C;
    }
    friend Mat operator*=(Mat &A,Mat &B){
        A=A*B;
        return A;
    }
    void q_pow(Mat &A,ll B){
        Mat S(A.n);//单位矩阵
        for(int i=0;i<=A.n;i++) S.a[i][i]=1;
        while(B>0){
            if(B&1){
                S*=A;
            }
            A*=A;
            B>>=1;
        }
        A=move(S);
    }
}M;
```

Listing 77: 矩阵快速幂 max-plus

```
const ll INF=1e18;
struct Mat{
    vector<vector<ll>>a;
    int n;
    Mat(int n):n(n){

a=vector<vector<ll>>(n+1,vector<ll>(n+1,-INF));
    };
    Mat(){};
    friend Mat operator*(const Mat &A,const Mat &
B){
        Mat C(A.n);
        for(int k=0;k<=A.n;k++){
            for(int i=0;i<=A.n;i++){
                for(int j=0;j<=A.n;j++){

C.a[i][j]=max(C.a[i][j],A.a[i][k]+B.a[k][j]);
                }
            }
        }
        return C;
    }
    friend Mat operator*=(Mat &A,Mat &B){
        A=A*B;
        return A;
    }
    void q_pow(Mat &A,ll B){
        Mat S(A.n);//单位矩阵
        for(int i=0;i<=A.n;i++) S.a[i][i]=0;
        while(B>0){
            if(B&1) S*=A;
            A*=A;
            B>>=1;
        }
        A=move(S);
    }
}M;
```

7.5 第二类斯特林数

第二类斯特林数（斯特林子集数）$\left\{ {n \atop k} \right\}$，也可记做 $S(n,k)$，表示将 n 个两两不同的元素划分为 k 个互不区分的非空子集的方案数。代码把第二维固定为 1000，即要求 $n \leq 10000$、$m \leq 1000$；超出时自行扩大 dp 的第二维。

Listing 78: Stirling 数

```
typedef unsigned long long ll;
const int N=10010;
ll dp[N][N/10],p;
void init(){
    dp[1][1]=1;
    for(int i=2;i<=10000;i++){
        for(int j=1;j<=1000;j++){

dp[i][j]=((j%p)*dp[i-1][j]%p+dp[i-1][j-1])%p;
```

```
        }
    }
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int n,m;
    cin>>n>>m>>p;
    init();
    cout<<dp[n][m]<<endl;
    return 0;
}
```

7.6 卡特兰数

Catalan 数有如下常见的表达式：

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{n!(n+1)!}, \quad n \geq 0. \tag{1}$$

$$C_n = \binom{2n}{n} - \binom{2n}{n+1}, \quad n \geq 0. \tag{2}$$

$$C_n = \frac{4n-2}{n+1}C_{n-1}, \quad n > 0,\ C_0 = 1. \tag{3}$$

数列的前几项为：1,1,2,5,14,42,132,429,1430,⋯

7.7 排列组合类-含 Lucas

```
const ll mod=998244353;
vector<ll>fac,invfac;
ll q_pow(ll x,ll y){
    ll s=1;
    x%=mod;
    while(y>0){
        if(y&1){
            s=s*x%mod;
        }
        x=x*x%mod;
        y>>=1;
    }
    return s;
}
void init(int n){
    fac.resize(n+1);
    invfac.resize(n+1);
    fac[0]=invfac[0]=1;
    for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%mod;
    invfac[n]=q_pow(fac[n],mod-2);
    for(int i=n-1;i>=1;i--)
invfac[i]=invfac[i+1]*(i+1)%mod;
}
ll C(ll n,ll m){
    if(n<m) return 0;
    if(n<mod&&m<mod) return
fac[n]*invfac[m]%mod*invfac[n-m]%mod;
    return C(n/mod,m/mod)*C(n%mod,m%mod)%mod;
}
```

7.8 常见计数恒等式与幂转换

曲棍球棒

$$\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}.$$

组合证明：从 $n+1$ 个元素中选 $r+1$ 个，按最大元素的位置 $i+1$ 分类。最大值恰为 $i+1$ 时，还需从前 i 个元素中选 r 个，共 $\binom{i}{r}$ 种，求和即得。

范德蒙德卷积

$$\sum_{k=0}^{n}\binom{a}{k}\binom{b}{n-k}=\binom{a+b}{n}.$$

由 $(1+x)^a(1+x)^b=(1+x)^{a+b}$ 比较 x^n 的系数即得，a,b 为任意非负整数。

普通幂转下降幂

定义下降幂 $x^{\underline{k}}=x(x-1)\cdots(x-k+1)$，并约定 $x^{\underline{0}}=1$，则

$$x^n=\sum_{k=0}^{n}S(n,k)\,x^{\underline{k}},$$

其中 $S(n,k)$ 是第二类斯特林数。组合意义：x^n 是把 n 个有标号球放入 x 个有标号盒子的方案数；按实际用到的盒子数 k 分类，先用 $S(n,k)$ 把球分成 k 个非空组，再选出并排列 k 个盒子，共 $x^{\underline{k}}$ 种。反向有

$$x^{\underline{n}}=\sum_{k=0}^{n}s(n,k)\,x^k,$$

其中 $s(n,k)$ 为带符号第一类斯特林数。代码用杨辉三角预处理组合数，用 $S(n,k)=S(n-1,k-1)+kS(n-1,k)$ 预处理第二类斯特林数，并验证上述恒等式与幂转换。

Listing 79: 曲棍球棒、范德蒙德卷积与幂转下降幂

```cpp
// 计数恒等式：曲棍球棒 / 范德蒙德 / 幂转下降幂
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int N = 1005;
const ll MOD = 998244353;
ll C[N][N];    // 组合数
ll S2[N][N];   // 第二类斯特林数

void init(int n) {
    for (int i = 0; i <= n; ++i) {
        C[i][0] = C[i][i] = 1;
        for (int j = 1; j < i; ++j) {
            C[i][j] = (C[i - 1][j - 1]
                     + C[i - 1][j]) % MOD;
        }
    }
    S2[0][0] = 1;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= i; ++j) {
            S2[i][j] = (S2[i - 1][j - 1]
                      + j * S2[i - 1][j]) % MOD;
        }
    }
}

// 曲棍球棒：sum C(i,r) = C(n+1,r+1)
ll hock(int n, int r) {
    ll s = 0;
    for (int i = r; i <= n; ++i)
        s = (s + C[i][r]) % MOD;
    return s;
}

// 范德蒙德：sum C(a,k)C(b,n-k) = C(a+b,n)
ll vander(int a, int b, int n) {
    ll s = 0;
    int lo = max(0, n - b);
    int hi = min(a, n);
    for (int k = lo; k <= hi; ++k) {
        s = (s + C[a][k] * C[b][n - k]) % MOD;
    }
    return s;
}

// x 的 k 次下降幂：x(x-1)...(x-k+1)
ll fall(ll x, int k) {
    x %= MOD;
    if (x < 0) x += MOD;
    ll r = 1;
    for (int i = 0; i < k; ++i) {
        r = r * ((x - i + MOD) % MOD) % MOD;
    }
    return r;
}

ll pw(ll a, ll e) {
    ll r = 1;
    a %= MOD;
    while (e) {
        if (e & 1) r = r * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}

int main() {
    init(1000);
    assert(hock(10, 3) == C[11][4]);
    ll v = vander(5, 7, 4);
```

```
    assert(v == C[12][4]);
    for (int n = 0; n <= 50; ++n)
        for (int r = 0; r <= n; ++r)
            assert(hock(n, r)
                   == C[n + 1][r + 1]);
    for (int a = 0; a <= 30; ++a)
        for (int b = 0; b <= 30; ++b)
            for (int n = 0; n <= a + b; ++n) {
                assert(vander(a, b, n)
                       == C[a + b][n]);
            }

    // x^20 = sum S2[20][k] * x 的下降幂
    for (ll x = 0; x <= 20; ++x) {
        ll rhs = 0;
        for (int k = 0; k <= 20; ++k) {
            rhs = (rhs + S2[20][k]
                   * fall(x, k)) % MOD;
        }
        assert(rhs == pw(x, 20));
    }
    cout << "ok\n";
    return 0;
}
```

7.9 高斯消元

Listing 80: 高斯消元

```
const int N=110;
const double eps=1e-8;
int n;
double a[N][N];
ll gauss(){
    ll c,r;
    for(c=0,r=0;c<n;c++){
        ll t=r;
        for(int i=r;i<n;i++){
            if(abs(a[i][c])>abs(a[t][c])) t=i;
        }
        if(abs(a[t][c])<eps) continue;
        for(int j=c;j<n+1;j++)
swap(a[t][j],a[r][j]);//将绝对值最大的换到最顶端
        for(int j=n;j>=c;j--)
a[r][j]/=a[r][c];//将当前行首位变成1
        for(int i=r+1;i<n;i++){
            if(abs(a[i][c])>eps){
                for(int j=n;j>=c;j--)
a[i][j]-=a[r][j]*a[i][c];
            }
        }
        r++;
    }
    if(r<n){
        for(int i=r;i<n;i++) if(abs(a[i][n])>eps)
return 2;
        return 1;
    }
    for(int i=n-1;i>=0;i--){
        for(int j=i+1;j<n;j++)
a[i][n]-=a[i][j]*a[j][n];
    }
    return 0;
}
void solve(){
    cin>>n;
    for(int i=0;i<n;i++){
        for(int j=0;j<n+1;j++){
            cin>>a[i][j];
        }
    }
    ll t=gauss();
    cout<<fixed<<setprecision(2);
    if(t==0) {
        for(int i=0;i<n;i++){
            if(abs(a[i][n])<eps)
a[i][n]=abs(a[i][n]);
            cout<<a[i][n]<<endl;
        }
    }
    else cout<<"No Solution"<<endl;
}
```

7.10 FFT

Listing 81: FFT 解决区间和是质数的数的区间个数

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
#define endl '\n'
struct FFT {
    using cd = complex<double>;
    const double PI = acos(-1);
    void fft(vector<cd>& a, bool invert) {
        int n = a.size();
        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1)
                j ^= bit;
            j ^= bit;
            if (i < j)
                swap(a[i], a[j]);
        }
        for (int len = 2; len <= n; len <<= 1) {
            double ang = 2 * PI / len * (invert ?
-1 : 1);
            cd wlen(cos(ang), sin(ang));
            for (int i = 0; i < n; i += len) {
                cd w(1);
                for (int j = 0; j < len / 2; j++) {
                    cd u = a[i + j], v = a[i + j +
len / 2] * w;
                    a[i + j] = u + v;
                    a[i + j + len / 2] = u - v;
```

```
                    w *= wlen;
                }
            }
        }
        if (invert) {
            for (cd& x : a)
                x /= n;
        }
    }
    template<typename T>
    vector<ll> multiply(const vector<T>& a, const vector<T>& b) {
        vector<cd> fa(a.size()), fb(b.size());
        for (int i = 0; i < a.size(); i++) fa[i] = (double)a[i];
        for (int i = 0; i < b.size(); i++) fb[i] = (double)b[i];
        int n = 1;
        while (n < (int)(a.size() + b.size() - 1)) n <<= 1;
        fa.resize(n);
        fb.resize(n);
        fft(fa, false);
        fft(fb, false);
        for (int i = 0; i < n; ++i) fa[i] *= fb[i];
        fft(fa, true);
        vector<ll> res(a.size() + b.size() - 1);
        for (int i = 0; i < res.size(); i++) res[i] = llround(fa[i].real());
        return res;
    }
};
const int N=1e6+5;
bitset<N>isp;
vector<int>p;
void init(int n){
    p.reserve(n/10);
    isp.set();
    isp[0]=isp[1]=false;
    for(int i=2;i<=n;i++){
        if(isp[i]) p.push_back(i);
        else continue;
        for(int j=i*2;j<=n;j+=i) isp[j]=false;
    }
}
void solve(){
    int n;cin>>n;
    vector<ll>a(n+1),pre(n+1);
    for(int i=1;i<=n;i++) cin>>a[i],pre[i]=pre[i-1]+a[i];
    vector<int>vis(pre[n]+1);
    vis[0]=1;
    for(int i=1;i<=n;i++) vis[pre[i]]=1;
    vector<int>B(vis.rbegin(),vis.rend());
    FFT fft;
    vector<ll>C=fft.multiply(vis,B);
    ll ans=0;
    for(int x:p){
        if(x>pre[n]) break;
        ans+=C[pre[n]+x];
    }
    cout<<ans<<endl;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    init(1e6);int T=1;cin>>T;
    while(T--) solve();
    return 0;
}
```

7.11 NTT

Listing 82: NTT 多项式乘法 (模 998244353)

```
struct NTT {
    static const int MOD = 998244353;
    static const int G = 3;
    /*
    ntt友好模数 原根
    998244353 3
    469762049 3
    167772161 3
    1004535809 3
    754974721 11
    */
    static ll qpow(ll a, ll b) {
        ll res = 1;
        while (b) {
            if (b & 1) res = res * a % MOD;
            a = a * a % MOD;
            b >>= 1;
        }
        return res;
    }
    static void ntt(vector<ll>& a, bool invert) {
        int n = a.size();
        // 位逆序
        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >> 1;
            while (j & bit) {
                j ^= bit;
                bit >>= 1;
            }
            j ^= bit;
            if (i < j) {
                swap(a[i], a[j]);
            }
        }
        // 蝴蝶操作
        for (int len = 2; len <= n; len <<= 1) {
            ll wlen = qpow(G, (MOD - 1) / len);
            if (invert) {
                wlen = qpow(wlen, MOD - 2);
            }
            for (int i = 0; i < n; i += len) {
                ll w = 1;
```

```
                for (int j = 0; j < len / 2; j++) {
                    ll u = a[i + j];
                    ll v = a[i + j + len / 2] * w %
 MOD;
                    ll x = u + v;
                    if (x >= MOD) x -= MOD;
                    ll y = u - v;
                    if (y < 0) y += MOD;
                    a[i + j] = x;
                    a[i + j + len / 2] = y;
                    w = w * wlen % MOD;
                }
            }
        }
        if (invert) {
            ll inv_n = qpow(n, MOD - 2);
            for (ll& x : a) {
                x = x * inv_n % MOD;
            }
        }
    }
    static vector<ll> multiply(vector<ll> a,
vector<ll> b) {
        if (a.empty() || b.empty()) {
            return {};
        }
        // 小规模直接暴力卷积，常数更小
        if (min(a.size(), b.size()) <= 32) {
            vector<ll> c(a.size() + b.size() - 1);
            for (int i = 0; i < (int)a.size();
i++) {
                for (int j = 0; j < (int)b.size();
j++) {
                    c[i + j] = (c[i + j] + a[i] *
b[j]) % MOD;
                }
            }
            return c;
        }
        int need = (int)a.size() + (int)b.size() -
1;
        int n = 1;
        while (n < need) {
            n <<= 1;
        }
        a.resize(n);
        b.resize(n);
        ntt(a, false);
        ntt(b, false);
        for (int i = 0; i < n; i++) {
            a[i] = a[i] * b[i] % MOD;
        }
        ntt(a, true);
        a.resize(need);
        return a;
    }
};
```

7.12 多项式乘幂级数的分治求和

给定模数 p、底数 x、次数 d，求

$$\sum_{k=1}^{n} k^d x^k \bmod p \qquad (n \text{ 可达} 10^{18}).$$

逐项算要 $O(n)$；把 k^d 看成多项式 $f(y)=y^d$ 之后可以用分治做到 $O(d^2 \log n)$，而且一次就能算出所有 $i=0..d$ 的 $\sum_{k=1}^{n} k^i x^k$，于是任意多项式 $f(y)=\sum_i c_i y^i$ 只要按系数 c_i 线性组合即可。

记 $S_i(m)=\sum_{k=1}^{m} k^i x^k$、$P(m)=x^m$。把区间 $[1,2m]$ 拆成 $[1,m]$ 与 $[m+1,2m]$，后一半令 $k=m+t$，用二项式定理展开：

$$S_i(2m)=S_i(m)+x^m \sum_{j=0}^{i} \binom{i}{j} m^{i-j} S_j(m),$$

奇数长度再补上最后一项：

$$S_i(2m+1)=S_i(2m)+(2m+1)^i x^{2m+1}.$$

于是按 n 二分（奇偶分别处理）：每层对 $i=0..d$ 各做一次 $O(d)$ 的求和，单层 $O(d^2)$，共 $O(\log n)$ 层，总复杂度 $O(d^2 \log n)$，额外空间 $O(d \log n)$（递归栈）。当 d 很大时，合并里那个二项式卷积可以用 NTT 加速到 $O(d \log d \log n)$。另外注意 x 与 n 都可能远大于 p，所有取模都必须在乘法之前完成，否则 `long long` 会在连乘时溢出。

Listing 83: 分治求幂级数和 (多项式乘幂级数)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// 分治求 S_i(n) = sum_{k=1..n} k^i x^k（i = 0..deg
）：推导、复杂度与用法见正文。
struct PolyExpSum {
    ll mod;                     // 模数
    int deg;                    // 多项式次数：f(y)
 = y^deg
    ll base;                    // 底数 x（已取模）
    vector<vector<ll>> C;       // 杨辉三角（组合
数）
    struct Node {
        vector<ll> s;           // s[i] = S_i(m) =
sum_{k=1}^{m} k^i x^k
        ll pw;                  // x^m
    };

    ll powmod(ll a, ll b) const {      // 快速幂：
先取模，避免底数 >= mod 时溢出
```

```
        a %= mod;
        ll r = 1 % mod;
        while (b) {
            if (b & 1) r = r * a % mod;
            a = a * a % mod;
            b >>= 1;
        }
        return r;
    }
    /* 返回 S_0..S_deg(n) 与 x^n */
    Node dfs(ll n) {
        if (n == 0) return {vector<ll>(deg + 1, 0), 1 % mod};
        if (n == 1) return {vector<ll>(deg + 1, base), base};   // S_i(1) = 1^i · x = x
        Node half = dfs(n / 2);
        ll m = n / 2;
        vector<ll> res(deg + 1);
        for (int i = 0; i <= deg; ++i) {
            // 先算 S_i(2m)
            ll add = 0;
            for (int j = 0; j <= i; ++j) {
            // sum_{j} C(i,j) m^{i-j} S_j(m)
                ll t = C[i][j] * powmod(m, i - j) % mod * half.s[j] % mod;
                add = (add + t) % mod;
            }
            res[i] = (half.s[i] + add * half.pw) % mod;          // 再乘上 x^m
        }
        ll cur = half.pw * half.pw % mod;
            // x^{2m}
        if (n & 1) {
            // 奇数长度补一项 S_i(2m+1)
            ll last = cur * base % mod;
            // x^{2m+1}
            for (int i = 0; i <= deg; ++i)
                res[i] = (res[i] + powmod(n, i) * last) % mod;  // +(2m+1)^i · x^{2m+1}
            cur = last;
        }
        return {res, cur};
    }

    PolyExpSum(int degree, ll a, ll m) : mod(m), deg(degree), base(a % m) {
        C.assign(deg + 1, vector<ll>(deg + 1, 0));
        for (int i = 0; i <= deg; ++i) {
            C[i][0] = C[i][i] = 1 % mod;
            for (int j = 1; j < i; ++j)
                C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % mod;
        }
    }
    /* sum_{k=1}^{n} k^deg · x^k mod mod */
    ll query(ll n) { return dfs(n).s[deg]; }
};

// 例：sum_{k=1}^{10} k^3 · 3^k，以及一般多项式 f(y)=2y^2+y（线性组合各次幂和）
int main() {
    const ll MOD = 1000000007LL;
    PolyExpSum f(3, 3, MOD);
    cout << f.query(10) << '\n';
    PolyExpSum g(2, 5, MOD);                    // 取出 S_0..S_2 自己组合
    auto s = g.dfs(10);
    ll ans = (2 * s.s[2] + s.s[1]) % MOD;       // f(y) = 2y^2 + y
    cout << ans << '\n';
    return 0;
}
```

7.13 线性递推 (Berlekamp-Massey)

用前若干项求出最短线性递推式，再用 $O(k^2 \log n)$ 的快速幂求第 n 项（如 Fibonacci 取 n 很大时）。

Listing 84: Berlekamp-Massey + 线性递推

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 1000000007LL;

ll qpow(ll a, ll b){ ll r=1; a%=MOD; for(;b;b>>=1,a=a*a%MOD) if(b&1) r=r*a%MOD; return r; }

vector<ll> BerlekampMassey(const vector<ll>& s){
    vector<ll> ls, cur; ll lf=0, ld=0;
    for(int i=0;i<(int)s.size();i++){
        ll t=0;
        for(int j=0;j<(int)cur.size();j++) t=(t+s[i-1-j]*cur[j])%MOD;
        if((s[i]-t)%MOD==0) continue;
        if(cur.empty()){ cur.resize(i+1); lf=i; ld=(s[i]-t)%MOD; continue; }
        ll k=(s[i]-t)%MOD*qpow(((ld%MOD)+MOD)%MOD, MOD-2)%MOD;
        vector<ll> c(i-lf-1); c.push_back(k);
        for(int j=0;j<(int)ls.size();j++) c.push_back((MOD-k)*ls[j]%MOD);
        if(c.size()<cur.size()) c.resize(cur.size());
        for(int j=0;j<(int)cur.size();j++) c[j]=(c[j]+cur[j])%MOD;
        if(i-lf+(int)ls.size() >= (int)cur.size()){ ls=cur; lf=i; ld=(s[i]-t)%MOD; }
        cur=c;
    }
    for(auto&x:cur) x=(x+MOD)%MOD;
    return cur;
}
```

```cpp
/* s[n] = rec[0]*s[n-1]+...+rec[k-1]*s[n-k] */
ll linearRecurrence(const vector<ll>& s, unsigned long long n){
    if(n < s.size()) return s[(int)n];
    vector<ll> rec = BerlekampMassey(s);
    int k = rec.size();
    if(k==0) return 0;
    vector<ll> c(k);            // x^k = c[0]x^{k-1}+...+c[k-1]
    for(int j=0;j<k;j++) c[j]=rec[j];
    auto mul = [&](const vector<ll>& p, const vector<ll>& q){
        vector<ll> t(2*k,0);
        for(int i=0;i<k;i++) if(p[i]) for(int j=0;j<k;j++) if(q[j])
            t[i+j]=(t[i+j]+p[i]*q[j])%MOD;
        for(int i=2*k-2;i>=k;i--) if(t[i])
            for(int j=0;j<k;j++) t[i-1-j]=(t[i-1-j]+t[i]*c[j])%MOD;
        vector<ll> r(k); for(int i=0;i<k;i++) r[i]=t[i]%MOD;
        return r;
    };
    vector<ll> r(k), a(k); r[0]=1; if(k>1) a[1]=1; else a[0]=rec[0];
    unsigned long long e=n;
    while(e){ if(e&1ULL) r=mul(r,a); a=mul(a,a); e>>=1ULL; }
    ll ans=0; for(int i=0;i<k;i++) ans=(ans+r[i]*s[i])%MOD;
    return ans;
}

int main(){
    unsigned long long n; cin>>n;
    vector<ll> s = {1,1,2,3,5,8,13,21,34,55};   // F_1..F_10
    cout<<linearRecurrence(s, n-1)<<"\n";       // 0-based: F_n = s[n-1]
    return 0;
}
```

7.14 Pollard-Rho

Listing 85: Pollard-Rho

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128;
// ========== 快速乘（取模） ==========
ll mul_mod(ll a, ll b, ll mod) {
    return (ll)((i128)a * b % mod);
}
// ========== 快速幂取模 ==========
ll q_pow(ll a, ll d, ll mod) {
    ll res = 1;
    a %= mod;
    while (d) {
        if (d & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        d >>= 1;
    }
    return res;
}
// ========== Miller-Rabin 确定性判素数 ==========
bool is_prime(ll n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;
    ll d = n - 1;
    int r = 0;
    while ((d & 1) == 0) {
        d >>= 1;
        ++r;
    }
    // 底数集合（对 2^64 以内确定性）
    static const ll bases[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    for (ll a : bases) {
        if (a % n == 0) continue;
        ll x = q_pow(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool comp = true;
        for (int i = 0; i < r - 1; ++i) {
            x = mul_mod(x, x, n);
            if (x == n - 1) {
                comp = false;
                break;
            }
        }
        if (comp) return false;
    }
    return true;
}
// ========== 最大公约数 ==========
// ========== Pollard-Rho 分解因子（参考你的高效写法） ==========
// 小质数表，用于提前试除
const vector<ll> small_primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
ll pollard_rho(ll n) {
    // 先试除小质数
    for (ll p : small_primes) {
        if (n % p == 0) return p;
    }

    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    uniform_int_distribution<ll> dist(1, n - 1);
    auto f = [&](ll x) -> ll {
        x = mul_mod(x, x, n) + 1;
        return x >= n ? x - n : x;
    };
    ll x = 0, y = 0, tot = 0, p = 1, q, g;
```

```
    for (ll i = 0; (i & 255) || (g = gcd(p, n)) == 1; ++i) {
        x = f(x);
        y = f(f(y));
        if (x == y) {
            x = ++tot;
            y = f(x);
        }
        q = mul_mod(p, (x > y ? x - y : y - x), n);
        if (q) p = q;
    }
    return g;
}
// ========== 递归分解，更新最大质因子 ==========
ll max_factor = 0 , n;
ll min_factor ;   // 初始化为 n
void factorize_mx(ll n) {
    if (n <= max_factor || n < 2) return;
    if (is_prime(n)) {
        max_factor = max(max_factor, n);
        return;
    }
    ll d = pollard_rho(n);
    factorize_mx(d);
    factorize_mx(n / d);
}
void factorize_mn(ll n) {
    if (n <= 1) return;
    if (is_prime(n)) {
        min_factor = min(min_factor, n);
        return;
    }
    ll d = pollard_rho(n);
    factorize_mn(d);
    factorize_mn(n / d);
}
int main() {
    cin.tie(nullptr)->ios::sync_with_stdio(false);
    int T;cin >> T;
    while (T--) {
        cin >> n;
        max_factor = 0,min_factor = n;
        factorize_mx(n);
        if (max_factor == n) cout << "Prime\n";
        else cout << max_factor << '\n';
    }
    return 0;
}
```

7.15 异或线性基

Listing 86: 异或线性基

```
struct LB{
    vector<ll>a;
    bool flag;
    LB(){
        flag=false;
        a.resize(63);
    }
    bool insert(ll x){
        for(int i=61;i>=0;i--){
            if(x>>i&1LL){
                if(!a[i]) {
                    a[i]=x;
                    return true;
                }
                x^=a[i];
            }
        }
        flag=true;
        return false;
    }
    ll ask_max(){
        ll ans=0;
        for(int i=61;i>=0;i--){
            if((ans^a[i])>ans) ans^=a[i];
        }
        return ans;
    }
    ll ask_min(){
        if(flag) return 0;
        for(int i=0;i<=61;i++){
            if(a[i]) return a[i];
        }
        return 0;
    }
};
```

7.16 数论分块

题面简述

给定一个正整数 x，求 $\sum_{i=1}^{x} d(i)$，其中 $d(i)$ 表示 i 的因数个数。

第一步：转化求和式

由因数个数的定义 $d(i)=\sum_{d|i}1$，将求和交换顺序：

$$\sum_{i=1}^{x} d(i)=\sum_{i=1}^{x}\sum_{d|i}1=\sum_{d=1}^{x}\left\lfloor\frac{x}{d}\right\rfloor$$

即问题转化为求 $\sum_{d=1}^{x}\left\lfloor\frac{x}{d}\right\rfloor$。

第二步：整除分块

$\left\lfloor\frac{x}{d}\right\rfloor$ 的取值至多只有 $O(\sqrt{x})$ 种。对于每一种取值 $v=\left\lfloor\frac{x}{l}\right\rfloor$，所有满足 $\left\lfloor\frac{x}{d}\right\rfloor=v$ 的 d 构成一个连续区间 $[l,r]$，其中：

$$r=\min\left(\left\lfloor\frac{x}{v}\right\rfloor,\ x\right)$$

在该区间内 $\left\lfloor \frac{x}{d} \right\rfloor$ 恒为 v，因此该块的贡献为：

$$v \times (r - l + 1)$$

遍历所有块，累加贡献即得答案：

$$\text{Ans} = \sum_{\text{block } [l,r]} \left\lfloor \frac{x}{l} \right\rfloor \times (r - l + 1)$$

每次令 $l \leftarrow r+1$ 跳到下一个块，直到 $l > x$。

复杂度分析

$\left\lfloor \frac{x}{d} \right\rfloor$ 的不同取值个数为 $O(\sqrt{x})$（当 $d \le \sqrt{x}$ 时至多 $\sqrt{x}$ 种，当 $d > \sqrt{x}$ 时 $\left\lfloor \frac{x}{d} \right\rfloor < \sqrt{x}$，同样至多 $\sqrt{x}$ 种），因此整除分块的块数为 $O(\sqrt{x})$，每块内仅需 $O(1)$ 计算。

总时间复杂度为 $O(\sqrt{x})$，空间复杂度为 $O(1)$。

Listing 87: 整除分块

```
void solve(){
    ll n;cin>>n;
    ll ans=0;
    for(ll l=1,r;l<=n;l=r+1) {
        ll v=(n/l);
        r=min((n/v),n);
        ans+=v*(r-l+1);
    }
    cout<<ans<<endl;
}
```

7.17 莫比乌斯反演

题面简述

给定 N, M，求满足 $1 \le x \le N$，$1 \le y \le M$ 且 $\gcd(x, y)$ 为质数的有序对 (x, y) 的个数。

多组询问，$T \le 10^4$，$N, M \le 10^7$。

第一步：枚举质数

设答案为 Ans，直接按定义展开：

$$\text{Ans} = \sum_{p \in \mathcal{P}} \sum_{x=1}^{N} \sum_{y=1}^{M} [\gcd(x, y) = p]$$

其中 $\mathcal{P}$ 为质数集合。令 $x = pa$，$y = pb$，则：

$$\text{Ans} = \sum_{p \in \mathcal{P}} \sum_{a=1}^{\lfloor N/p \rfloor} \sum_{b=1}^{\lfloor M/p \rfloor} [\gcd(a, b) = 1]$$

第二步：莫比乌斯反演

利用经典恒等式 $[\gcd(a, b) = 1] = \sum_{d \mid \gcd(a,b)} \mu(d)$，代入得：

$$\text{Ans} = \sum_{p \in \mathcal{P}} \sum_{d=1}^{\min(\lfloor N/p \rfloor, \lfloor M/p \rfloor)} \mu(d) \left\lfloor \frac{N}{pd} \right\rfloor \left\lfloor \frac{M}{pd} \right\rfloor$$

第三步：换元合并

令 $T = pd$，则 $d = T/p$，对于固定的 T，p 遍历 T 的所有质因子。交换求和顺序：

$$\text{Ans} = \sum_{T=1}^{\min(N,M)} \left\lfloor \frac{N}{T} \right\rfloor \left\lfloor \frac{M}{T} \right\rfloor \sum_{p \mid T,\, p \in \mathcal{P}} \mu\left(\frac{T}{p}\right)$$

定义函数：

$$f(T) = \sum_{p \mid T,\, p \in \mathcal{P}} \mu\left(\frac{T}{p}\right)$$

则答案化为：

$$\text{Ans} = \sum_{T=1}^{\min(N,M)} f(T) \left\lfloor \frac{N}{T} \right\rfloor \left\lfloor \frac{M}{T} \right\rfloor$$

第四步：预处理 f 及其前缀和

$f(T)$ 的预处理方式：线性筛求出所有质数与 μ 值后，枚举每个质数 p，再枚举 p 的倍数 $T = kp$，将 $\mu(k)$ 累加到 $f(T)$ 上。这一步复杂度为：

$$\sum_{p \in \mathcal{P}} \left\lfloor \frac{n}{p} \right\rfloor \approx O\left(\frac{n \ln \ln n}{1}\right) = O(n \log \log n)$$

随后求出 f 的前缀和 $F(n) = \sum_{T=1}^{n} f(T)$。

第五步：整除分块回答询问

对每组询问，利用 $\lfloor N/T \rfloor$ 和 $\lfloor M/T \rfloor$ 的取值分块性质（整除分块/数论分块），将 T 的求和区间划分为 $O(\sqrt{N} + \sqrt{M})$ 个块，每块内 $\lfloor N/T \rfloor$ 和 $\lfloor M/T \rfloor$ 均为常数，利用前缀和 F 即可 $O(1)$ 求出每块的贡献：

$$\text{Ans} = \sum_{\text{block } [l,r]} \left\lfloor \frac{N}{l} \right\rfloor \left\lfloor \frac{M}{l} \right\rfloor \big(F(r) - F(l-1)\big)$$

复杂度分析

- **预处理**：线性筛求 μ 和质数表为 $O(n)$；枚举质数倍数计算 f 为 $O(n \log \log n)$；前缀和为 $O(n)$。总预处理复杂度 $O(n \log \log n)$，其中 $n = \max(N, M) = 10^7$。
- **单次询问**：整除分块为 $O(\sqrt{n})$。
- **总复杂度**：$O(n \log \log n + T\sqrt{n})$。

  对于本题 $n = 10^7$，$T = 10^4$：

- 预处理：$10^7 \times \log \log(10^7) \approx 10^7 \times 3 \approx 3 \times 10^7$。
- 查询：$10^4 \times \sqrt{10^7} \approx 10^4 \times 3162 \approx 3.16 \times 10^7$。

总操作量约 6×10^7，可以通过。空间复杂度为 $O(n)$。

Listing 88: 莫比乌斯反演

```cpp
// P2257 YY的GCD: 求 1<=x<=N, 1<=y<=M 且 gcd(x,y) 为质数的有序对数量。
// 多组询问，T<=1e4，N,M<=1e7；线性筛 + 预处理 f(T) 前缀和 + 整除分块。
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int MAXN = 10000000;
signed char mu[MAXN + 1];   // 莫比乌斯函数，取值 -1/0/1
int f[MAXN + 1];            // f(T) = sum_{p|T, p prime} mu(T/p)
int pre[MAXN + 1];          // f 的前缀和
vector<int> primes;
bitset<MAXN + 1> composite;

void init(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!composite[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if (1LL * i * p > n) break;
            composite[i * p] = 1;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            }
            mu[i * p] = -mu[i];
        }
    }
    // 计算 f(T)
    for (int p : primes) {
        for (int t = p; t <= n; t += p) {
            f[t] += mu[t / p];
        }
    }
    for (int i = 1; i <= n; ++i) {
        pre[i] = pre[i - 1] + f[i];
    }
}

ll solve(int n, int m) {
    if (n > m) swap(n, m);
    ll ans = 0;
    for (int l = 1, r; l <= n; l = r + 1) {
        r = min(n / (n / l), m / (m / l));
        ans += 1LL * (n / l) * (m / l)
             * (pre[r] - pre[l - 1]);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    vector<pair<int, int>> qs(T);
    int max_n = 1;
    for (auto &[n, m] : qs) {
        cin >> n >> m;
        max_n = max({max_n, n, m});
    }
    init(max_n);
    for (auto [n, m] : qs) {
        cout << solve(n, m) << '\n';
    }
    return 0;
}
```

7.18 exgcd

Listing 89: exgcd

```cpp
//P1516青蛙的约会
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
ll exgcd(ll a,ll b,ll &x,ll &y){
    if(!b){
        x=1,y=0;
        return a;
    }
    ll d=exgcd(b,a%b,y,x);
    y-=a/b*x;
    return d;
```

```
}
void solve(){
    int x,y,m,n,l;
    cin>>x>>y>>m>>n>>l;
    ll X,Y;
    ll d=exgcd(n-m,l,X,Y);
    if((x-y)%d) cout<<"Impossible"<<endl;
    else{
        ll s=(x-y)/d;
        ll mod=abs(l/d);
        X*=s;
        cout<<(X%mod+mod)%mod<<endl;
    }
}
```

7.19 exgcd 求逆元与线性同余

`exgcd(a,b,x,y)` 返回 $d = \gcd(a,b)$，并给出一组 $ax + by = d$ 的解。`inv(a,m)` 在 $\gcd(a,m) = 1$ 时返回 $[0,m)$ 内的逆元；不互质时模板用 `assert` 拦截。`lineq(a,b,m,x)` 解 $ax \equiv b \pmod m$：无解返回 `false`，有解返回最小非负特解 x，通解为 $x + k \cdot m/\gcd(a,m)$。三者均为 $O(\log m)$。

Listing 90: exgcd 求逆元与线性同余

```
// exgcd 求逆元与线性同余：用法、推导与复杂度见正文。
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// 返回 gcd(a,b)，并求出一组 ax+by=gcd(a,b) 的解
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

// a 在模 m 下的逆元，要求 gcd(a,m)=1；返回 [0,m)
ll inv(ll a, ll m) {
    a %= m;
    if (a < 0) a += m;
    ll x, y;
    ll d = exgcd(a, m, x, y);
    assert(d == 1);              // 不互质时没有逆元
    return (x % m + m) % m;
}

// 解 ax ≡ b (mod m)。无解返回 false；
// 有解返回最小非负特解 x，通解为 x + k*(m/gcd(a,m))
bool lineq(ll a, ll b, ll m, ll &x) {
    if (m < 0) m = -m;
    a %= m;
    if (a < 0) a += m;
    b %= m;
    if (b < 0) b += m;
    ll g = __gcd(a, m);
    if (b % g != 0) return false;
    a /= g, b /= g, m /= g;
    __int128 x0 = (__int128)b * inv(a, m) % m;
    x = (ll)x0;
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {                  // 求逆元
            ll a, m;
            cin >> a >> m;
            cout << inv(a, m) << '\n';
        } else {                        // ax=b (mod m)
            ll a, b, m, x;
            cin >> a >> b >> m;
            if (lineq(a, b, m, x))
                cout << x << '\n';
            else
                cout << "No\n";
        }
    }
    return 0;
}
```

7.20 exBSGS

Listing 91: exBSGS

```
ll q_pow(ll x,ll y,ll mod){
    ll s=1;
    while(y>0){
        if(y&1){
            s=s*x%mod;
        }
        x=x*x%mod;
        y>>=1;
    }
    return s;
}
ll exgcd(ll a,ll b,ll &x,ll &y){
    if(!b){
        x=1,y=0;
        return a;
```

```
    }
    ll d=exgcd(b,a%b,y,x);
    y-=a/b*x;
    return d;
}
ll inv(ll a,ll mod){
    ll x,y;
    exgcd(a,mod,x,y);
    return (x%mod+mod)%mod;
}
ll exBSGS(ll a,ll b,ll p){
    if(p==1) return 0;
    a%=p,b%=p;
    if(b==1) return 0;          // x=0 时 a^0=1
    if(a==0) return b==0?1:-1;  // 0^0=1, 0^x=0
    ll cnt=0;
    ll d=gcd(a,p);
    while(d>1){
        if(b%d!=0) return -1;
        p/=d;
        b=b/d*inv(a/d,p)%p;
        ++cnt;
        if(b==1) return cnt;
        d=gcd(a,p);
    }
    if(p==1) return cnt;
    unordered_map<ll,ll>mp;
    ll m=ceil(sqrt(p));
    ll e=1;
    for(ll j=0;j<m;++j){
        if(!mp.count(e)) mp[e]=j;
        e=e*a%p;
    }
    ll inv_am=inv(q_pow(a,m,p),p);
    ll cur=b;
    for(ll i=0;i<=m;i++){
        if(mp.count(cur)){
            return i*m+mp[cur]+cnt;
        }
        cur=cur*inv_am%p;
    }
    return -1;
}
```

7.21 博弈论与 SG 打表

例题：洞烛与摩耶的游戏

给定长度为 n 的正整数数组 $a = [a_1, a_2, \ldots, a_n]$，**洞烛先手**，两人轮流操作。每一轮当前玩家必须：

1. 选一个下标 i 满足 $a_i > 0$；
2. 对所有 $j \neq i$，令 $a_j \leftarrow \min(a_j, a_i)$；
3. 令 $a_i \leftarrow a_i - 1$。

轮到某人时若数组已全为 0（无法选出正数），该玩家输，另一方胜。两人都取最优策略，判断谁赢。$T \le 10^4$，$\sum n \le 2 \times 10^6$，$1 \le a_i \le 10^9$；摩耶（后手）胜输出 `Maya`，洞烛（先手）胜输出 `Insight`。

第一步：SG 打表找规律

状态只和多重集有关，所以把数组**排序后**作为记忆化的键（不排序状态数会爆炸，规律也看不出来）。一个状态的 SG 值等于所有后继 SG 值的 mex，SG $= 0$ 即先手必败。暴力程序对”相同取值”只枚举一次（同值元素等价），值域和 n 都开小，并且 **只打印 SG $= 0$ 的必败态**——P 态列表最容易读出规律。

小规模（值域 $1 \sim 4$）打出的 P 态：

- $n = 1$：(2) (4)——唯一的数是偶数；
- $n = 2$：(1,1) (1,3) (2,2) (2,4) (3,3) (4,4)——两数同奇偶；
- $n = 3$：(2,2,2) (2,2,4) (2,3,3) (2,4,4) (4,4,4)——最小值为偶，且后两数同奇偶；
- $n = 4$：可归纳为下面的两两配对规则。

第二步：归纳规律

把排序后的数组从最小值开始配对：

- n 为奇数：最小值 a_1 落单（要求 a_1 为偶数），剩下一对一对配成 $(a_2, a_3), (a_4, a_5), \ldots$；
- n 为偶数：配成 $(a_1, a_2), (a_3, a_4), \ldots$。

若每一对内部奇偶性都相同（且落单元素为偶数），就是必败态，否则必胜。每组数据只需一次排序，$O(n \log n)$，$\sum n \le 2 \times 10^6$ 轻松通过。

代码

打表程序用 `#if 0` 包住，把 `0` 改成 `1` 即可单独编译运行、观察 P 态；后面是正解。

Listing 92: SG 打表与正解

```
#include <bits/stdc++.h>
using namespace std;

// 博弈论 SG 打表找规律：本文件含打表程序（#if 0 包住，改成 #if 1 可单独运行）与正解；
```

```cpp
// 例题题面、P 态规律与用法见正文。

/* ========== 1) 暴力 SG 打表（只打印必败态，规律最直观） ========== */
#if 0
map<vector<int>, int> memo;
int sg(vector<int> a) {
    sort(a.begin(), a.end());                   // 状态规范化：只和多重集有关
    if (a.back() == 0) return memo[a] = 0;      // 全 0：当前玩家无法行动，负
    if (memo.count(a)) return memo[a];
    set<int> nxt;
    int n = a.size();
    for (int i = 0; i < n; i++) {
        if (a[i] == 0) continue;
        if (i && a[i] == a[i - 1]) continue;  // 同值只试一次
        vector<int> b = a;
        for (int j = 0; j < n; j++) b[j] = min(b[j], a[i]);  // 先对所有 j != i 取 min
        b[i]--;
          // 再把自己减 1
        nxt.insert(sg(b));
    }
    int g = 0;
    while (nxt.count(g)) g++;                   // mex
    return memo[a] = g;
}
void dfs(int pos, int minv, int n, int V, vector<int>& a) {
    if (pos == n) {
        if (sg(a) == 0) {                       // 只打印 P 态
            for (int x : a) printf("%d ", x);
            printf("\n");
        }
        return;
    }
    for (int v = minv; v <= V; v++) { a[pos] = v; dfs(pos + 1, v, n, V, a); }
}
int main() {
    for (int n = 1; n <= 4; n++) {
        printf("n=%d:\n", n);
        vector<int> a(n, 1);
        dfs(0, 1, n, 4, a);                     // 值域 [1,4]
    }
    return 0;
}
#endif

/* ========== 2) 正解：排序 + 配对判奇偶 ========== */
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int n;
        scanf("%d", &n);
        vector<int> a(n);
        for (auto& x : a) scanf("%d", &x);
        sort(a.begin(), a.end());
        bool lose = true;
// 先手是否必败
        if (n & 1) {
            if (a[0] % 2 != 0) lose = false;
// 落单的最小值必须为偶数
            for (int i = 1; i + 1 < n; i += 2)
                if ((a[i] ^ a[i + 1]) & 1) lose = false;   // 每对同奇偶
        } else {
            for (int i = 0; i + 1 < n; i += 2)
                if ((a[i] ^ a[i + 1]) & 1) lose = false;
        }
        puts(lose ? "Maya" : "Insight");
    }
    return 0;
}
```

8 计算几何

8.1 基础定义

Listing 93: 基础计算几何 (精度与符号函数)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;
const ld INF = 1e100L;
const ld PI = acosl(-1.0L);
bool eq(ld a, ld b) {
    return fabsl(a - b) <= EPS;
}
int sgn(ld x) {
    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}
```

8.2 点与向量

Listing 94: 二维点 (Point 2D)

```cpp
struct P {
    ld x = 0, y = 0;
    P() = default;
    P(ld x, ld y) : x(x), y(y) {}
    P operator + (const P& p) const { return {x + p.x, y + p.y};}
    P operator - (const P& p) const { return {x - p.x, y - p.y};}
```

```
    P operator * (ld k) const { return {x * k, y * k};}
    P operator / (ld k) const {return {x / k, y / k};}
    P& operator += (const P& p) { x += p.x; y += p.y; return *this;}
    P& operator -= (const P& p) { x -= p.x; y -= p.y; return *this;}
};
P operator * (ld k, const P& p) { return p * k;}
ld dot(P a, P b) { return a.x * b.x + a.y * b.y;}// 点积
ld cross(P a, P b) { return a.x * b.y - a.y * b.x;}// 叉积
ld cross(P a, P b, P c) { return cross(b - a, c - a);}
ld len2(P a) { return dot(a,a);}
ld len(P a) { return sqrtl(len2(a));}
P unit(P a) {
    ld l = len(a);
    if (l <= EPS) return {0, 0};
    return a / l;
}
ld dist(P a, P b) { return len(a - b);}
P rot90(P a) { return {-a.y, a.x};}// 逆时针旋转 90°
ld deg2rad(ld deg) { return deg * PI / 180.0L;}
ld rad2deg(ld rad) { return rad * 180.0L / PI;}
P rot(P p, ld rad) {
    ld c = cosl(rad),s = sinl(rad);
    return { p.x * c - p.y * s,p.x * s + p.y * c};
}
ld angle(P a,P b){ return atan2l(fabsl(cross(a,b)),dot(a,b));}//向量夹角
P rotAt(P p, P center, ld rad) { return center + rot(p - center, rad);}//点绕点旋转
int ori(P a,P b,P c){ return sgn(cross(b-a,c-a));}
```

Listing 95: 三维点 (Point 3D)

```
struct P3 {
    ld x = 0, y = 0, z = 0;
    P3() = default;
    P3(ld x, ld y, ld z): x(x), y(y), z(z) {}
    P3 operator + (const P3& p) const { return { x + p.x, y + p.y,z + p.z};}
    P3 operator - (const P3& p) const { return { x - p.x, y - p.y,z - p.z};}
    P3 operator * (ld k) const { return { x * k, y * k,z * k};}
    P3 operator / (ld k) const { return { x / k, y / k, z / k};}
    P3& operator += (const P3& p) {
        x += p.x;
        y += p.y;
        z += p.z;
        return *this;
    }
};
P3 operator * (ld k, const P3& p) { return p * k; }
// -------------------- 3D 基础运算 --------------------
ld dot(P3 a, P3 b) { return a.x * b.x + a.y * b.y + a.z * b.z;}
// 3D 叉积返回向量
P3 cross(P3 a, P3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}
ld len2(P3 a) { return dot(a, a);}
ld len(P3 a) { return sqrtl(len2(a));}
P3 unit(P3 a) {
    ld l = len(a);
    if (l <= EPS) return {0, 0, 0};
    return a / l;
}
ld dist(P3 a, P3 b) { return len(a - b);}
P3 rotAxis(P3 v, P3 axis, ld rad) {
    ld l = len(axis);
    if (l <= EPS) return v;
    axis = axis / l;
    ld c = cosl(rad);
    ld s = sinl(rad);
    return v * c + cross(axis, v) * s + axis * dot(axis, v) * (1 - c);
}
```

8.3 直线

Listing 96: 直线常用操作

```
struct L {
    P p,v;  // 直线上一点,方向向量
    L() = default;
};//直线,注意此为点+方向向量表示
L line(P a,P b){
    return {a,b-a};
}
L perpL(P p,const L& l) { return {p,rot90(l.v)};}//过一点做已知直线垂线
bool valid(const L& l) {
    return len(l.v) > EPS;
}
bool parallel(const L& a, const L& b) {
    if (!valid(a) || !valid(b)) return false;
    ld scale = len(a.v) * len(b.v);
    return fabsl(cross(a.v, b.v)) <= EPS * scale;
}
bool perpendicular(const L& a, const L& b) {
    if (!valid(a) || !valid(b)) return false;
    ld scale = len(a.v) * len(b.v);
    return fabsl(dot(a.v, b.v)) <= EPS * scale;
}
bool interLL(const L& a, const L& b, P& res) {
```

```
    if (!valid(a) || !valid(b)) return false;
    ld d = cross(a.v, b.v);
    ld scale = len(a.v) * len(b.v);
    if (fabsl(d) <= EPS * scale) return false;
    ld t = cross(b.p - a.p, b.v) / d;
    res = a.p + a.v * t;
    return true;
}// false: 非法直线 / 平行 / 重合
bool onL(P p, const L& l) {
    if (!valid(l))
        return dist(p, l.p) <= EPS;

    // |cross| / |v| = 点到直线距离
    return fabsl(cross(l.v, p - l.p)) <= EPS *
len(l.v);
}
P proj(P p, const L& l) {
    ld vv = len2(l.v);
    if (vv <= EPS * EPS) return l.p;
    ld t = dot(p - l.p, l.v) / vv;
    return l.p + l.v * t;
}//点到直线垂足
P foot(P p, const L& l) {
    return proj(p, l);
}
P reflect(P p, const L& l) {
    P h = proj(p, l);
    return h * 2 - p;
}//点关于直线对称点
ld dPL(P p,const L& l) {
    ld d = len(l.v);
    if (d <= EPS) return dist(p, l.p);
    return fabsl( cross(l.v, p - l.p)) / d;
}//点到直线距离
P closeS(P p,P a,P b) {
    P v = b - a;
    ld vv = len2(v);
    if (vv <= EPS * EPS) return a;
    ld t = dot(p - a, v) / vv;
    t = max((ld)0, min((ld)1, t));
    return a + v * t;
}//点到线段的最近点
ld dPS(P p,P a,P b) {
    return dist(p,closeS(p, a, b));
}//点到线段距离
```

8.4 线段

Listing 97: 线段常用操作

```
bool onS(P p, P a, P b) {
    if (dist(a, b) <= EPS) return dist(p, a) <=
EPS;
    if (ori(a, b, p) != 0) return false;
    return p.x >= min(a.x, b.x) - EPS &&
           p.x <= max(a.x, b.x) + EPS &&
           p.y >= min(a.y, b.y) - EPS &&
           p.y <= max(a.y, b.y) + EPS;
}// 点在线段上，包含端点
bool properSS(P a, P b, P c, P d) {
    int s1 = ori(a, b, c);
    int s2 = ori(a, b, d);
    int s3 = ori(c, d, a);
    int s4 = ori(c, d, b);
    return s1 * s2 < 0 && s3 * s4 < 0;
}// 两线段严格相交：交点都不在端点
bool interSS(P a, P b, P c, P d) {
    if (properSS(a, b, c, d)) return true;
    if (onS(a, c, d)) return true;
    if (onS(b, c, d)) return true;
    if (onS(c, a, b)) return true;
    if (onS(d, a, b)) return true;
    return false;
}// 两线段是否相交：包含端点接触和共线重叠
ld dSS(P a, P b, P c, P d) {
    if (interSS(a, b, c, d)) return 0;
    return min({
        dPS(a, c, d),
        dPS(b, c, d),
        dPS(c, a, b),
        dPS(d, a, b)
    });
}
```

8.5 三角形

Listing 98: 三角形五心与面积

```
// incenter: 要求三角形非退化
// circumcenter: 要求三点不共线
// orthocenter: 要求三点不共线
P centroid(P a, P b, P c) { return (a + b + c) /
3;}// 三角形重心
P incenter(P a, P b, P c) {
    ld A = dist(b, c); // a 的对边
    ld B = dist(c, a); // b 的对边
    ld C = dist(a, b); // c 的对边
    return (a * A + b * B + c * C)/ (A + B + C);
}// 三角形内心
P circumcenter(P a, P b, P c) {
    P u = b - a, v = c - a;
    ld d = 2 * cross(u, v);
    ld u2 = len2(u);
    ld v2 = len2(v);
    P q {(v.y * u2 - u.y * v2) / d,(u.x * v2 - v.x
* u2) / d};
    return a + q;
}// 三角形外心
P orthocenter(P a, P b, P c) {
    P o = circumcenter(a, b, c);
    return a + b + c - o * 2;
}// 三角形垂心
ld area2(P a, P b, P c) {
    return fabsl(cross(a, b, c));
}
ld area(P a, P b, P c) {
```

```
    return area2(a, b, c) / 2;
}
// -1 外部, 0 边界, 1 内部
int inTri(P p, P a, P b, P c) {
    if (onS(p, a, b) ||onS(p, b, c) ||onS(p, c, a)) return 0;
    int s1 = ori(a, b, p);
    int s2 = ori(b, c, p);
    int s3 = ori(c, a, p);
    if ((s1 > 0 && s2 > 0 && s3 > 0) || (s1 < 0 && s2 < 0 && s3 < 0)) return 1;
    return -1;
}
```

8.6 圆

Listing 99: 圆常用操作与最小覆盖圆

```
using VP=vector<P>;
struct Circle {
    P o;
    ld r = -1;    // r < 0 表示无效圆
};
bool inside(const Circle& c, P p) {
    if (c.r < 0) return false;
    return dist(c.o, p) <= c.r + EPS;
}// 点是否在圆内 / 圆周
Circle diameter(P a, P b) {
    P o = (a + b) / 2;
    return { o,dist(a, b) / 2};
}// 两点作为直径
Circle circle3(P a, P b, P c) {
    P u = b - a,v = c - a;
    ld cr = cross(u, v);
    ld mx = max({ dist(a, b),dist(a, c),dist(b, c) });// 最大边长
    if (mx <= EPS) return {{0, 0}, -1};
    // 2S / max_edge 本质上是对应高度
    if (fabsl(cr) <= EPS * mx) return {{0, 0}, -1};
// 高度 <= EPS, 视为退化
    P o = circumcenter(a, b, c);
    return {o, dist(o, a)};
}
ld dPC(P p,const Circle& c){
    return fabsl(dist(p,c.o)-c.r);
}
VP tangents(P p,const Circle& c){
    P v=p-c.o;
    ld d2=len2(v),r2=c.r*c.r;
    if(d2<r2-EPS) return {};
    if(fabsl(d2-r2)<=EPS) return {p};
    P q=c.o+v*(r2/d2);
    P h=rot90(v)*(c.r*sqrtl(d2-r2)/d2);
    return {q+h,q-h};
}//点到圆的两个切点

Circle mec_two_points(const VP& pts,int lim,P p,P q) {
    Circle base = diameter(p, q);
    Circle left{{0, 0}, -1};
    Circle right{{0, 0}, -1};
    P pq = q - p;
    for (int i = 0; i < lim; ++i) {
        P r = pts[i];
        if (inside(base, r)) continue;
        ld side = cross(pq, r - p);
        Circle c = circle3(p, q, r);
        if (c.r < 0) continue;
        ld center_side = cross(pq, c.o - p);
        if (side > 0) {
            if (left.r < 0 ||center_side >cross(pq, left.o - p))  left = c;
        } else if (side < 0) {
            if (right.r < 0 ||center_side <cross(pq, right.o - p)) right = c;
        }
    }
    if (left.r < 0 && right.r < 0) return base;
    if (left.r < 0) return right;
    if (right.r < 0) return left;
    return left.r <= right.r? left: right;
}
// 最小覆盖圆
// 返回圆心 + 半径
// 随机增量
// 期望 O(n)
Circle mec(VP pts) {
    if (pts.empty()) return {{0, 0}, 0};
    static mt19937_64 rng( chrono::steady_clock::now().time_since_epoch().count());
    shuffle(pts.begin(), pts.end(), rng);
    Circle c{{0, 0}, -1};
    for (int i = 0; i < (int)pts.size(); ++i) {
        if (inside(c, pts[i])) continue;
        c = {pts[i], 0};
        for (int j = 0; j < i; ++j) {
            if (inside(c, pts[j])) continue;
            c = mec_two_points(pts, j, pts[i], pts[j]);
        }
    }
    return c;
}
// 返回 0 / 1 / 2 个交点
VP interLC( const L& l,const Circle& c) {
    VP res;
    if (!valid(l) || c.r < 0) return res;
    P h = proj(c.o, l);
    ld d = dist(h, c.o);
    if (d > c.r + EPS) return res;
    if (fabsl(d - c.r) <= EPS) {
        res.push_back(h);
        return res;
    }
    ld len = sqrtl(max((ld)0, c.r * c.r - d * d));
    P e = unit(l.v);
```

```
    res.push_back(h + e * len);
    res.push_back(h - e * len);
    return res;
}
int interCC(const Circle& a,const Circle& b,VP& res) {
    res.clear();
    if (a.r < 0 || b.r < 0) return 0;
    ld d = dist(a.o, b.o);// 同心
    if (d <= EPS) {
        if (fabsl(a.r - b.r) <= EPS) return -1; // 重合
        return 0;
    }
    ld sum = a.r + b.r;
    ld dif = fabsl(a.r - b.r);
    if (d > sum + EPS) return 0; // 相离
    if (d < dif - EPS) return 0; // 内含
    P e = (b.o - a.o) / d;
    ld x = (a.r * a.r - b.r * b.r + d * d)/ (2 * d);
    P base = a.o + e * x;
    ld h2 = a.r * a.r - x * x;
    ld h = sqrtl(max((ld)0, h2));
    if (h <= EPS) {
        res.push_back(base);
        return 1;
    }
    P off = rot90(e) * h;
    res.push_back(base + off);
    res.push_back(base - off);
    return 2;
}
```

8.7 多边形

Listing 100: 多边形面积/重心/点在多边形内

```
using Poly=vector<P>;
P centroid(const Poly& p) {
    int n = (int)p.size();
    ld s = 0,sx = 0,sy = 0;
    for (int i = 0; i < n; ++i) {
        const P& a = p[i];
        const P& b = p[(i + 1) % n];
        ld c = cross(a, b);
        s += c;
        sx += (a.x + b.x) * c;
        sy += (a.y + b.y) * c;
    }
    return {sx / (3 * s),sy / (3 * s)};
}//重心, requires area2(p) != 0
ld area2(const Poly& p) {
    ld s = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; i++) s += cross(p[i], p[(i + 1) % n]);
    return s;
}// 有向面积 * 2,CCW > 0, CW < 0
ld area(const Poly& p) {
    return fabsl(area2(p)) / 2;
}
ld peri(const Poly& p) {
    ld ans = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; ++i) ans += dist(p[i], p[(i + 1) % n]);
    return ans;
}
enum PointPolygonRelation {
    POLY_OUT = -1,
    POLY_ON  = 0,
    POLY_IN  = 1
};
int inPoly(const Poly& poly, P p) {
    int n = (int)poly.size();
    int winding = 0;
    for (int i = 0; i < n; ++i) {
        P a = poly[i];
        P b = poly[(i + 1) % n];
        if (onS(p, a, b)) return POLY_ON;
        int s = ori(a, b, p);
        if (a.y <= p.y) {
            if (b.y > p.y && s > 0) ++winding;
        } else {
            if (b.y <= p.y && s < 0) --winding;
        }
    }
    return winding == 0 ? POLY_OUT : POLY_IN;
}// 简单多边形, CW / CCW 均可
```

8.8 球

Listing 101: 三维球与三维最小覆盖球

```
struct Sphere {
    P3 o;
    ld r = -1;
};
bool inside(const Sphere& s, P3 p) {
    if (s.r < 0) return false;
    return dist(s.o, p) <= s.r + EPS;
}
Sphere sphere1(P3 a) { return {a, 0};}// 一个点确定半径 0 球
Sphere sphere2(P3 a, P3 b) {
    P3 o = (a + b) / 2;
    return {o,dist(a, b) / 2};
}// 两点确定直径球
// 三个非共线点
// 返回它们所在平面中的外接圆
// 把这个圆视为 3D 球的截面
// 这是经过三点的最小半径球心
Sphere sphere3(P3 a, P3 b, P3 c) {
    P3 u = b - a,v = c - a,w = cross(u, v);
```

```
    ld mx = max({ dist(a, b), dist(a, c), dist(b, c)});
    if (mx <= EPS) return {{0, 0, 0}, -1};
    // |u×v| / max_edge 是长度量
    if (len(w) <= EPS * mx) return {{0, 0, 0}, -1};
    ld w2 = len2(w),u2 = len2(u),v2 = len2(v);
    P3 q = (cross(v, w) * u2 + cross(w, u) * v2)/(2 * w2);
    P3 o = a + q;
    return {o, dist(o, a)};
}
Sphere sphere4(P3 a, P3 b, P3 c, P3 d) {
    P3 u = b - a,v = c - a,w = d - a;
    P3 vxw = cross(v, w);
    ld area = len(vxw);
    ld base_scale = max({len(v),len(w),len(v - w)});
    // v,w 基本共线
    if (base_scale <= EPS ||area <= EPS * base_scale) return {{0, 0, 0}, -1};
    ld det = dot(u, vxw);
    // |det| / |v×w| = u 到 vw 所在平面的距离
    if (fabsl(det) <= EPS * area) return {{0, 0, 0}, -1};
    P3 q = (cross(v, w) * len2(u) + cross(w, u) * len2(v) + cross(u, v) * len2(w)) / (2 * det);
    P3 o = a + q;
    return {o, dist(o, a)};
}// 四个非共面点唯一确定外接球
// R <= 4
// 暴力枚举哪个子集真正决定最小球
// 这样可以处理:
// - 重复点
// - 三点共线
// - 四点共面
// - 钝角三角形实际上由两个点决定
Sphere trivial_ball(const vector<P3>& R) {
    int m = (int)R.size();
    if (m == 0) return {{0, 0, 0}, -1};
    Sphere best{{0, 0, 0},INF};
    for (int mask = 1;mask < (1 << m);++mask) {
        vector<P3> q;
        for (int i = 0; i < m; ++i) {
            if (mask >> i & 1) q.push_back(R[i]);
        }
        Sphere s{ {0, 0, 0},-1};
        if (q.size() == 1) s = sphere1(q[0]);
        else if (q.size() == 2) s = sphere2(q[0],q[1]);
        else if (q.size() == 3) s = sphere3(q[0],q[1],q[2]);
        else if (q.size() == 4) s = sphere4(q[0],q[1],q[2],q[3]);
        if (s.r < 0) continue;
        bool ok = true;
        for (P3 p : R) {
            if (!inside(s, p)) {
                ok = false;
                break;
            }
        }
        if (ok && s.r < best.r) best = s;
    }
    return best;
}
// WARNING:
// Welzl recursion depth = O(n).
// n 很大（例如 1e6 级）有栈溢出风险。
Sphere welzl( vector<P3>& pts,int n,vector<P3>& R)
{
    // 3D 最多 4 个边界点决定球
    if ( n == 0 ||(int)R.size() == 4 ) { return trivial_ball(R);}
    P3 p = pts[n - 1];
    // 先假设 p 不需要成为边界点
    Sphere s = welzl(pts,n - 1,R);
    if (inside(s, p)) return s;
    // p 在旧球外 => p 必须成为新最小球边界点
    R.push_back(p);
    Sphere ans = welzl(pts,n - 1,R);
    R.pop_back();
    return ans;
}
// 三维最小覆盖球
// 固定维度下期望 O(n)
// 注意 Welzl 是递归实现，递归深度 O(n)
Sphere mes( vector<P3> pts) {
    if (pts.empty()) return {{0, 0, 0}, 0};
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(pts.begin(),pts.end(),rng);
    vector<P3> R; R.reserve(4);
    return welzl(pts,(int)pts.size(),R);
}
```

8.9 极角排序

Listing 102: 极角排序求夹角最近两向量

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
typedef long double ld;
const ll INF=1e18;
const ld pi=acos(-1);
struct Point{
    ll x,y;
    int id,idx;
    Point(ll x,ll y,int idx):x(x),y(y),idx(idx){
        if(x>=0&&y>=0) id=1;
        else if(x>=0&&y<0) id=2;
        else if(x<0&&y<0) id=3;
        else id=4;
    };
    Point(){};
```

```
    friend bool operator<(Point A,Point B){
        if(A.id!=B.id) return A.id<B.id;
        return A.x*B.y-A.y*B.x<0;
    }//极角排序
};
void solve(){
    int n;cin>>n;
    vector<Point>a(n+2);
    for(int i=1;i<=n;i++) {
        ll x,y;
        cin>>x>>y;
        a[i]=Point(x,y,i);
    }
    sort(a.begin()+1,a.begin()+1+n);
    ll x=0,y=0;
    auto calc=[&](Point &A,Point &B) ->ld {
        ld cross=A.x*B.y-A.y*B.x;
        ld dot=A.x*B.x+A.y*B.y;
        return fabsl(atan2l(cross,dot));
    };//较小精度损失的计算角度
    ++n;
    a[n]=a[1];//让 n和1比一次
    ld mn=INF;
    for(int i=1;i<n;i++){
        ld res=calc(a[i],a[i+1]);
        if(res<mn){
            x=a[i].idx,y=a[i+1].idx;
            mn=res;
        }
    }
    cout<<x<<' '<<y<<endl;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    solve();
    return 0;
}
```

8.10 凸包与旋转卡壳

单调链返回逆时针凸包并去掉边上的中间共线点；旋转卡壳返回最远点对的平方距离。叉积与平方距离使用 __int128，仍需保证结果不超出 128 位有符号整数范围。

Listing 103: 凸包与凸包直径

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
using i128 = __int128_t;

struct Point {
    i64 x, y;

    bool operator<(const Point &o) const {
        return tie(x, y) < tie(o.x, o.y);
    }

    bool operator==(const Point &o) const {
        return x == o.x && y == o.y;
    }
};

i128 cross(const Point &o, const Point &a, const Point &b) {
    i128 ax = (i128)a.x - o.x;
    i128 ay = (i128)a.y - o.y;
    i128 bx = (i128)b.x - o.x;
    i128 by = (i128)b.y - o.y;
    return ax * by - ay * bx;
}

i128 dis2(const Point &a, const Point &b) {
    i128 dx = (i128)a.x - b.x;
    i128 dy = (i128)a.y - b.y;
    return dx * dx + dy * dy;
}

// 逆时针凸包，去掉边上的中间共线点
vector<Point> hull(vector<Point> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1) {
        return p;
    }

    vector<Point> h(2 * p.size());
    int sz = 0;
    for (const Point &x : p) {
        while (sz >= 2 && cross(h[sz - 2], h[sz - 1], x) <= 0) {
            --sz;
        }
        h[sz++] = x;
    }
    int lo = sz;
    for (int i = (int)p.size() - 2; i >= 0; --i) {
        while (sz > lo && cross(h[sz - 2], h[sz - 1], p[i]) <= 0) {
            --sz;
        }
        h[sz++] = p[i];
    }
    h.resize(sz - 1);
    return h;
}

struct Dia {
    Point a{}, b{};
    i128 d2 = 0;
};

Dia diameter(vector<Point> p) {
    vector<Point> h = hull(move(p));
    int n = h.size();
```

```
    if (n <= 1) {
        return {h.empty() ? Point{} : h[0],
h.empty() ? Point{} : h[0], 0};
    }

    Dia ans{h[0], h[1], dis2(h[0], h[1])};
    auto upd = [&](int i, int j) {
        i128 d = dis2(h[i], h[j]);
        if (d > ans.d2) {
            ans = {h[i], h[j], d};
        }
    };
    if (n == 2) {
        return ans;
    }

    int j = 1;
    for (int i = 0; i < n; ++i) {
        int ni = (i + 1) % n;
        auto area = [&](int k) {
            return cross(h[i], h[ni], h[k]);
        };
        while (area((j + 1) % n) > area(j)) {
            j = (j + 1) % n;
        }
        upd(i, j);
        upd(ni, j);
        if (area((j + 1) % n) == area(j)) {
            upd(i, (j + 1) % n);
            upd(ni, (j + 1) % n);
        }
    }
    return ans;
}
```

9 Other Trick

9.1 多路归并 (含二分解)

题面简述

给定两个长度为 N 的单调不降序列 A, B，从 A, B 中各取一个数相加可得到 N^2 个和，求这 N^2 个和中最小的 N 个。

第一步：小根堆初始化与去重策略

由于序列 A, B 均单调不降，最小的和必然是 $A_1 + B_1$（下标从 1 开始）。我们使用一个小根堆（优先队列）维护候选和，堆中元素为三元组 (sum, i, j)，表示 $A_i + B_j$ 的值及其对应下标。

为避免重复访问同一 (i, j) 对，采用如下策略：

- 初始将 $(A_1 + B_1, 1, 1)$ 入堆，并标记 $(1, 1)$ 为已访问。
- 每次取出堆顶 (sum, i, j) 加入答案后：
  1. 若 $j + 1 \le N$ 且 $(i, j + 1)$ 未访问，将 $(A_i + B_{j+1}, i, j + 1)$ 入堆并标记。
  2. 若 $j = 1$（当前是第 i 行首个元素）且 $i + 1 \le N$，将 $(A_{i+1} + B_j, i + 1, j)$ 入堆并标记。

该策略确保每个 (i, j) 仅入堆一次，避免重复计算。

第二步：提取前 N 个最小和

重复执行 N 次以下操作：

1. 取出堆顶元素，将其和加入答案数组。
2. 按上述策略将后续候选和入堆。

最终答案数组即为所求的 N 个最小和。

正确性说明

由于 A, B 单调不降，对任意 (i, j) 有 $A_i + B_j \le A_{i+1} + B_j$ 且 $A_i + B_j \le A_i + B_{j+1}$。小根堆保证每次取出当前最小候选和，去重策略确保无重复入堆，且不遗漏任何可能的最小和。

Listing 104: 堆

```cpp
#include<bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<ll, pair<int, int>> P;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int N;cin >> N;
    vector<ll> A(N), B(N);
    for (int i = 0; i < N; ++i) cin >> A[i];
    for (int i = 0; i < N; ++i) cin >> B[i];
    priority_queue<P, vector<P>, greater<P>> pq;
    pq.push({A[0] + B[0], {0, 0}});
    for (int k = 0; k < N; ++k) {
        P top = pq.top();
        pq.pop();
        ll sum = top.first;
        int i = top.second.first;
        int j = top.second.second;
        if (k > 0) cout << ' ';
        cout << sum;
        if (j + 1 < N) {
            pq.push({A[i] + B[j+1], {i, j+1}});
        }
        if (j == 0 && i + 1 < N) {
            pq.push({A[i+1] + B[j], {i+1, j}});
        }
    }
    cout << endl;
    return 0;
}
```

第一步：二分查找确定第 N 小的和的值 x

定义函数 $\text{calc}(mid)$，用于计算序列 A, B 中满足 $A_i + B_j \le mid$ 的数对 (i, j) 的数量。利用单调性，该函数可通过双指针法在 $O(N)$ 时间内实现：

- 初始化指针 $j = N - 1$（指向 B 的末尾）。
- 遍历 A 的每个元素 A_i：
  - 若 $A_i + B_j > mid$，则 j 左移（减小 j）。
  - 此时 B_0 到 B_j 与 A_i 的和均 $\le mid$，贡献 $j + 1$ 个计数。

二分查找的初始区间设为 $[l, r]$，其中 $l = A_0 + B_0$（最小和），$r = A_{N-1} + B_{N-1}$（最大和）。二分过程如下：

1. 取中点 $mid = l + \frac{r-l}{2}$。
2. 若 $\text{calc}(mid) \geq N$，说明第 N 小的和 $\leq mid$，令 $r = mid$。
3. 否则，说明第 N 小的和 $> mid$，令 $l = mid + 1$。
4. 当 $l = r$ 时，$x = l$ 即为第 N 小的和的值。

第二步：收集所有严格小于 x 的和

再次利用双指针法，遍历序列 A：

- 对每个 A_i，找到最大的 j 使得 $A_i + B_j < x$。
- 将 $A_i + B_0, A_i + B_1, \ldots, A_i + B_j$ 全部加入结果数组 res。

第三步：补充等于 x 的和至 N 个

计算还需补充的数量 $\text{need} = N - \text{res.size}()$。若 $\text{need} > 0$，再次遍历序列 A：

- 对每个 A_i，找到最大的 j 使得 $A_i + B_j \leq x$。
- 从 j 开始向左遍历，若 $A_i + B_k == x$，则将 x 加入 res，并减少 need，直到 $\text{need} = 0$。

第四步：排序并输出

对结果数组 res 进行排序，取前 N 个元素依次输出，即为所求的 N 个最小和。

正确性说明

二分查找确保找到的 x 是满足“和 $\leq x$ 的数对数量 $\geq N$”的最小值，因此 x 必为第 N 小的和。通过先收集所有小于 x 的和，再补充等于 x 的和，确保结果数组包含恰好 N 个最小和，排序后顺序正确。

复杂度分析

- **时间复杂度：**
  - 二分查找部分：共进行 $O(\log S)$ 次二分（S 为和的范围，约 2×10^9），每次 calc 函数耗时 $O(N)$，总耗时 $O(N \log S)$。
  - 收集与补充和部分：两次双指针遍历均为 $O(N)$，最后排序为 $O(N \log N)$。
  - 综上，总时间复杂度为：

$$O(N \log N + N \log S)$$

- **空间复杂度：**结果数组 res 需 $O(N)$ 空间，总空间复杂度为 $O(N)$。

Listing 105: 二分

```cpp
#include<bits/stdc++.h>
using namespace std;
long long calc(const vector<long long>& a, const vector<long long>& b, long long mid) {
    int n = a.size();
    long long cnt = 0;
    int j = n - 1;
    for (int i = 0; i < n; ++i) {
        while (j >= 0 && a[i] + b[j] > mid) j--;
        cnt += j + 1;
    }
    return cnt;
}
int main() {
    cin.tie(nullptr)->ios::sync_with_stdio(false);
    int n;cin >> n;
    vector<long long> a(n), b(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int i = 0; i < n; ++i) cin >> b[i];
    long long l = a[0] + b[0], r = a.back() + b.back();
    while (l < r) {
        long long mid = l + (r - l) / 2;
        if (calc(a, b, mid) >= n) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    long long x = l;
    vector<long long> res;
    res.reserve(n);
    int j = n - 1;
    for (int i = 0; i < n; ++i) {
        while (j >= 0 && a[i] + b[j] >= x) j--;
        for (int k = 0; k <= j; ++k) {
            res.push_back(a[i] + b[k]);
        }
    }
    int need = n - res.size();
    j = n - 1;
    for (int i = 0; i < n && need > 0; ++i) {
        while (j >= 0 && a[i] + b[j] > x) j--;
        int k = j;
        while (k >= 0 && a[i] + b[k] == x && need > 0) {
            res.push_back(x);
            need--;
```

```
            k--;
        }
    }
    sort(res.begin(), res.end());
    for (int i = 0; i < n; ++i) {
        cout << res[i] << " ";
    }
    return 0;
}
```

9.2 使用栈做递归防溢出

Listing 106: 原始代码

```
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
const ll INF=1e18;
const int N=1e7;
void solve(){
    int n;cin>>n;
    vector<ll>a(n+1);
    vector<vector<int>>G(n+1);
    vector<ll>f(n+1);
    ll sum=0;
    for(int i=1;i<=n;i++) cin>>a[i],sum+=a[i];
    for(int i=1;i<n;i++){
        int x,y;cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    vector<ll>dp(n+1);
    vector<ll>siz(n+1);
    auto dfs=[&](auto &&dfs,int u,int fa) ->void {
        dp[u]=INF;
        f[u]=a[u];
        siz[u]=1;
        for(int v:G[u]){
            if(v==fa) continue;
            dfs(dfs,v,u);
            f[u]+=f[v];
            siz[u]+=siz[v];
            dp[u]=min(dp[u],dp[v]);
        }
        dp[u]=min(dp[u],f[u]/siz[u]);
    };
    dfs(dfs,1,0);
    for(int i=1;i<=n;i++) cout<<dp[i]<<' ';
    cout<<endl;
}
int main(){
    cin.tie(0)->ios::sync_with_stdio(false);
    int size(256<<20);
    __asm__ ( "movq %0, %%
rsp\n"::"r"((char*)malloc(size)+size));
    int T=1;cin>>T;
    while(T--) solve();
    exit(0);
    return 0;
}
```

Listing 107: 栈递归版

```
struct Statu{
    int u,fa,op;//当前节点,父节点,操作方式
};
int x,y;
void solve() {
    int n; cin>>n;
    vector<ll>a(n+1),f(n+1),dp(n+1);
    vector<Statu>st;
    st.reserve(n);
    vector<int>siz(n+1);
    vector<vector<int>>G(n+1);
    for(int i=1;i<=n;i++) cin>>a[i];
    for(int i=1;i<n;i++){
        cin>>x>>y;
        G[x].push_back(y);
        G[y].push_back(x);
    }
    st.push_back({1,0,0});
    while(!st.empty()){
        Statu &cur=st.back();
        int u=cur.u,fa=cur.fa;
        if(!cur.op){
            dp[u]=INF;
            f[u]+=a[u];
            siz[u]++;
            cur.op=1;
            for(int v:G[u]){
                if(v==fa) continue;
                st.push_back({v,u,0});
            }
        }
        else{
            for(int v:G[u]){
                if(v==fa) continue;
                dp[u]=min(dp[u],dp[v]);
                siz[u]+=siz[v];
                f[u]+=f[v];
            }
            dp[u]=min(dp[u],f[u]/siz[u]);
            st.pop_back();
        }
    }
    for(int i=1;i<=n;i++) cout<<dp[i]<<' ';
    cout<<endl;
}
```

9.3 Log Trick

前缀 GCD ,OR , AND 等操作变化的级别是呈现对数级别的.

问题简述 (Hdu 5869)

给定一个包含 N 个正整数 $a_1, a_2, \ldots, a_{N-1}, a_N$ 的数组 a；定义 a 的子数组为 a_1 到 a_N 之间的连续区间。换句话说，对于 $1 \le i \le j \le N$ 而言，$a_i, a_{i+1}, \ldots, a_{j-1}, a_j$ 就是 a 的一个子数组。对于形如 (L, R) 的查询，请回答区间 $[L, R]$ 内所有子数组产生的不同 GCD 值的数量。

数据保证：

- $1 \le N, Q \le 100000$
- $1 \le a_i \le 1000000$

核心性质观察

对于数组中的每个位置 i（作为子数组的右端点），所有以 i 为右端点的子数组的 GCD 具有以下性质：

- 这些 GCD 的数量是 $O(\log a_i)$ 级别的。因为每次将新的 a_i 与前序 GCD 取新的 GCD 时，数值至少减半（或保持不变），因此不同 GCD 的数量随右端点递增呈对数级增长。
- 可以通过维护一个列表来记录这些 GCD 及其对应的最左左端点。例如，列表中的元素为 (g, l)，表示以 i 为右端点、GCD 为 g 的子数组的左端点起始位置为 l。

离线处理与查询排序

1. 将所有查询按右端点 R 从小到大排序，并记录每个查询的原始索引，以便最后按输入顺序输出答案。
2. 从左到右遍历数组，每次处理到位置 i 时，同步处理所有 $R = i$ 的查询。

维护以 i 为右端点的 GCD 列表

当遍历到位置 i 时，按以下步骤维护当前的 GCD 列表：

1. 初始化临时列表 cur，先加入 (a_i, i)（对应子数组 $[i, i]$，其 GCD 为 a_i）。
2. 取上一次处理 $i-1$ 时得到的列表 prev，对其中每个元素 (g, l)，计算新的 GCD 值 $g' = \gcd(g, a_i)$，并将 (g', l) 加入 cur。
3. 合并 cur 中相同的 GCD 值：对相同的 g'，仅保留最小的左端点 l（因为更小的 l 对应覆盖范围更广的子数组）。合并后的列表作为新的 prev，用于处理 $i+1$。

利用树状数组维护不同 GCD 的计数

为了快速查询区间 $[L, R]$ 内的不同 GCD 数量，我们使用树状数组维护标记信息：

- 定义数组 las[g]，表示 GCD 值 g 最后一次被标记的左端点位置，初始为 0。
- 对于当前 cur 列表中的每个元素 (g, l)：
  1. 若 las[g] $\neq 0$，说明之前在位置 las[g] 标记过 g，需在树状数组中对 las[g] 位置执行单点减 1 操作（取消旧标记）。
  2. 在树状数组中对位置 l 执行单点加 1 操作（标记新的最左位置）。
  3. 更新 las[g] $= l$。
- 此时，树状数组中区间 $[L, R]$ 的和即为查询 $[L, R]$ 的答案：每个 GCD 值仅被标记一次，且仅当标记位置 $l \in [L, R]$ 时被计数。

复杂度分析

- **时间复杂度：**
  - 维护 GCD 列表：每个元素 a_i 对应的列表长度为 $O(\log a_i)$，总耗时 $O(N \log A)$（A 为 a_i 的最大值，约 10^6）。
  - 树状数组操作：每次单点更新和区间求和均为 $O(\log N)$，总耗时 $O((N+Q)\log N)$。
  - 综上，总时间复杂度为：

$$O(N \log A + Q \log N)$$

```cpp
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
typedef long long ll;
struct BIT {
    vector<ll>a;
    int n;
    BIT(int n):n(n+1){ a.resize(n<<1);}
    void add(int x,ll k) {
        while (x<=n) {
            a[x]+=k;
            x+=x&-x;
        }
    }
    int qry(int x) {
        ll ans=0;
        while (x) {
            ans+=a[x];
            x&=x-1;
        }
        return ans;
    }
};
struct Query{
    int L, R, idx;
    bool operator<(const Query& other) const {
        return R < other.R;
    }
};
int n,q;
void solve(){
    vector<int>a(n+1);
    for (int i=1;i<=n;i++) cin>>a[i];
    vector<Query>qs(q);
    for (int i=0;i<q;i++) {
        cin>>qs[i].L>>qs[i].R;
        qs[i].idx=i;
    }
    sort(qs.begin(),qs.end());
    BIT S(n+1);
    vector<pair<int, int>> prev;
    map<int,int>las;
    vector<int>ans(q);
    int q_ptr=0;
    for (int i=1;i<=n;i++) {
        vector<pair<int,int>>cur;
        cur.emplace_back(a[i],i);
        for (auto& p:prev)
cur.emplace_back(gcd(p.first,a[i]),p.second);
        vector<pair<int,int>> merged;
        for (auto& p:cur) {
            if
(merged.empty()||merged.back().first!=p.first) {
                merged.push_back(p);
            }
        }
        for(auto& [g,l] :merged) {
            if (las.count(g)) S.add(las[g],-1);
            S.add(l,1);
            las[g]=l;
        }
        while(q_ptr<q&&qs[q_ptr].R==i) {
            int L=qs[q_ptr].L;
            int R=qs[q_ptr].R;
            int idx=qs[q_ptr].idx;
            ans[idx]=S.qry(R)-S.qry(L - 1);
            q_ptr++;
        }
        prev=move(merged);
    }
    for (int x:ans) cout<<x<<endl;
}
int main() {
    cin.tie(nullptr)->ios::sync_with_stdio(false);
    while(cin>>n>>q) solve();
    return 0;
}
```

9.4 异或前缀模 4 小规律

设 $S(n)=1\oplus 2\oplus 3\oplus\cdots\oplus n$，则有：

$$S(n)=\begin{cases} n, & n \bmod 4=0\\ 1, & n \bmod 4=1\\ n+1, & n \bmod 4=2\\ 0, & n \bmod 4=3 \end{cases}$$

9.5 高阶差分下等差求通项

给定：

- 阶数 k
- 第 k 阶差分为常数 d（即“k 阶公差”为 d，同时“第 k 阶差分初值”也为 d）

则原数列的第 n 项为：

$$\boxed{a_n=d\cdot\binom{n+k-1}{k}}$$

9.6 曼哈顿距离与切比雪夫距离的相互转化

曼哈顿距离转切比雪夫距离

设原坐标系下两点坐标为 (x_1,y_1) 和 (x_2,y_2)。定义变换：

$$u=x+y,\qquad v=x-y$$

则在新坐标系 (u, v) 下，曼哈顿距离等于切比雪夫距离：

$$|x_1 - x_2| + |y_1 - y_2| = \max(|u_1 - u_2|,\ |v_1 - v_2|)$$

因此，处理最大曼哈顿距离问题时，只需在新坐标系下维护 u 和 v 的 $\min$ / $\max$ 值，即可 $\mathcal{O}(1)$ 得到结果。

切比雪夫距离转曼哈顿距离

设切比雪夫坐标系下两点坐标为 (u_1, v_1) 和 (u_2, v_2)。定义逆变换：

$$x = \frac{u+v}{2}, \qquad y = \frac{u-v}{2}$$

则原坐标系下的曼哈顿距离等于切比雪夫距离：

$$\max(|u_1 - u_2|,\ |v_1 - v_2|) = |x_1 - x_2| + |y_1 - y_2|$$

注意：若原坐标为整数，则 u 与 v 同奇偶，上式中的除法结果仍为整数或半整数，但距离公式依然成立。实际编程时可先全部乘以 2 避免浮点数。

9.7 判定可重集合是否相等

定义

可重集合（Multiset）：是指允许同一元素出现多次的集合。

问题描述

给定数组 a，多次询问：判断区间 $[L_1, R_1]$ 与 $[L_2, R_2]$ 所对应的可重集合是否相等。设每次询问的两个可重集合分别为 S 和 T。

朴素做法

直接的做法包括：

- 将两个区间内的元素排序逐一比较。
- 使用哈希表统计元素出现次数。

设 $|S| \le |T|$，单次询问时间复杂度为 $\mathcal{O}(|S| \cdot \log |S|)$。

随机化做法（随机赋权哈希）

定理

如果两个可重集合相等，那么两个集合的元素和相等。但仅靠元素和无法判定，反例：$S = \{2, 4\},\ T = \{3, 3\}$。

改进思路

为每一种元素 x 随机赋予一个权值 $w(x)$，定义集合的哈希值为：

$$H(S) = \sum_{x \in S} w(x)$$

判断两个集合是否相等，只需比较 $H(S)$ 和 $H(T)$。

正确性分析

- 若 $S = T$，则 $H(S) = H(T)$。
- 若 $S \neq T$，设 $cnt_S(x), cnt_T(x)$ 分别表示元素 x 在集合中的出现次数，则

  $$\sum \big(cnt_S(x) - cnt_T(x)\big) \cdot w(x) = 0$$

  这是一个关于随机变量 $w(x)$ 的线性组合。在 $w(x)$ 独立随机且值域足够大的情况下，上式成立的概率极低。

其他哈希形式

也可以定义 $H(S) = \prod_{x \in S} w(x)$，同样可用于判定可重集合相等（本质仍为随机映射）。

区间查询实现

- 不带修：使用前缀和维护，单次查询 $\mathcal{O}(1)$。
- 待修：使用树状数组/线段树维护区间和，单次查询 $\mathcal{O}(\log n)$。

扩展问题

给定数组 a，多次询问：判断区间 $[L_1, R_1]$ 中所有元素加上 k 后，是否与区间 $[L_2, R_2]$ 的可重集合相等。

设常数 B，定义哈希函数：

$$H(S) = \sum B^{a_i}$$

则有：

$$H([L, R]) = \sum_{i=L}^{R} B^{a_i}$$

区间整体加上 k 后：

$$\begin{aligned} H'([L, R]) &= \sum_{i=L}^{R} B^{a_i + k} = B^k \cdot \sum_{i=L}^{R} B^{a_i} \\ &= B^k \cdot H([L, R]) \end{aligned}$$

因此只需判断 $B^k \cdot H([L_1, R_1])$ 与 $H([L_2, R_2])$ 的关系即可。